Malicious Security Comes Free in SPDZ

Junru Li
Tsinghua University
and Shanghai Qi Zhi Institute
jr-li24@mails.tsinghua.edu.cn

Yifan Song
Tsinghua University
and Shanghai Qi Zhi Institute
yfsong@mail.tsinghua.edu.cn

Abstract

In this work, we study the concrete efficiency of SPDZ-type MPC protocols in the dishonest majority setting with maximum corruption, where $t = n - 1$ out of n parties can be corrupted. In the semi-honest setting, the state-of-the-art SPDZ protocol achieves an amortized communication cost of $4n$ field elements per multiplication gate, assuming a pseudorandom correlation generator (PCG) that prepares random Beaver triples silently in the offline phase. However, achieving security against malicious adversaries typically incurs a substantial overhead. Existing works either require communication of at least $10n$ field elements per gate, or additionally require an additively homomorphic encryption scheme.

In this work, we construct a maliciously secure SPDZ-type protocol with the same amortized communication complexity as the semi-honest variant, i.e., $4n$ field elements per multiplication gate, assuming a PCG for *tensor-product* correlations. We note that the tensor-product correlations come free as by-products from almost all existing constructions of PCGs for random Beaver triples. These by-products allow us to construct an efficient preprocessing phase and a recursive check protocol, which enables an online evaluation of the circuit without authenticating all the wire values. The correctness of the online computation can also be checked efficiently with a sublinear communication cost in the circuit size.

1 Introduction

Secure multiparty computation (MPC) [Yao82, GMW87, CCD88, BGW88, RB89] enables a set of n mutually distrustful parties to jointly evaluate a function on their private inputs, while revealing nothing beyond the prescribed output. Over the decades, the efficiency of MPC protocols has been greatly improved, turning them from purely theoretical primitives into practical cryptographic tools.

The security of an MPC protocol is typically characterized by two primary dimensions: the corruption threshold and the adversarial model. Protocols with an honest majority (corruption threshold $t < n/2$) can achieve information-theoretic security. A long line of works [BGW88, DN07, LN17, CGH+18, GS20, BGIN20, GLO+21, EGPS22] has focused on optimizing the communication efficiency in this setting. Notably, the state-of-the-art amortized communication cost is $4n$ field elements per multiplication gate, applicable to both semi-honest security and malicious security with abort [GLO+21].

In contrast, MPC protocols allowing for a dishonest majority (up to $t = n - 1$) address the worst-case corruption scenario. It is well-established that achieving unconditional security is impossible in this setting [BGW88]. To avoid using heavyweight public-key machineries, practical solutions for dishonest majority MPC often consider the preprocessing model. In this paradigm, parties generate correlated randomness in an offline phase, enabling a highly efficient, information-theoretic online phase. Specifically, the seminal SPDZ protocol [DPSZ12] introduced an online phase with communication complexity linear in the number of parties, assuming preprocessed random Beaver triples in the offline phase. To be more concrete, the online communication is $4|C|n$ field elements for computing a circuit of size $|C|$. Subsequently, numerous works [DKL+13, LOS14, KOS16, KPR18, BCS19, BNO19, BGIN21] have significantly improved upon the SPDZ paradigm in both the preprocessing and online phases. Leveraging recent advancements in pseudorandom correlation generators (PCGs) [BCGI18, BCG+19, BCG+20, WYKW21], an amortized

cost of $O(n)$ field elements per multiplication gate can be achieved across both phases even for malicious security [BGIN22, RS22].

A semi-honest secure SPDZ protocol can achieve an amortized communication cost of $4n$ field elements per multiplication gate simply from PCGs for Beaver triples. However, achieving malicious security typically incurs a large communication or computation overhead. In particular, the best-known malicious SPDZ protocols are constructed in two different ways:

- One direction is to prepare authenticated Beaver triples [RS22, GLM+24] during a circuit-independent preprocessing phase. However, current constructions of PCGs do not support the correlation of fully authenticated triples, so additional communication is required for the authentication steps. In particular, the amortized cost in [RS22] grows to $12n$ per multiplication gate. Although a recent work [GLM+24] implicitly reduces the cost to $10n$ field elements, it remains significantly less efficient than in the semi-honest setting.

- The other direction is to evaluate the circuit as the semi-honest SPDZ protocol, while using zero-knowledge on distributed data (D-ZK) to ensure the correctness of the computation [BGIN21, BGIN22]. This approach can achieve $4n$ elements per multiplication gate, matching the semi-honest SPDZ protocol, but requires a *circuit-dependent* preprocessing phase. Moreover, this approach additionally assumes an additively homomorphic public-key encryption (AHE) scheme [BGIN22]. Compared to the semi-honest SPDZ protocol, $O(|C|)$ homomorphic operations and $O(\sqrt{|C|})$ AHE encryptions are required, resulting in a significant computational overhead.

A natural question then arises:

"Can we lift the semi-honest SPDZ protocol to malicious security with the same amortized communication complexity per multiplication gate and comparable computation efficiency?"

1.1 Our Contributions

In this work, we give a positive answer to the above question. In particular, we present a maliciously secure SPDZ-type MPC protocol for arithmetic circuits over large fields ($\log|\mathbb{F}| = \Omega(\kappa)$) in the dishonest majority setting ($t = n - 1$) with the following properties:

- **Amortized Communication Cost.** Our construction achieves an amortized communication cost of $4n$ field elements per multiplication gate, matching the cost of the semi-honest variant. This represents a significant improvement over the state-of-the-art results [RS22, GLM+24] without additively homomorphic encryption schemes or circuit-dependent preprocessing.

- **Silent Preprocessing Generation from LPN.** For a parameter k, our construction requires $|C|/k$ groups of preprocessing data in the form of $(\{[\![a_i]\!]\}_{i=1}^k, \{[\![b_i]\!]\}_{j=1}^k, \{[a_i \cdot b_j]\}_{i,j=1}^k)$, where $[\![\cdot]\!]$ denotes an additive sharing *with* authentication, and $[\cdot]$ denotes an additive sharing *without* authentication. We refer to this type of preprocessing data as a length-k partially authenticated tensor product. Our preprocessing data is independent of the circuit topology and can be silently generated by almost all existing PCGs for random Beaver triples.

- **Information-Theoretic Online Phase.** Our online phase maintains information-theoretic security and only contains the semi-honest SPDZ online protocol plus an efficient verification on the correctness of evaluation, following the recursive check of [GS20] in the honest majority setting. The verification requires $O(|C|n \cdot \frac{\log k}{k})$ communication. When $k = w(1)$, the amortized communication complexity per gate remains $4n$ elements.

We further show that the amortized communication cost can be reduced to $2n$ per multiplication gate by employing the Turbospeedz online protocol [BNO19] in our framework, albeit at the cost of increased computational complexity. This may suit the case for small circuits or SIMD circuits.

Protocol	Circuit Dependency	Preprocessing from PCG	Comm. per Gate	Computation (beyond PCG)
[RS22] + [GLM+24]	No	$O(\|C\|)$ PA triples	$10n$	$O(n\|C\|)$ FO
[BGIN22]	Yes	$O(\|C\|)$ triples	$4n$	$O(n\|C\| \log \|C\|)$ FO + $O(\|C\|)$ AHE
Ours	No	$O(\|C\|/k)$ length-k PA tensor products	$4n + O(n \cdot \frac{\log k}{k})$	$O(nk\|C\|)$ FO
Ours	No	$O(1)$ length-$\|C\|$ PA tensor products	$2n$	$O(n\|C\|^2)$ FO

Table 1: Comparison of malicious SPDZ-type protocols. PA stands for partially authenticated. A partially authenticated Beaver triple is in the form of $([\![a]\!], [\![b]\!], [c])$, i.e., only the first two additive sharings are authenticated. FO stands for field operations. AHE stands for additively homomorphic public-key encryption operations.

We provide a comparison between our protocols and the malicious SPDZ-type protocols from [RS22, GLM+24, BGIN22] in Table 1.

To achieve this result, we revisit the current construction of malicious SPDZ-type protocols from authenticated Beaver triples [RS22, GLM+24]. Recall that an authenticated Beaver triple is in the form of $([\![a]\!], [\![b]\!], [\![c]\!])$ where a, b are random field elements and $c = a \cdot b$. In [RS22], the authors construct a PCG that can silently prepare *partially* authenticated Beaver triples in the form of $([\![a]\!], [\![b]\!], [c])$, i.e., only the first two sharings are authenticated. In addition, their PCG allows an adversary to insert a linear error on c by selecting δ_a, δ_b and causing $c = a \cdot b + \delta_a \cdot a + \delta_b \cdot b$. The main overhead compared to the semi-honest SPDZ protocol comes from (1) the verification of partially authenticated Beaver triples, and (2) the authentication of c sharings.

We observe that when preparing a group of k partially authenticated Beaver triples $\{([\![a_i]\!], [\![b_i]\!], [c_i])\}_{i=1}^k$ using the PCG in [RS22], parties can also *locally* compute shared cross product $[a_i \cdot b_j]$ for all $i, j \in \{1, \ldots, k\}$. We refer to this type of preprocessing data as a length-k partially authenticated tensor product. With these by-products from the PCG in [RS22], we construct an efficient verification that detects linear errors with sublinear communication in k, which solves the first issue.

To avoid the overhead of authenticating the c sharings, our idea is to directly evaluate the online phase using partially authenticated Beaver triples. We note that in this way, the inputs of all multiplication gates are authenticated during the computation. Thus, it suffices to verify that all authenticated wire values satisfy the correct correlations defined by the circuit. This in turn can be translated to a verification of an inner-product correlation. For this verification, we adapt the recursive check framework of [GS20], originally designed for the honest majority setting, which only incurs a sublinear communication in the circuit size. However, the communication efficiency of the technique in [GS20] crucially relies on the fact that in the honest majority setting, an inner-product operation over shared data can be computed at the same cost as a single multiplication operation, which does not hold in the dishonest majority setting. To overcome this obstacle, we again leverage the length-k partially authenticated tensor product and show that each block of k multiplication gates can be verified with sublinear communication in k. When $k = w(1)$, we obtain the desired result.

We refer the readers to Section 2 for a more detailed overview of these techniques.

2 Technical Overview

We give a high-level overview of the techniques used in this work. We use $[x]$ to denote an additive sharing of x over all n parties. We use $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$ to denote the inner product of two vectors $\boldsymbol{x}, \boldsymbol{y}$. We focus on maliciously

secure dishonest majority MPC ($t = n - 1$) for an arithmetic circuit C with $|C|$ multiplication gates over a large field $\mathbb{F}$ with $\log |\mathbb{F}| = \Omega(\kappa)$. The communication cost is measured by the number of field elements.

2.1 Review of SPDZ Protocol

We first briefly review the state-of-the-art malicious SPDZ-type protocol from [RS22, GLM+24].

The SPDZ Template. A (semi-honest) SPDZ-type MPC [DPSZ12] operates in two phases. In the *circuit-independent* preprocessing phase, parties together generates $|C|$ random Beaver triples in the form of $([a], [b], [c])$ where a, b are random field elements, and $c = a \cdot b$. From [RS22], random Beaver triples can be generated via a pseudorandom correlation generator (PCG) [BCG+19, BCG+20, WYKW21] with $o(1)$ amortized communication cost per triple.

Then in the online phase, parties evaluate the circuit gate-by-gate and compute an additive secret sharing for each wire in the circuit. For each addition gate with input sharings $([x], [y])$, parties can locally add up their shares to obtain $[z] := [x] + [y]$. For each multiplication gate, a random Beaver triple $([a], [b], [c])$ is consumed:

1. All parties locally compute $[x + a] := [x] + [a], [y + b] := [y] + [b]$ and send their shares to a single party P_{king}.
2. P_{king} reconstructs the secrets $x + a, y + b$ and sends the result to all parties.
3. Then all parties locally compute the output sharing $[z] := (x+a) \cdot (y+b) - (x+a) \cdot [b] - (y+b) \cdot [a] + [c]$.

As a result, the online phase only involves 2 public reconstructions per multiplication gate, which incurs $4n$ field elements of communication.

Towards Malicious Security. To achieve malicious security, [DKL+13] employs information-theoretic MACs to authenticate additively shared values. Specifically, all parties hold a random additive sharing $[\Delta]$, where Δ is a global MAC key. An authenticated sharing $[\![x]\!]$ is defined by three additive sharings $[\![x]\!] := ([x], [\Delta], [\Delta \cdot x])$. Later on, when parties open the secret x, they can reconstruct Δ and $\Delta \cdot x$ to verify the correctness of x. Intuitively, if an adversary wants honest parties to accept an incorrect $x' \neq x$, he needs to correctly guess the MAC key Δ, which happens with negligible probability.

Now in the preprocessing phase, all parties prepare $|C|$ authenticated Beaver triples in the form of $([\![a]\!], [\![b]\!], [\![c]\!])$. In the online phase, parties first run the semi-honest SPDZ protocol and then use MACs to verify the correctness of all public reconstruction. In particular, the verification can be done with sublinear communication by checking a random linear combination of all opened authenticated sharings. Thus, the amortized communication per multiplication gate remains to be $4n$ field elements.

Preparing Authenticated Triples. To maintain the amortized communication of $4n$ elements per multiplication gate, the only task is to prepare authenticated Beaver triples with sublinear communication in the offline phase. Unfortunately, it is not known how this can be acheived in the literature.

In [RS22], the authors construct a PCG which can efficiently generate *partially* authenticated Beaver triples in the form of $([\![a]\!], [\![b]\!], [c])$, i.e., only the first two additive sharings in each triple are authenticated. To obtain authenticated Beaver triples, the c sharing is authenticated by consuming a random authenticated sharing $[\![r]\!]$. In short, parties publicly reconstruct $[c + r]$ and compute $[\![c]\!] := (c + r) - [\![r]\!]$. The public reconstruction costs $2n$ elements of communication.

Making it even worse, the PCG in [RS22] does not ensure the correct correlation: an adversary may insert linear errors on c by choosing arbitrary constant $\delta_a, \delta_b, \delta_c$ and causing $c = a \cdot b + \delta_a \cdot a + \delta_b \cdot b + \delta_c$. As a result, after authenticating the c sharing, parties also need to verify the correctness of the authenticated Beaver triple, which can be done by using the sacrifice technique [DPSZ12]. This requires to consume another (possibly incorrect) authenticated Beaver triple with public reconstruction of 2 authenticated additive sharings. In total, the overall communication complexity becomes $12n$ elements per multiplication gate

in [RS22]. This is improved implicitly in [GLM+24] to $10n$ elements per multiplication gate with a better sacrifice technique [KOS16].

2.2 Generating Partially Authenticated Beaver Triples without Linear Errors

Our idea starts from the following key observation of the PCG protocol in [RS22]. Let k be the number of partially authenticated Beaver triples produced by the PCG. During the computation, all parties obtain not only $\{([\![a_i]\!], [\![b_i]\!], [c_i])\}_{i=1}^k$, but also $[a_i \cdot b_j]$ for all $i, j \in \{1, 2, \ldots, k\}$ as by-products. We refer to this type of correlated randomness $(\{[\![a_i]\!]\}_{i=1}^k, \{[\![b_j]\!]\}_{j=1}^k, \{[a_i \cdot b_j]\}_{i,j=1}^k)$ as a length-k partially authenticated tensor product.

Remark 1 *We note that length-k partially authenticated tensor products can be prepared by directly using almost all existing constructions of PCGs for random Beaver triples in the literature, not just restricting to the PCG in [RS22]. We refer the readers to Section 4.2 for more discussion. In particular, we construct the (n-party) PCG for length-k partially authenticated tensor products based on PCGs for n-party VOLE and (programmable) 2-party length-k tensor products, while 2-party PCG for tensor products can be replaced by (programmable) PCG for OLEs if only requiring Beaver triples. Nevertheless, in Section 4.3, we also construct a 2-party PCG for length-k tensor products by making black-box use of a (programmable) 2-party PCG for OLEs.*

Our idea is to make use of these additional additive sharings of $a_i \cdot b_j$ to simplify the verification of multiplication correlations. The main benefit of having tensor-product correlation is that for all a that is a linear combination of $\{a_1, \ldots, a_k\}$ and for all b that is a linear combination of $\{b_1, \ldots, b_k\}$, all parties can locally compute an additive secret sharing $[a \cdot b]$ from $\{[a_i \cdot b_j]\}_{i,j=1}^k$). To be more concrete, for all public vectors $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{F}^k$, let $[\![a]\!] = \sum_{i=1}^k u_i \cdot [\![a_i]\!]$ and $[\![b]\!] = \sum_{i=1}^k v_i \cdot [\![b_i]\!]$. All parties can locally compute

$$[a \cdot b] := \sum_{i=1}^{k} \sum_{j=1}^{k} u_i \cdot v_j \cdot [a_i \cdot b_j].$$

Now, to verify the correctness of the multiplication correlation for each (a_i, b_i, c_i), all parties do the following steps.

1. Generate random public vectors $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{F}^k$ and locally compute $[a \cdot b]$.
2. Convert $[a \cdot b]$ to $[\![a \cdot b]\!]$ by consuming a random authenticated sharing $[\![r]\!]$.
3. Robustly open $[\![a]\!], [\![b]\!], [\![a \cdot b]\!]$ and verify the multiplication correlation of the secrets.

Note that the communication complexity of the above verification is sublinear in k.

To see why this check is effective, recall that during the computation of the PCG in [RS22], an adversary may insert linear errors on c_i. In fact, it may insert linear errors for every $[a_i \cdot b_j]$ by choosing $\delta_{i,j}^{(a)}, \delta_{i,j}^{(b)}$ and causing the secret of $[a_i \cdot b_j]$ to be $a_i \cdot b_j + \delta_{i,j}^{(a)} \cdot a_i + \delta_{i,j}^{(b)} \cdot b_j$. (Here we omit the additive error on the secret since for an additive sharing, corrupted parties can insert additive errors at any time by changing their shares locally. Our verification only ensures that there is no linear error on c_i.) As a result, the secret of $[\![a \cdot b]\!]$ becomes

$$a \cdot b + \sum_{i=1}^{k} a_i \cdot \left(\sum_{j=1}^{k} u_i v_j \delta_{i,j}^{(a)} \right) + \sum_{j=1}^{k} b_j \cdot \left(\sum_{i=1}^{k} u_i v_j \delta_{i,j}^{(b)} \right) + \delta^{(c)}$$

for some additive error $\delta^{(c)}$ chosen by the adversary as well. Note that the coefficients of each a_i and b_j in the above equation are known to the adversary. The key observation is that, when $\boldsymbol{u}$ and $\boldsymbol{v}$ are randomly chosen, if at least one of $\delta_{i,j}^{(a)}$ or $\delta_{i,j}^{(b)}$ is not zero, with overwhelming probability, the secrets of $[\![a]\!], [\![b]\!], [\![a \cdot b]\!]$ do not satisfy the correct multiplication correlation.

As a result, if the verification passes, all parties hold a correct length-k tensor product (without linear errors), which also implies the correct multiplication correlation for each (a_i, b_i, c_i) (without linear errors).

Remark 2 *Unfortunately, the above verification is not useful for the protocol in [RS22]. Indeed, an additive secret sharing without authentication cannot prevent corrupted parties from inserting an additive error in the secret. When authenticating the c sharing to obtain an authenticated Beaver triple, the adversary still has a chance to insert an additive error on c, leading to incorrect multiplication correlation again. As a result, parties still need to verify the authenticated Beaver triples as in [RS22].*

However, we will show that such a verification is important in our construction. Jumping ahead, we will directly use partially authenticated Beaver triples in the online evaluation phase, without authenticating the c sharing. The above verification ensures that an adversary can only insert additive errors in the online phase, so that we can apply the recursive check in [GS20] to verify the online computation with sublinear communication.

2.3 Online Evaluation with Partially Authenticated Beaver Triples

To further avoid the cost of authenticating the c sharing in each Beaver triple, our idea is to directly use partially authenticated Beaver triple $(\llbracket a \rrbracket, \llbracket b \rrbracket, [c])$ in the online computation. We note that for each multiplication gate with input sharings $[x], [y]$, after all parties reconstruct $x + a$ and $y + b$, they can obtain *authenticated* sharings for the inputs $\llbracket x \rrbracket := (x + a) - \llbracket a \rrbracket$ and $\llbracket y \rrbracket := (y + b) - \llbracket b \rrbracket$ while the output $[z]$ is only additively shared. As a result, after the evaluation, all parties will obtain authenticated sharings for the input wires of each multiplication gate.

We note that for each multiplication gate, when reconstructing $x + a$ from $[x + a]$, the adversary may insert an additive error on the secret. In [DKL+13], such an attack can be detected relying on authenticated sharings for all wires in the circuit. I.e., parties hold $\llbracket x \rrbracket$ and $\llbracket a \rrbracket$ *before* evaluating this multiplication gate, and it is sufficient to verify that $x+a$ is the correct opening of $\llbracket x \rrbracket + \llbracket a \rrbracket$. In our case, however, since $[c]$ in the Beaver triple is not authenticated, parties may not hold $\llbracket x \rrbracket$. For example, x might be an output wire of a multiplication gate in the last layer, where parties only hold an additive sharing of x without authentication. As a result, we cannot verify the computation as [DKL+13].

To verify the computation, we will check that all authenticated wire values satisfy the correct correlations defined by the circuit. To be more concrete, suppose (x_i, y_i) are the input wires of the i-th multiplication gate which have been authenticated during the computation. Then x_i is computed as a suitable linear combination of the outputs of the previous $i - 1$ multiplication gates, and each such output z_j (with $j < i$) can be further represented as the multiplication of the two authenticated inputs $x_j \cdot y_j$. Thus, we obtain the following inner-product restriction for x_i:

$$x_i = \sum_{j=1}^{i-1} \alpha_{i,j} \cdot x_j \cdot y_j,$$

where $\{\alpha_{i,j}\}_{j=1}^{i-1}$ are constants determined by the circuit structure, and parties hold authenticated sharings for each variable in the equation. Thus, the verification of the computation boils down to check $O(|C|)$ inner-product tuples, one for every input wire of each multiplication gate.

In addition, thanks to the verification of the partially authenticated Beaver triples in the preprocessing phase, what an adversary can do in the online phase is restricted to inserting an additive error on each x_i. As a result, the difference $x_i - \sum_{j=1}^{i-1} \alpha_{i,j} \cdot (x_j \cdot y_j)$ (which should be 0 if the computation is correct) is known to the adversary.

Inner-Product Verification. To perform the verification efficiently, all parties check a random linear combination of all inner-product tuples, which becomes a single inner product of size $|C|$:

$$z = \sum_{i=1}^{|C|} \alpha_i \cdot (x_i \cdot y_i),$$

where α_i is a public coefficient determined by the circuit structure and the random challenge for the linear combination. In addition, this inner-product tuple is only subject to additive attack where the adversary knows $z - \sum_{i=1}^{|C|} \alpha_i \cdot (x_i \cdot y_i)$.

Our idea is to follow the recursive check in [GS20] for inner-product relation subject to additive attacks, which only incurs a sublinear communication in the size of input vectors. However, the original technique in [GS20] is designed for values shared by degree-t Shamir sharings in the honest majority setting. In particular, the communication efficiency of their verification heavily relies on the fact that computing inner-product of two shared vectors costs the same as one single multiplication, independent of the size of the input vectors. This is because with degree-t Shamir sharings and honest majority where $n = 2t + 1$, all parties can first locally compute a degree-$2t$ Shamir sharing of the inner-product result, and then apply the DN technique [DN07] to reduce the degree to t. This unfortunately does not work in the dishonest majority setting: To compute the inner-product of two vectors of authenticated sharings, parties need to compute each multiplication using a fresh Beaver triple and then sum up all multiplication results. This leads to a linear communication in the input vector size of the inner product.

To overcome this issue, we rely on the length-k partially authenticated tensor product again. For simplicity, let's assume that $k = |C|$, i.e., all random Beaver triples are generated from one instance of PCG. The key insight is that when applying the recursive check in [GS20], each inner product with input vectors $\tilde{\boldsymbol{x}}, \tilde{\boldsymbol{y}}$ satisfy that every element in $\tilde{\boldsymbol{x}}$ is a linear combination of $\{x_1, \ldots, x_{|C|}\}$, i.e., the first input wires of all multiplication gates, and every element in $\tilde{\boldsymbol{y}}$ is a linear combination of $\{y_1, \ldots, y_{|C|}\}$, i.e., the second input wires of all multiplication gates. Therefore, $\langle \tilde{\boldsymbol{x}}, \tilde{\boldsymbol{y}} \rangle$ can be computed by a linear combination of $\{x_i \cdot y_j\}_{i,j=1}^{|C|}$. Note that

- During the evaluation, parties have reconstructed $x_i + a_i$ and $y_i + b_i$ when evaluating the i-th multiplication gate using the Beaver triple $(\llbracket a_i \rrbracket, \llbracket b_i \rrbracket, [c_i])$;
- Parties also hold $[a_i \cdot b_j]$ for all $i, j \in \{1, \ldots, |C|\}$.

Thus, for every $i, j \in \{1, \ldots, |C|\}$, all parties can locally compute

$$[x_i \cdot y_j] = (x_i + a_i)(y_j + b_j) - (x_i + a_i)[b_j] - (y_j + b_j)[a_i] + [a_i \cdot b_j].$$

As a result, all parties can further locally compute an additive sharing $[\langle \tilde{\boldsymbol{x}}, \tilde{\boldsymbol{y}} \rangle]$ and then authenticate it via a random authenticated sharing $\llbracket r \rrbracket$. In this way, the cost of each inner product operation required by the recursive check in [GS20] is also independent of the vector size. We refer the readers to Section 7 for the formal treatment of the verification.

Optimizing Computation Complexity. The above verification assumes a length-$|C|$ partially authenticated tensor product. While efficient in communication, it leads to a quadratic *computation complexity* in the circuit size! We note that it is sufficient to use length-k partially authenticated tensor products, at the cost of an additional $O(|C| n \cdot \frac{\log k}{k})$ elements of communication. At a high level, we will divide the single inner-product tuple to be checked into $O(|C|/k)$ inner-product tuples of size k, such that each small inner-product tuple only involves Beaver triples from the same length-k partially authenticated tensor product. Then we may apply the same idea to verify each small inner-product tuple separately.

2.4 Protocol Summary and Further Optimization

We summarize our construction as follows. For simplicity, we assume the circuit C only contains public outputs. Let m_I/m_O be the number of input/output wires in C and $k = \omega(1)$.

1. **Preprocessing Phase.** All parties prepare $m_O + m_I + O(|C| \cdot \frac{\log k}{k})$ random authenticated sharings and $|C|/k$ length-k partially authenticated tensor products. Note that the latter implies $|C|$ partially authenticated Beaver triples.

For each length-k partially authenticated tensor product, all parties run the verification protocol in Section 2.2 to ensure that there are no linear errors. (The verification in fact would sacrifice one random Beaver triple. In our construction, parties will prepare length-$(k+1)$ partially authenticated tensor products instead.)

2. **Online Phase.** Parties first authenticate each input wire with a random authenticated sharing. Then all parties evaluate the circuit gate by gate using semi-honest SPDZ online protocol. Finally, the parties use m_O random authenticated sharings to authenticate the output wires. For each input wire value x of a multiplication gate, a masked value $x + a$ has been opened with $[\![a]\!]$ held by the parties. Thus, x has been authenticated by $[\![x]\!] = (x + a) - [\![a]\!]$.

3. **Verification Phase.** All parties determine $O(|C|)$ inner-product restrictions, one for each authenticated wire. Then all parties generate random coefficients and compute a random linear combination of all inner-product restrictions. As a result, the verification is reduced to check a single inner-product tuple of size $|C|$.

   All parties divide this single inner-product tuple into $|C|/k$ small inner-product tuples such that each inner-product tuple only involves Beaver triples from the same length-k partially authenticated tensor product. Then all parties follow [GS20] to verify each small inner-product with sublinear communication.

4. **Output Phase.** Upon successful verification, the parties robustly reconstruct the secret value for each output of C.

Turbospeedz Optimization. We note that utilizing a variant of length-$|C|$ partially authenticated tensor product, we can further reduce the communication complexity per gate to just $2n$ field elements, following the online protocol from Turbospeedz [BNO19]. We briefly outline this optimization here.

In this approach, parties will only compute an authenticated sharing for each input wire of the circuit and the *output wire* of each multiplication gate. In this way, when evaluating a multiplication gate, parties may compute authenticated input sharings $[\![x]\!], [\![y]\!]$. Then they will use the length-$|C|$ partially authenticated tensor product to compute an additive sharing of the output, which is then authenticated via a random authenticated sharing.

With more details, all parties will prepare preprocessing data in the form of $(\{[\![a_i]\!]\}_{i=1}^{|C|}, \{[a_i \cdot a_j]\}_{i,j=1}^{|C|})$. Effectively, this is a length-$|C|$ partially authenticated tensor product with $\boldsymbol{a} = \boldsymbol{b}$. Such a correlation can also be prepared from the PCG in [RS22]. Suppose all input wires of the circuit and output wires of all multiplication gates are denoted by $z_1, z_2, \ldots, z_{|C|}$. In the online phase, parties will compute an authenticated sharing $[\![z_i]\!]$ for every $i \in \{1, \ldots, |C|\}$ as follows.

1. All parties first authenticate each input wire z_i with $[\![a_i]\!]$. As a result, all parties hold $[\![z_i]\!] = (z_i + a_i) - [\![a_i]\!]$.

2. For each multiplication gate with input wires x_i, y_i, all parties first compute $[\![x_i]\!], [\![y_i]\!]$, which are linear combinations of $\{[\![z_j]\!]\}_{j<i}$ computed in previous layers.

3. Note that $x_i \cdot y_i$ can be computed as a linear combination of $\{z_{j_1} \cdot z_{j_2}\}_{j_1,j_2=1}^{|C|}$. Also note that all parties can locally compute $[z_{j_1} \cdot z_{j_2}]$ using public values $z_{j_1} + a_{j_1}, z_{j_2} + a_{j_2}$ and additive sharings $[a_{j_1}], [a_{j_2}], [a_{j_1} \cdot a_{j_2}]$. Thus, all parties locally compute an additive sharing $[x_i \cdot y_i]$ and authenticate it using $[\![z_i]\!]$. As a result, all parties hold $[\![z_i]\!] = (z_i + a_i) - [\![a_i]\!]$.

Following the same analysis, after the verification in the preprocessing phase, an adversary can only launch additive attacks in the online phase. This time, parties can compute authenticated sharings for all wires in the circuits. Thus, the verification is reduced to checking the multiplication relation for each multiplication gate. We may again convert it to the check of a single inner-product of size $|C|$, and then apply the recursive check in [GS20]. For further details, we refer the reader to Section 9.

3 Preliminaries

Notation. Let κ denote the secure parameter. $\mathbb{F}$ is a finite field with $\log|\mathbb{F}| = \Omega(\kappa)$. We consider MPC protocols run among n parties for an arithmetic circuit C over $\mathbb{F}$. We use $[x]$ to denote the additive sharing for a value x among the parties, and we use $[\![x]\!]$ to denote the authenticated sharing $[\![x]\!] = ([x], [\Delta], [\Delta \cdot x])$ among them. We use $\langle \boldsymbol{u}, \boldsymbol{v} \rangle, \boldsymbol{u} * \boldsymbol{v}, \boldsymbol{u} \otimes \boldsymbol{v}$ to denote the inner product, the coordinate-wise product, and the tensor product of two vectors $\boldsymbol{u}, \boldsymbol{v}$, respectively.

Security Model. We define the security of multiparty computation in the *real and ideal world paradigm* [Can00]. We consider a protocol Π to be secure if any adversary's view in its execution in the real world can also be simulated in the ideal world. We consider malicious security with abort against a static adversary $\mathcal{A}$ corrupting up to $t = n - 1$ parties. We denote the n parties by $P_1, \ldots, P_n$.

3.1 Standard Subprotocols

We give the standard functionalities for subprotocols of generating commitments and random public coins in this section. The functionality $\mathcal{F}_{\mathsf{Commit}}$ for commitments is given in Figure 1.

Functionality $\mathcal{F}_{\mathsf{Commit}}$

Commit: On input $(\mathsf{commit}, P_i, x, \mathsf{mid}_x)$ from P_i, where mid_x is the message index of x, the trusted party stores (P_i, x, mid_x) and sends $(\mathsf{committed}, P_i, \mathsf{mid}_x)$ to all the parties. In particular, if the commitment request has been called for the same message index before, the trusted party ignores the request.

Open: On input $(\mathsf{open}, P_i, \mathsf{mid}_x)$ from all the honest parties, the trusted party retrieves x and sends (x, mid_x) to all the parties.

Figure 1: Functionality for commitment.

The coin generation functionality $\mathcal{F}_{\mathsf{Coin}}$ is given in Figure 2. To realize $\mathcal{F}_{\mathsf{Coin}}$, each party commits a randomly chosen element in $\mathbb{F}^*$ and then opens it to all parties. Then, the product of the chosen values is taken to be the random coin. Finally, the parties can perform a cross-check to make sure that the received coins are the same.

Functionality $\mathcal{F}_{\mathsf{Coin}}$

1. On receiving $\mathsf{RandCoin}$ from all the honest parties, the trusted party samples $r \in \mathbb{F}^*$.
2. The trusted party sends r to Sim. If abort is received from Sim, the trusted party sends abort to all the honest parties and aborts the functionality. Otherwise, the trusted party sends r to all the parties.

Figure 2: Functionality for generating a common coin.

3.2 MAC Check on Opened Values

With random coins and commitments, the opened authenticated values are able to be checked. In particular, for an authenticated sharing $[\![x]\!]$ whose secret x needs to be reconstructed to all the parties, the parties call $\Pi_{\mathsf{Open}}([\![x]\!])$ (see Figure 3), where P_{king} can be any of the parties.

Protocol $\Pi_{\mathsf{Open}}([\![x]\!])$

1. All the parties send their shares of $[x]$ to P_{king}.
2. P_{king} reconstructs x and sends it to all the parties.

Figure 3: Protocol to open a SPDZ sharing.

When the secrets of some authenticated sharings are opened via Π_{Open}, they can check their correctness via a standard SPDZ MAC-check protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ [DKL+13] in the $\{\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}\}$-hybrid model (see Figure 4).

Protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$

Let all the parties agree on a pseudorandom generator $\mathsf{PRG} : \mathbb{F} \to \mathbb{F}^m$. The parties want to check the MACs on opened values $(A_1, \ldots, A_m)$ for sharings $([\![A_1]\!], \ldots, [\![A_m]\!])$.

1. The parties call $\mathcal{F}_{\mathsf{Coin}}$ to get a random element χ in $\mathbb{F}^*$.
2. The parties compute $A = \sum_{i=1}^{m} \chi^i \cdot A_i$ and $[\gamma] = \sum_{i=1}^{m} \chi^i \cdot [\Delta \cdot A_i]$.
3. The parties compute $[\sigma] = [\gamma] - [\Delta] \cdot A$. Each P_i calls $\mathcal{F}_{\mathsf{Commit}}$ with input $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$, where σ_i is P_i's share of $[\sigma]$.
4. The parties open their commitments via sending $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ for each party P_i and check that $\sum_{i=1}^{n} \sigma_i = 0$. If not, the parties abort the protocol.

Figure 4: Protocol for MAC checking.

4 Pseudorandom Correlation Generators

In this section, we introduce the pseudorandom correlation generators (PCGs) required for our preprocessing data.

4.1 Functionalities for Pseudorandom Correlations

We first give the functionalities for the correlations that need to be generated by pseudorandom correlation generators (PCGs).

Multiparty VOLE. Vector oblivious linear evaluation (VOLE) establishes a relation $\boldsymbol{w} = \boldsymbol{u} \cdot x + \boldsymbol{v}$ for $\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{v} \in \mathbb{F}^m$, where $x \in \mathbb{F}$ is a scalar held by one party. In the multiparty VOLE functionality $\mathcal{F}_{\mathsf{nVOLE}}$, each pair of parties (P_i, P_j) receives a random VOLE instance defined by $\boldsymbol{w}_j^{(i)} = \boldsymbol{u}^{(i)} \cdot x^{(j)} + \boldsymbol{v}_i^{(j)}$. The functionality guarantees consistency, ensuring that the same $\boldsymbol{u}^{(i)}$ and $x^{(j)}$ values are used in all instances involving P_i and P_j, respectively. We require the functionality to output a short seed to P_i that expands to $\boldsymbol{u}^{(i)}$ via an expansion function $\mathsf{Expand}_m : S \to \mathbb{F}^m$ (which is a specific pseudorandom generator) parametrized by a variable m. We give the functionality below in Figure 5.

Functionality $\mathcal{F}_{\mathsf{nVOLE}}$

Let $\{\mathsf{Expand}_m : S \to \mathbb{F}^m\}_{m \in \mathbb{Z}^+}$ be a collection of expansion functions with seed space S.

Init: On receiving $(\mathsf{Init}, \Delta^{(i)})$ from each party P_i, the trusted party stores $\Delta^{(i)} \in \mathbb{F}$ and ignores all the subsequent init requests from P_i.

Extend: On receiving $(\mathsf{Extend}, \mathcal{P}, m)$ from every P_i:

1. For each honest P_i, the trusted party randomly samples $\mathsf{seed}^{(i)} \in S$.
2. For each P_i, let $\boldsymbol{u}^{(i)} = \mathsf{Expand}_m(\mathsf{seed}^{(i)})$. The trusted party samples $(\boldsymbol{v}_i^{(j)})_{j \neq i} \in \mathbb{F}^k$, retrieves $\Delta^{(j)}$ and computes $\boldsymbol{w}_j^{(i)} = \boldsymbol{u}^{(i)} \cdot \Delta^{(j)} + \boldsymbol{v}_i^{(j)}$.
3. If P_j is corrupt, Sim can send a set I to the trusted party. If $\mathsf{seed}^{(i)} \in I$, the trusted party sends $\mathsf{success}$ to P_j and continues. Otherwise, the trusted party sends abort to both parties, outputs $\mathsf{seed}^{(i)}$ to P_j and aborts.
4. The trusted party outputs $((\mathsf{seed}^{(i)}, (\boldsymbol{w}_j^{(i)}, \boldsymbol{v}_j^{(i)})_{j \neq i})$ to P_i.

Corrupted Parties: A corrupted P_i can choose $\Delta^{(i)}$ and each $\mathsf{seed}^{(i)}$. It can also choose $\boldsymbol{w}_j^{(i)}$ (and then $\boldsymbol{v}_i^{(j)}$ is recomputed accordingly) and $\boldsymbol{v}_j^{(i)}$.

Global key query: If P_i is corrupted, the trusted party receives $(\mathsf{guess}, \boldsymbol{\Delta}')$ from Sim with $\boldsymbol{\Delta}' \in \mathbb{F}^n$. If $\boldsymbol{\Delta}' = \boldsymbol{\Delta}$ where $\boldsymbol{\Delta}' = (\Delta^{(1)}, \ldots, \Delta^{(n)})$, the trusted party sends $\mathsf{success}$ to P_i and ignores any subsequent global key query, otherwise, the trusted party sends $(\mathsf{abort}, \boldsymbol{\Delta})$ to P_i, abort to all the honest parties, and aborts the functionality.

Figure 5: Functionality of multiparty VOLE.

Programmable PCG for Tensor Products. We use a functionality $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ for generating tensor product correlations. This is a two-party functionality, which computes a batch of secret-shared products, i.e. random tuples $(u_i, x_j), (v_{i,j}, w_{i,j})$, where $w_{i,j} = u_i \cdot x_j + v_{i,j}$, over the field $\mathbb{F}$ for each pair of $(i, j) \in \{1, \ldots, k\}^2$. In other words, the two parties obtain an additive sharing of the tensor product of two vectors $\boldsymbol{u}, \boldsymbol{v}$ in $\mathbb{F}^k$. The *programmability* requirement is that, for any given instance of the functionality, the party obtaining $\boldsymbol{u}$ or $\boldsymbol{x}$ can program them from a chosen random seed. This allows the same $\boldsymbol{u}, \boldsymbol{x}$ to be used in different instances of $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$. The programmability is modeled with an expansion function $\mathsf{Expand} : S \to \mathbb{F}^m$, which generates $\ell = m/k$ blocks of tensor product correlations together. The functionality $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ is provided below in Figure 6.

Functionality $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$

Let $\mathsf{Expand}_{m,k} : S \to \mathbb{F}^m$ be an expansion function with seed space S and output length m, parameterized by a factor k. Let $\ell = m/k$. On receiving $s_a \in S$ from P_A and $s_b \in S$ from P_B:

1. The trusted party computes $\boldsymbol{u} = \mathsf{Expand}_{m,k}(s_a), \boldsymbol{x} = \mathsf{Expand}_{m,k}(s_b)$. Let $\boldsymbol{u} = (\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\ell), \boldsymbol{x} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_\ell) \in (\mathbb{F}^k)^\ell$. The trusted party then samples $\boldsymbol{v}_i \in \mathbb{F}^{k^2}$ for $i = 1, \ldots, \ell$.
2. For each $i = 1, \ldots, \ell$, the trusted party outputs $\boldsymbol{w}_i = \boldsymbol{u}_i \otimes \boldsymbol{x}_i + \boldsymbol{v}_i$ to P_A and $\boldsymbol{v}_i$ to P_B.

Corrupted Parties: If P_B is corrupted, each $\boldsymbol{v}_i$ is chosen by Sim. For a corrupt P_A, Sim can choose $\boldsymbol{w}_i$ (and then $\boldsymbol{v}$ is recomputed accordingly).

Figure 6: Functionality of programmable PCG for tensor product correlation.

4.2 Instantiation of the Functionalities via PCGs

In this section, we briefly discuss the instantiations for the PCGs realizing the functionalities $\mathcal{F}_{\mathsf{nVOLE}}, \mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ from various assumptions.

Instantiation from LPN. The most efficient way of realizing $\mathcal{F}_{\mathsf{nVOLE}}, \mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ is to use PCGs from variants of the learning parity with noise (LPN) assumption. The instantiation of $\mathcal{F}_{\mathsf{nVOLE}}$ is given by the PCG for n-party VOLE, which is given by [RS22]. It relies on the regular dual-LPN assumption defined below.

Definition 1 (d-Regular Vector). *A d-regular vector $\boldsymbol{e}$ is defined as a set of d vectors $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_d$ concatenated, wherein each $\boldsymbol{e}_i$ is a sparse vector with Hamming weight one.*

Definition 2 (Regular Dual-LPN Assumption). *Let $H \in \mathbb{F}^{\gamma \times \gamma'}$, and consider the following game $G_b(\kappa)$ with a PPT adversary $\mathcal{A}$, parameterised by a bit b and the security parameter κ:*

1. *Sample a random, d-regular vector $\boldsymbol{e} \in \mathbb{F}^{\gamma'}$.*
2. *If $b = 1$, let $\boldsymbol{y} = H \cdot \boldsymbol{e}$, otherwise sample $\boldsymbol{y} \leftarrow \mathbb{F}^\gamma$.*
3. *Send $\boldsymbol{y}$ to $\mathcal{A}$, which then outputs a bit b' (in case of abort, define the output of $\mathcal{A}$ to be $\perp$).*

The assumption states that $\mathcal{A}$ has a negligible advantage in distinguishing $G_0(\kappa)$ and $G_1(\kappa)$.

The expansion function used in $\mathcal{F}_{\mathsf{nVOLE}}, \mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ is defined as follows: Fix a dual-LPN matrix $H \in \mathbb{F}^{\gamma \times \gamma'}$ and consider the seed space $S \subset (\mathbb{F}^{\gamma'})^{m/\gamma}$ cotaining length-m/γ vectors of d-regular vectors in $\mathbb{F}^{\gamma'}$. The LPN-based expansion function $\mathsf{Expand}_m : S \to \mathbb{F}^m$ is defined by

$$\mathsf{Expand}_m(\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{m/\gamma}) = (H \cdot \boldsymbol{e}_1, \ldots, H \cdot \boldsymbol{e}_{m/\gamma}).$$

$\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ is realized using a PCG for tensor product correlations in [BCG+19, BCG+20][1]. The idea of such a PCG is that, to prepare the tensor product for two pseudorandom vectors $\boldsymbol{u}_i, \boldsymbol{x}_i$ from two parties, we can let them be expanded from two sparse vectors $\boldsymbol{e}, \boldsymbol{e}'$ using the expansion function (where $\boldsymbol{e}, \boldsymbol{e}'$ can be chosen by the two parties, providing the programmability). Then, we only need to secret-share $\boldsymbol{e} \otimes \boldsymbol{e}'$, which is also sparse, and then the secret sharing of $\boldsymbol{u}_i \otimes \boldsymbol{x}_i$ can be computed linearly. Due to the progress on function secret sharing (FSS) [GI14, BGI15], $\boldsymbol{e} \otimes \boldsymbol{e}'$ can be secretly shared via a sublinear share size in the vector length of $\boldsymbol{u}_i, \boldsymbol{x}_i$, resulting in a sublinear communication cost in realizing $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$.

We observe that the expansion function of this PCG is compatible with the expansion function of $\mathcal{F}_{\mathsf{nVOLE}}$ from [RS22] defined above, parameterized by $\gamma = k$. Furthermore, its seed generation algorithm is a random algorithm in the seed given by $\mathcal{F}_{\mathsf{nVOLE}}$. Thus, the PCG of [BCG+19, BCG+20] easily supports programmable inputs corresponding to Expand_m. The instantiations of both functionalities only require sublinear communication cost in m. We refer the readers to [BCG+19, BCG+20, RS22] for more details.

In [RS22], PCG for tensor product correlations is not required for generating the Beaver triples. The parties only need a PCG for n-party VOLE and a programmable PCG for oblivious linear evaluation correlations (OLEs), which replaces each $\boldsymbol{u}_i \otimes \boldsymbol{w}_i$ in $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ by $\boldsymbol{u}_i * \boldsymbol{w}_i$, to generate partially authenticated Beaver triples. Notably, the programmable PCG for OLEs necessarily relies on the PCG for tensor products if only assuming the regular dual-LPN assumption.

Instantiation from Ring-LPN. The PCG for tensor product correlations is not necessarily needed if assuming Ring-LPN [BCG+20]. Nevertheless, the idea of achieving the PCG is similar, and the PCG for OLE from Ring-LPN can naturally be generalized to any degree-2 correlations, as demonstrated in [BCG+20]. Thus, $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ can also be realized from Ring-LPN. Notably, the PCG for n-party VOLE can also be modified from Ring-LPN to fit the expansion function required in $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ [RS22], so our PCGs can also be instantiated from Ring-LPN.

More concretely, the Ring-LPN assumption states that given a polynomial a in a ring $R = \mathbb{F}[X]/F(X)$ for a degree-N polynomial $F(X)$, $a \cdot e + e'$ is computationally indistinguishable from a random element in R for random sparse polynomials e, e', i.e. e, e' have random sparse vectors as their coefficients. In the PCG construction for OLEs from Ring-LPN, $\boldsymbol{u}_i$ is expanded from two sparse polynomials (e, e') by computing the coefficients of $u_i = a \cdot e + e'$, and $\boldsymbol{x}_i$ is expanded in the same way from (f, f') with polynomial $x_i = a \cdot f + f'$. Crucially, $u_i \cdot x_i$ can be computed linearly from $(e, e') \otimes (f, f')$. Therefore, by sharing $(e, e') \otimes (f, f')$ using FSS, a secret sharing of $u_i \cdot x_i$ can be computed, thus providing an OLE over R. When $F(X)$ splits to a product of degree-1 polynomials, the OLE over R is equivalent to N OLEs over $\mathbb{F}$, providing an efficient way for generating OLEs.

Note that in the above approach, the FSS for $(e, e') \otimes (f, f')$ is still instantiated by sharing the tensor products of the coefficient vectors $(\boldsymbol{e}, \boldsymbol{e}'), (\boldsymbol{f}, \boldsymbol{f}')$ of $(e, e'), (f, f')$, and the tensor product $\boldsymbol{u}_i \otimes \boldsymbol{v}_i$ is linear to $(\boldsymbol{e}, \boldsymbol{e}') \otimes (\boldsymbol{f}, \boldsymbol{f}')$. Therefore, the secret sharing of the tensor product $\boldsymbol{u}_i \otimes \boldsymbol{x}_i$ can still be computed in the same way as the LPN-based approach. However, unlike the PCG for OLEs, the computational cost of the PCG from Ring-LPN is similar to that from the regular dual-LPN assumption. We refer the readers to [BCG+20] for more details.

PCG for Beaver triples can also be prepared from the quasi-abelian syndrome decoding (QA-SD) assumption [BCCD23, LLX+26], which can be regarded as a multivariate version of Ring-LPN. The construction structure is similar to the Ring-LPN-based approach, which can also be generalized to realize our functionalities.

[1] The correlations are called bilinear correlations in [BCG+19, BCG+20].

Instantiation from HSS. Another approach to generating PCG for Beaver triples is via homomorphic secret sharings (HSS) [BCG+19]. Note that the required correlations can be expressed as additive sharings of $F(r)$ for some function F on a random input r. The idea of using HSS to construct the PCGs is that we can use a pseudorandom generator PRG to generate the randomness r, and we only need to share the PRG seed via HSS. As long as the HSS is able to evaluate the function $F \circ \mathsf{PRG}(\cdot)$, the target correlations can be generated. The restriction of this approach is the function class supported by the HSS scheme, which often requires $F \circ \mathsf{PRG}(\cdot)$ to be of a low circuit depth. Since our target correlations are all of degree 2, which is the same as the required correlations for Beaver triples, such PCGs can naturally be generalized to support our target correlations.

4.3 Realizing $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ from Programmable PCG for OLEs.

Finally, we show that $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ can also be constructed from k^2 programmable OLE correlations used in [RS22] in a black-box way corresponding to expansion functions of the form

$$\mathsf{Expand}_m(\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{m/\gamma}) = (\mathsf{Expand}^{(\gamma)}(\boldsymbol{e}_1), \ldots, \mathsf{Expand}^{(\gamma)}(\boldsymbol{e}_{m/\gamma})).$$

Such a form of expansion function is general, as the pseudorandom shares generated by $\mathcal{F}_{\mathsf{nVOLE}}$ can always be generated block-by-block. We first give the programmable OLE functionality as follows in Figure 7.

Functionality $\mathcal{F}_{\mathsf{OLE}}^{\mathsf{prog}}$

Let $\mathsf{Expand}^{(\gamma)} : S_\gamma \to \mathbb{F}^\gamma$ be an expansion function with seed space S_γ and output length γ. On receiving $s_a \in S_\gamma$ from P_A and $s_b \in S_\gamma$ from P_B:

1. The trusted party computes $\boldsymbol{u} = \mathsf{Expand}^{(\gamma)}(s_a), \boldsymbol{x} = \mathsf{Expand}^{(\gamma)}(s_b)$ and samples $\boldsymbol{v} \in \mathbb{F}^\gamma$.
2. The trusted party outputs $\boldsymbol{w} = \boldsymbol{u} * \boldsymbol{x} + \boldsymbol{v}$ to P_A and $\boldsymbol{v}$ to P_B.

Corrupted Parties: If P_B is corrupt, $\boldsymbol{v}$ may be chosen by Sim. For a corrupt P_A, Sim can choose $\boldsymbol{w}$ (and then $\boldsymbol{v}$ is recomputed accordingly).

Figure 7: Functionality for programmable OLE.

Then, we construct Π_{tensor} to realize $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ in the $\mathcal{F}_{\mathsf{OLE}}^{\mathsf{prog}}$-hybrid model as follows. The expansion function for $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ is $\mathsf{Expand}_{m,k} : (S_\ell)^k \to \mathbb{F}^m$, where the $(i\ell + j)$-th bit $(i \in \{0, \ldots, k-1\}, j \in \{1, \ldots, \ell\})$ of $\mathsf{Expand}_{m,k}(\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{m/\gamma})$ is the $(i+1)$-th bit of $\mathsf{Expand}^{(\ell)}(\boldsymbol{e}_j)$.

Protocol Π_{tensor}

To achieve $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ for expansion function $\mathsf{Expand}_{m,k}$:

1. P_A parses $\boldsymbol{e}_{a,1}, \ldots, \boldsymbol{e}_{a,k}$ from his input seed s_a. Similarly, P_B parses $\boldsymbol{e}_{b,1}, \ldots, \boldsymbol{e}_{b,k}$ from his input seed s_b.
2. For each pair of $(i,j) \in \{1, \ldots, k\}^2$, P_A sends $\boldsymbol{e}_{a,i}$ and P_B sends $\boldsymbol{e}_{a,j}$ to $\mathcal{F}_{\mathsf{OLE}}^{\mathsf{prog}}$ corresponding to the expansion function $\mathsf{Expand}^{(\ell)}$. P_A receives $\boldsymbol{w}_{i,j}$, and P_B receives $\boldsymbol{v}_{i,j}$. Now, for each $j = 1, \ldots, \ell$, the parties hold an additive sharing of the tensor product of the vector containing the j-th bits of $\mathsf{Expand}^{(\ell)}(\boldsymbol{e}_{a,1}), \ldots, \mathsf{Expand}^{(\ell)}(\boldsymbol{e}_{a,k})$ and the vector containing the j-th bits of $\mathsf{Expand}^{(\ell)}(\boldsymbol{e}_{b,1}), \ldots, \mathsf{Expand}^{(\ell)}(\boldsymbol{e}_{b,k})$. This matches the correlation we need in $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$.
3. Finally, P_A obtains his output $\{\boldsymbol{w}_i\}_{i=1,\ldots,\ell}$ from $\{\boldsymbol{w}_{i,j}\}_{(i,j)\in\{1,\ldots,k\}^2}$, and P_B obtains his output $\{\boldsymbol{v}_i\}_{i=1,\ldots,\ell}$ from $\{\boldsymbol{v}_{i,j}\}_{(i,j)\in\{1,\ldots,k\}^2}$.

Figure 8: The protocol achieving $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ in the $\mathcal{F}_{\mathsf{OLE}}^{\mathsf{prog}}$-hybrid model.

5 Preprocessing Phase

In this section, we introduce the preprocessing phase of our protocol.

5.1 Preprocessing Functionality

We first give the preprocessing functionality $\mathcal{F}_{\mathsf{prep}}$, which is defined in Figure 9. It allows the parties to prepare the following sharings:

- **Random Values:** Authenticated sharings of the form $[\![r]\!]$, where r is a random element in $\mathbb{F}$.
- **Input Masks:** The mask sharings for each party P_i's inputs. Input mask sharings are of the form $([\![r]\!], [\Delta_i \cdot r])$, where Δ_i is the global MAC key chosen by P_i, i.e. P_i's share of $[\Delta]$, and r is a random element in $\mathbb{F}$.
- **Authenticated Triples:** Authenticated triples of the form $([\![a]\!], [\![b]\!], [\![c]\!])$ with $c = a \cdot b$. These authenticated sharings are only used for verification during the online phase.
- **Tensor Products:** Length-k partially authenticated tensor products of the form $\{([\![a_i]\!], [\![b_i]\!])\}_{i=1,\ldots,k}, \{[a_i \cdot b_j]\}_{i,j\in\{1,\ldots,k\}}$, where $\{a_i, b_i\}_{i=1,\ldots,k}$ are all random elements in $\mathbb{F}$.

Functionality $\mathcal{F}_{\mathsf{prep}}$

Init: On receiving $(\mathsf{Prep}, m_R, (I_1, \ldots, I_n), m_A, m_T)$ from all the honest parties:

1. The trusted party samples a random element in $\mathbb{F}$ as Δ.
2. The trusted party receives corrupted parties' shares of $[\Delta]$ from Sim and samples the honest parties' shares of $[\Delta]$ based on Δ and the corrupted parties' shares of $[\Delta]$.
3. The trusted party distributes $[\Delta]$ to the parties.

Random Values: Repeat the following m_R times:

1. The trusted party randomly samples $r \in \mathbb{F}$ and computes $\Delta \cdot r$.
2. The trusted party receives corrupted parties' shares of $[\![r]\!]$ from Sim and samples the honest parties' shares of $[\![r]\!]$ based on the secret and the corrupted parties' shares.
3. The trusted party distributes $[\![r]\!]$ to the parties.

Input Masks: For each $j = 1, \ldots, n$, repeat the following I_j times:

1. Run steps 1 and 2 from **Random Values** to create the sharing $[\![r]\!]$.
2. Let P_j's share of $[\Delta]$ be Δ_j. The trusted party computes $\Delta_j \cdot r$ and receives corrupted parties' shares of $[\Delta_j \cdot r]$ from Sim. Then, the trusted party samples honest parties' shares of $[\Delta_j \cdot r]$ based on the secret and corrupted parties' shares.
3. The trusted party distributes $[\![r]\!], [\Delta_j \cdot r]$ to the parties.

Authenticated Triples: Repeat the following m_A times:

1. The trusted party samples $a, b \in \mathbb{F}$ randomly and computes $c = a \cdot b$. Then the trusted party computes $\Delta \cdot a, \Delta \cdot b, \Delta \cdot c$.
2. The trusted party receives corrupted parties' shares of $[\![a]\!], [\![b]\!], [\![c]\!]$ from Sim and samples the honest parties' shares of $[\![a]\!], [\![b]\!], [\![c]\!]$ based on the secret and the corrupted parties' shares.
3. The trusted party distributes $[\![a]\!], [\![b]\!], [\![c]\!]$ to the parties.

Tensor Products: Repeat the following m_T times:

1. Run steps 1 and 2 from **Random Values** $2k$ times to create sharings $\{[\![a_i]\!], [\![b_i]\!]\}_{i=1,\ldots,k}$.
2. For each pair of $(i, j) \in \{1, \ldots, k\}^2$, the trusted party computes $c_{i,j} = a_i \cdot b_j$ and receives corrupted parties' shares of $[c_{i,j}]$ from Sim. Then, the trusted party samples the honest parties' shares of it based on the secret and the corrupted parties' shares.

3. The trusted party distributes $\{([\![a_i]\!], [\![b_i]\!])\}_{i=1,\dots,k}, \{[c_{i,j}]\}_{i,j\in\{1,\dots,k\}}$ to the parties.

Abort: If (abort, P) is received from Sim for an honest party P, the trusted party sends Δ to Sim and sends abort to P.

Figure 9: Functionality for the preprocessing phase.

5.2 Preprocessing Protocol

Now, we begin to introduce our preprocessing protocol. During the initialization, the parties choose their shares of $[\Delta]$. The random sharings and authenticated triples are prepared using the technique of [GLM+24]. For the tensor products, the parties run the PCGs to prepare them and then verify them to ensure they do not include linear errors. The protocol is given in Π_{prep} (Figure 10), and then we will give the subprotocols $\Pi_{\mathsf{rand}}, \Pi_{\mathsf{auth}}, \Pi_{\mathsf{tens}}$ for preparing random sharings, authenticated triples, and tensor products respectively.

Protocol Π_{prep}

Each party P_i samples a random element in $\mathbb{F}$ as his share Δ_i of $[\Delta]$. Then, P_i sends $(\mathsf{Init}, \Delta_i)$ to $\mathcal{F}_{\mathsf{nVOLE}}$. Then, the parties run Π_{rand}, Π_{mask}, Π_{auth}, and Π_{tens}.

Figure 10: Protocol for the preprocessing phase.

Preparing Random Authenticated Sharings. Each random authenticated sharing is obtained by conversion from a sharing $\langle\!\langle x \rangle\!\rangle$, which contains an additive sharing $[x] = (x^{(1)}, \dots, x^{(n)})$ and a pair-wise MAC $(M_j^{(i)}, K_i^{(j)})$ between each pair of parties (P_i, P_j), where P_j holds a random local key $K_i^{(j)}$ and P_i holds the MAC of $x^{(i)}$ defined by

$$M_j^{(i)} = K_i^{(j)} + \Delta_j \cdot x^{(i)}.$$

It can be converted to $[\![x]\!]$ locally by letting each P_i compute

$$\left(x^{(i)}, \Delta_i, \Delta_i \cdot x^{(i)} + \sum_{j\neq i}(M_j^{(i)} - K_j^{(i)})\right)$$

as his share of $[\![x]\!]$.

The sharings of the form $\langle\!\langle x \rangle\!\rangle$ are generated by $\mathcal{F}_{\mathsf{nVOLE}}$. More concretely, the parties need to invoke $\mathcal{F}_{\mathsf{nVOLE}}$ in order to receive the seeds and pair-wise MACs from $\mathcal{F}_{\mathsf{nVOLE}}$. Then, the parties expand their own seeds to get their shares of random additive sharings. The protocol Π_{rand} for generating random authenticated sharings is given in Figure 11.

Protocol Π_{rand}

Random Values:

- *Setup:* Each party P_i calls $\mathcal{F}_{\mathsf{nVOLE}}$ with input (Extend, m_R). P_i receives $(s_r^{(i)}, (\boldsymbol{M}_j^{(i)}, \boldsymbol{K}_j^{(i)})_{j\neq i})$ from $\mathcal{F}_{\mathsf{nVOLE}}$. Each P_i's $\mathsf{Expand}_{m_R}(s_r^{(i)}), (\boldsymbol{M}_j^{(i)}, \boldsymbol{K}_j^{(i)})_{j\neq i}$ form his shares of a vector of k sharings $\langle\!\langle \boldsymbol{r} \rangle\!\rangle$.
- To get the j-th random sharing ($1 \leq j \leq m_R$), each party takes the j-th share from the sharing $\langle\!\langle \boldsymbol{r} \rangle\!\rangle$ obtained from $\mathcal{F}_{\mathsf{nVOLE}}$ and locally converts it to a random authenticated additive sharing $[\![r]\!]$.

Figure 11: Protocol for preparing random sharings.

Preparing Input Masks. The input mask sharings are prepared in a similar way to the random sharings. Note that $\langle\!\langle r \rangle\!\rangle$ actually contains $[\Delta_i \cdot r]$ for each party P_i, the mask sharings can also be converted from the output of $\mathcal{F}_{\mathsf{nVOLE}}$ locally.

The protocol Π_{mask} is given in Figure 12.

Protocol Π_{mask}

Input Masks:

- *Setup:* The parties call $\mathcal{F}_{\mathsf{nVOLE}}$ with input $(\mathsf{Extend}, I = I_1 + \cdots + I_n)$. P_i receives $(s_r^{(i)}, (\boldsymbol{M}_j^{(i)}, \boldsymbol{K}_j^{(i)})_{j \neq i})$ from $\mathcal{F}_{\mathsf{nVOLE}}$. Each P_i's $\mathsf{Expand}_I(s_r^{(i)}), (\boldsymbol{M}_j^{(i)}, \boldsymbol{K}_j^{(i)})_{j \neq i}$ form his shares of a vector of k sharings $\langle\langle \boldsymbol{r} \rangle\rangle$.
- To get the j-th input mask sharing for P_i $(1 \leq j \leq I_i)$, each party takes the $(I_1 + \cdots + I_{i-1} + j)$-th share from the sharing $\langle\langle \boldsymbol{r} \rangle\rangle$ obtained from $\mathcal{F}_{\mathsf{nVOLE}}$ and locally converts it to $[\![r]\!].[\Delta_i \cdot r]$.

Figure 12: Protocol for preparing input mask sharings.

Preparing Authenticated Triples. We follow [GLM+24] to prepare authenticated triples. The protocol Π_{auth} is provided in Figure 13.

Protocol Π_{auth}

Authenticated Triples:

- *Setup:* Each party P_i calls $\mathcal{F}_{\mathsf{nVOLE}}$ with input (Extend, m_A) five times.
  1. P_i receives the seeds $s_a^{(i)}, s_b^{(i)}, s_a^{(i)'}, s_\ell^{(i)}, s_\ell^{(i)'}$. The outputs define vectors of shares $\langle\langle \boldsymbol{a} \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle, \langle\langle \boldsymbol{a}' \rangle\rangle, \langle\langle \boldsymbol{\ell} \rangle\rangle, \langle\langle \boldsymbol{\ell}' \rangle\rangle$ such that $\boldsymbol{a}^{(i)} = \mathsf{Expand}_{m_A}(s_a^{(i)})$, $\boldsymbol{b}^{(i)} = \mathsf{Expand}_{m_A}(s_b^{(i)})$, $\boldsymbol{a}^{(i)'} = \mathsf{Expand}_{m_A}(s_a^{(i)'})$, $\boldsymbol{\ell}^{(i)} = \mathsf{Expand}_{m_A}(s_\ell^{(i)})$, and $\boldsymbol{\ell}^{(i)'} = \mathsf{Expand}_{m_A}(s_\ell^{(i)'})$.
  2. Every pair of parties (P_i, P_j) calls $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ (corresponding to $\mathsf{Expand}_{m_A,k}$) with P_i sending $s_a^{(i)}$ and P_j sending $s_b^{(j)}$. From the outputs received from $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$, P_i extracts $\boldsymbol{u}_{i,j}$ and P_j extracts $\boldsymbol{v}_{i,j}$ such that $\boldsymbol{u}_{i,j} + \boldsymbol{v}_{j,i} = \boldsymbol{a}^{(i)} * \boldsymbol{b}^{(j)}$. Similarly, (P_i, P_j) calls $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ with $s_a^{(i)'}, s_b^{(j)}$ to obtain $\boldsymbol{u}'_{i,j} + \boldsymbol{v}'_{j,i} = \boldsymbol{a}^{(i)'} * \boldsymbol{b}^{(j)}$.
- To get the η-th triple $(1 \leq \eta \leq m_A)$:
  1. The parties take j-th shares from $\langle\langle \boldsymbol{a} \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle$, and $\langle\langle \boldsymbol{\ell} \rangle\rangle$. Then, the parties set them to be $\langle\langle a \rangle\rangle, \langle\langle b \rangle\rangle, \langle\langle \ell \rangle\rangle$. Each P_i computes $c^{(i)} = a^{(i)} \cdot b^{(i)} + \sum_{j \neq i}(u_{i,j,\eta} + v_{i,j,\eta})$ as his share of $[c]$, where $u_{i,j,\eta}, v_{i,j,\eta}$ are the η-th entries of $u_{i,j}, v_{i,j}$ respectively. Then the parties locally convert $\langle\langle a \rangle\rangle, \langle\langle b \rangle\rangle, \langle\langle \ell \rangle\rangle$ to $[\![a]\!], [\![b]\!], [\![\ell]\!]$.
  2. The parties compute their shares of $[\ell + c] = [\ell] + [c]$ and send them to P_{king}. P_{king} reconstructs $\ell + c$ and sends it to all the parties.
  3. Each P_i locally computes $[\![c]\!] = (\ell + c) - [\![\ell]\!]$. Then the parties get the η-th triple $([\![a]\!], [\![b]\!], [\![c]\!])$.
  4. For further verification, the parties repeat steps 1-3 on $\langle\langle \boldsymbol{a}' \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle, \langle\langle \boldsymbol{\ell}' \rangle\rangle$ in place of $\langle\langle \boldsymbol{a} \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle, \langle\langle \boldsymbol{\ell} \rangle\rangle$ to obtain $([\![a']\!], [\![b]\!], [\![c']\!])$.
- *Verification:* The parties call $\mathcal{F}_{\mathsf{Coin}}$ to get a random value $\alpha \in \mathbb{F}^*$. For each authenticated triple $([\![a]\!], [\![b]\!], [\![c]\!])$ with index η attached with $([\![a']\!], [\![b]\!], [\![c']\!])$:
  1. The parties run Π_{Open} on $[\![e]\!] = [\![\alpha^\eta \cdot a - a']\!]$.
  2. The parties compute $[\![\tau_\eta]\!] = [\![\alpha^\eta \cdot c]\!] - e \cdot [\![b]\!] - [\![c']\!]$ and run Π_{Open} on it.

  After obtaining $[\![\tau_\eta]\!]$ for $\eta = 1, \ldots, m_A$, the parties run Π_{Open} on $[\![\tau']\!] = \sum_{\eta=1}^{m_A} [\![\tau_\eta]\!]$. If $\tau' \neq 0$, the parties abort the protocol. Otherwise, they run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to check all the opened values (i.e. e for each triple and τ').

Figure 13: Protocol for preparing authenticated triples.

Preparing Partially Length-k Authenticated Tensor Products. To prepare each length-k tensor product, the parties first prepare random authenticated sharings $([\![a_i]\!], [\![b_i]\!])_{i=1,\ldots,k}$ as in Π_{rand}. Then, the parties call $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ with their received seeds to prepare the additive sharings of the cross-terms. In particular, the expansion functions in the two functionalities are both set to be $\mathsf{Expand}_{km_T,k}$ given in Section 4.2. The protocol Π_{tens} is provided in Figure 14.

Protocol Π_{tens}

Tensor Products:

– *Setup:* Each party P_i calls $\mathcal{F}_{\mathsf{nVOLE}}$ with input $(\mathsf{Extend}, m = k \cdot m_T + k \cdot \lceil m_T/k \rceil)$ two times.

1. P_i receives the seeds $s_a^{(i)}, s_b^{(i)}$ and obtains shares of $\langle\langle \boldsymbol{a} \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle$, where $s_a^{(i)}, s_b^{(i)}$ can be expanded to P_i's shares (denoted by $\boldsymbol{a}^{(i)}, \boldsymbol{b}^{(i)}$) of $[\boldsymbol{a}], [\boldsymbol{b}]$.
2. Each (ordered) pair of parties (P_i, P_j) calls $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ (corresponding to $\mathsf{Expand}_{m,k}$) with P_i sending $s_a^{(i)}$ and P_j sending $s_b^{(j)}$. It sends $\boldsymbol{u}_{i,j}$ back to P_i and $\boldsymbol{v}_{i,j}$ to P_j.

– To get the α-th ($1 \leq \alpha \leq m_T$) tensor product, let $\boldsymbol{a}_\alpha, \boldsymbol{b}_\alpha$ be the vectors of the $k^2(\alpha-1)+1, \ldots, k^2\alpha$-th entries of $\boldsymbol{a}, \boldsymbol{b}$ respectively.

1. Each party P_i computes $\boldsymbol{a}^{(i)} \otimes \boldsymbol{b}^{(i)} + \sum_{j \neq i} (\boldsymbol{u}_{i,j} + \boldsymbol{v}_{j,i})$ and extracts the $k^2(\alpha-1)+1, \ldots, k^2\alpha$-th entries of it as his share of $[\boldsymbol{a}_\alpha \otimes \boldsymbol{b}_\alpha]$.
2. The parties extract their shares of $\langle\langle \boldsymbol{a}_\alpha \rangle\rangle, \langle\langle \boldsymbol{b}_\alpha \rangle\rangle$ from $\langle\langle \boldsymbol{a} \rangle\rangle, \langle\langle \boldsymbol{b} \rangle\rangle$ and locally convert them to $[\![\boldsymbol{a}_\alpha]\!], [\![\boldsymbol{b}_\alpha]\!]$. Then, the parties obtain the α-th tensor product $[\![\boldsymbol{a}_\alpha]\!], [\![\boldsymbol{b}_\alpha]\!], [\boldsymbol{a}_\alpha \otimes \boldsymbol{b}_\alpha]$. By setting $\boldsymbol{a}_\alpha = (a_1, \ldots, a_k), \boldsymbol{b}_\alpha = (b_1, \ldots, b_k), c_{i,j} = a_i \cdot b_j$, the tensor product is transformed into the form in $\mathcal{F}_{\mathsf{prep}}$.

Here, the $(m_T+1), \ldots, (m_T + \lceil m_T/k \rceil)$-th tensor products are regarded as m_T triples $\{([\![x'_\alpha]\!], [\![y'_\alpha]\!], [v'_\alpha])\}_{\alpha=1,\ldots,m_T}$, which are used for checking the first m_T tensor products.

– *Verification:* Let PRG be a public pseudorandom generator with seed space $\mathbb{F}^*$ and image space $(\mathbb{F}^*)^{2k}$. For each length-k tensor product (with index α) $\{([\![a_i]\!], [\![b_i]\!])\}_{i=1,\ldots,k}, \{[c_{i,j}]\}_{i,j \in \{1,\ldots,k\}}$:

1. The parties send $\mathsf{RandCoin}$ to $\mathcal{F}_{\mathsf{Coin}}$ and receive a random element $r_\alpha \in \mathbb{F}^*$ from it. Then, the parties locally compute
$$\mathsf{PRG}(r_\alpha) \to (r_a^{(1)}, \ldots, r_a^{(k)}, r_b^{(1)}, \ldots, r_b^{(k)}) \in (\mathbb{F}^*)^{2k}.$$
2. The parties locally compute $[\![x_\alpha]\!] = \sum_{i=1}^k r_a^{(i)} [\![a_i]\!]$ and $[\![y_\alpha]\!] = \sum_{i=1}^k r_b^{(i)} [\![b_i]\!]$. Then, the parties locally compute
$$[v_\alpha] = \sum_{i=1}^k \sum_{j=1}^k r_a^{(i)} r_b^{(j)} [c_{i,j}].$$
3. The parties locally compute $[\![x_\alpha + x'_\alpha]\!], [\![y_\alpha + y'_\alpha]\!]$ and run Π_{Open} on them. Then, the parties locally compute
$$[\tau^{(\alpha)}] = (x_\alpha + x'_\alpha) \cdot (y_\alpha + y'_\alpha) - (x_\alpha + x'_\alpha) \cdot [y'_\alpha] - (y_\alpha + y'_\alpha) \cdot [x'_\alpha] + [v'_\alpha] - [v_\alpha].$$

After computing $[\tau^{(\alpha)}]$ for each tensor product, the parties compute $[\tau] = \sum_{\alpha=1}^{m_T} [\tau^{(\alpha)}]$. The parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ on all the opened values during the verification, i.e., $x_\alpha + x'_\alpha, y_\alpha + y'_\alpha$ for $\alpha = 1, \ldots, m_T$. Finally, each party P_i calls $\mathcal{F}_{\mathsf{Commit}}$ with input $(\mathsf{commit}, P_i, \tau_i, \mathsf{mid}_{\tau_i})$, where τ_i is P_i's share of $[\tau]$. The parties then open their commitments via sending $(\mathsf{open}, P_i, \mathsf{mid}_{\tau_i})$ for each party P_i and check that $\sum_{i=1}^n \tau_i = 0$. If not, the parties abort the protocol.

Figure 14: Protocol for preparing length-k partially authenticated tensor product.

Lemma 1 *In the* $\{\mathcal{F}_{\mathsf{nVOLE}}, \mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}, \mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}\}$*-hybrid model,* Π_{prep} *securely realizes* $\mathcal{F}_{\mathsf{prep}}$ *against* $\mathcal{A}$*.*

The security proof for the preprocessing protocol is provided in Section A.

6 Evaluation Phase

In this section, we show how the parties share their inputs and perform online evaluation. The online evaluation phase Π_{eval} is run after the preprocessing. The parties first transform the circuit C into a circuit C' of public outputs. Then, they evaluate the circuit C' gate by gate and compute authenticated sharings for:

- All inputs/outputs of C'.

- All input wire values of multiplication gates in C'.

The protocol Π_{eval} is shown in Figure 15.

Protocol Π_{eval}

For an arithmetic circuit C over $\mathbb{F}$ to compute, the parties first transform the circuit to a circuit C' with $C'(x_1\|\mathsf{mask}_1, \ldots, x_m\|\mathsf{mask}_n) = C(x_1, \ldots, x_n) \oplus (\mathsf{mask}_1, \ldots \mathsf{mask}_n)$. The parties initialize a set $\mathcal{S}_{\mathsf{check}} = \emptyset$ at the beginning. Then:

1. **Authenticating Inputs.** For each input wire of C' attached to party P_i with input value x, the parties consume a pair of input mask sharings $([\![r]\!], [\Delta_i \cdot r])$ (from the outputs of $\mathcal{F}_{\mathsf{prep}}$) and send their shares of $[r], [\Delta_i \cdot r]$ to P_i. P_i then reconstructs the secrets and verifies whether they match his MAC key Δ_i. If not, he aborts the protocol. Otherwise, he sends $x + r$ to all the parties. Finally, the parties obtain $[\![x]\!] = (x + r) - [\![r]\!]$.
2. **Evaluating the Circuit.** The multiplication gates of C' are partitioned into $\ell = |C|/k$ groups[a]. The j-th group of multiplication gates has input wire values denoted by $(x_1^{(j)}, y_1^{(j)}), \ldots, (x_k^{(j)}, y_k^{(j)})$ and output wire values denoted by $z_1^{(j)}, \ldots, z_k^{(j)}$. For each gate in C':
   - If the gate is an addition gate with input values x, y and output z, the parties locally compute $[z] = [x] + [y]$.
   - If the gate is a multiplication gate with inputs $x_\alpha^{(j)}, y_\alpha^{(j)}$:
     (a) The parties take the j-th length-k tensor product of the form $\{([\![a_i]\!], [\![b_i]\!])\}_{i=1,\ldots,k}, \{[a_i \cdot b_{i'}]\}_{i,i' \in \{1,\ldots,k\}}$ prepared by $\mathcal{F}_{\mathsf{prep}}$ and parse $([\![a_\alpha]\!], [\![b_\alpha]\!], [c_{\alpha,\alpha}])$ from it.
     (b) * If $x_\alpha^{(j)}$ (resp. $y_\alpha^{(j)}$) can be linearly computed from inputs of C', then the parties locally compute $[\![x_\alpha^{(j)} + a_\alpha]\!] = [\![x_\alpha^{(j)}]\!] + [\![a_\alpha]\!]$ (resp. $[\![y_\alpha^{(j)} + b_\alpha]\!] = [\![y_\alpha^{(j)}]\!] + [\![b_\alpha]\!]$). Then, the parties run Π_{Open} on $[\![x_\alpha^{(j)} + a_\alpha]\!]$ (resp. $[\![y_\alpha^{(j)} + b_\alpha]\!]$).
     * Otherwise, the parties locally compute $[x_\alpha^{(j)} + a_\alpha] = [x_\alpha^{(j)}] + [a_\alpha]$ (resp. $[y_\alpha^{(j)} + b_\alpha] = [y_\alpha^{(j)}] + [b_\alpha]$) and it them to P_{king}. P_{king} reconstructs the secret and sends it back to all the parties. The parties then add $x_\alpha^{(j)}$ to $\mathcal{S}_{\mathsf{check}}$.
     The parties then locally compute $[\![x_\alpha^{(j)}]\!] = (x_\alpha^{(j)} + a_\alpha) - [\![a_\alpha]\!], [\![y_\alpha^{(j)}]\!] = (y_\alpha^{(j)} + b_\alpha) - [\![b_\alpha]\!]$.
     (c) The parties locally compute
     $$[z_\alpha^{(j)}] = (x_\alpha^{(j)} + a_\alpha) \cdot (y_\alpha^{(j)} + b_\alpha) - (x_\alpha^{(j)} + a_\alpha) \cdot [b_\alpha] - (y_\alpha^{(j)} + b_\alpha) \cdot [a_\alpha] + [c_{\alpha,\alpha}].$$

   Finally, for each output wire with value v of C', the parties obtain $[v]$. The parties consume a random sharing $[\![r]\!]$ obtained from the output of $\mathcal{F}_{\mathsf{prep}}$ and do the following:
   - If v can be linearly computed from inputs of C', then the parties locally compute $[\![r + v]\!] = [\![r]\!] + [\![v]\!]$. Then, the parties run Π_{Open} on $[\![r + v]\!]$.
   - Otherwise, send their shares of $[r + v] = [r] + [v]$ to P_{king}. P_{king} reconstructs $r + v$ and sends it to all the parties. The parties then add v to $\mathcal{S}_{\mathsf{check}}$.

   The parties then locally obtain $[\![v]\!] = (r + v) - [\![r]\!]$.
3. **MAC Check.** The parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to check all the values opened via Π_{Open} during the evaluation phase.

[a] We assume without loss of generality that k divides $|C|$. If not, we only need to add some multiplication gates with public inputs $(0, 0)$, such that the multiplication gates can be grouped into $\ell = \lceil |C|/k \rceil$ groups.

Figure 15: Online evaluation protocol.

During the evaluation phase, a mask sharing associated with P_i is required for each input wire of C' attached to P_i. Consequently, the total number of required mask sharings, $([\![r]\!], [\Delta_i \cdot r])$, corresponds to the total number of input and output wires attached to P_i in circuit C. Furthermore, since each length-k tensor product facilitates the evaluation of k AND gates, the process requires $\lceil |C|/k \rceil$ length-k tensor products. Finally, a random authenticated sharing is required for each output wire of C', so the total number of such

sharings consumed in Π_{eval} is equal to the number of output wires in C.

7 Verification Phase

In this section, we show how to verify the correctness of online evaluation. The protocol is provided in Figure 7.

Protocol Π_{ver}

1. **Reducing to Inner-Product Checks.** Denote all pairs of input wire values for the multiplication gates in C' by $(x_1, y_1), \ldots, (x_{|C|}, y_{|C|})$, and denote all the inputs of C' by $v_1, \ldots, v_I$. Let all the values in $\mathcal{S}_{\mathsf{check}}$ be $z_1, \ldots, z_M$, including all the input wire values of multiplication gates and the outputs of C'. Each z_i is computed via a linear combination of $x_1 \cdot y_1, \ldots, x_{|C|} \cdot y_{|C|}$ and $v_1, \ldots, v_I$:

$$\sum_{j=1}^{|C|} \alpha_i^{(j)} x_j y_j + \sum_{j=1}^{I} \beta_i^{(j)} v_j = z_i,$$

where $\{\alpha_i^{(j)}\}_{j=1,\ldots,|C|}, \{\beta_i^{(j)}\}_{j=1,\ldots,I}$ are constants determined by the circuit's linear gates and overall topology for each $i = 1, \ldots, M$. For each $j = 1, \ldots, |C|$, $\{\alpha_i^{(j)}\}_{j=1,\ldots,|C|}, \{\beta_i^{(j)}\}_{i=1,\ldots,M}$ are not all zero (or the checked value can be directly computed from the inputs without evaluating multiplication gates). Then:

   (a) The parties send $\mathsf{RandCoin}$ to $\mathcal{F}_{\mathsf{Coin}}$ and obtain a random element $s \in \mathbb{F}^*$.

   (b) Recall that the parties have $[\![x_i]\!], [\![y_i]\!]$ for each multiplication gate with input values x_i, y_i, and they also have an authenticated sharing for each input/output of C'. Thus, the parties have the authenticated sharings for all the values in $\mathcal{S}_{\mathsf{check}}$, so they can locally compute

$$[\![\boldsymbol{x}]\!] = \left(\sum_{i=1}^{M} s^i \cdot \alpha_i^{(1)} \cdot [\![x_1]\!], \ldots, \sum_{i=1}^{M} s^i \cdot \alpha_i^{(|C|)} \cdot [\![x_{|C|}]\!] \right), \quad [\![\boldsymbol{y}]\!] = ([\![y_1]\!], \ldots, [\![y_{|C|}]\!])$$

   and

$$[\![z]\!] = \sum_{i=1}^{M} s^i [\![z_i]\!] - \sum_{i=1}^{M} \sum_{j=1}^{I} s^i \beta_i^{(j)} [\![v_j]\!].$$

   With high probability, the coefficient $\theta_j = \sum_{i=1}^{M} s^i \cdot \alpha_i^{(j)}$ corresponding to $[\![x_j]\!]$ is non-zero for each $j = 1, \ldots, |C|$. If not, the parties abort the protocol.

2. **Perform the Inner-Product Check.** The parties run the protocol Π_{IPcheck} (Figure 17) to verify that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = z$.

Figure 16: Protocol for the verification phase.

The Inner-product check process Π_{IPcheck} is provided below.

Protocol Π_{IPcheck}

To verify that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = z$ with $[\![\boldsymbol{x}]\!], [\![\boldsymbol{y}]\!], [\![z]\!]$:

1. **Separate into Blocks.**

   (a) For each group (of index j) of multiplication gates with input wires $(x_1^{(j)}, y_1^{(j)}), \ldots, (x_k^{(j)}, y_k^{(j)})$, let $\boldsymbol{x}^{(j)} = (x_1^{(j)}, \ldots, x_k^{(j)})$ and $\boldsymbol{y}^{(j)} = (y_1^{(j)}, \ldots, y_k^{(j)})$. Let the coefficient corresponding to $[\![x_i^{(j)}]\!]$ in $[\![x]\!]$ be $\theta_i^{(j)}$ and let $\boldsymbol{\theta}^{(j)} = (\theta_1^{(j)}, \ldots, \theta_k^{(j)})$. The parties compute $[z^{(j)}]$ with $z^{(j)} = \langle \boldsymbol{\theta}^{(j)} * \boldsymbol{x}^{(j)}, \boldsymbol{y}^{(j)} \rangle$ by $[z^{(j)}] = \sum_{i=1}^{\ell} \theta_i^{(j)} [z_i^{(j)}]$, where each $[z_i^{(j)}]$ has been computed during the evaluation phase.

   (b) For each $j = 1, \ldots, \ell - 1$, the parties consume a random sharing $[\![r^{(j)}]\!]$ (from the output of $\mathcal{F}_{\mathsf{prep}}$, same below) and locally compute $[r^{(j)} + z^{(j)}] = [r^{(j)}] + [z^{(j)}]$. Then, they send their shares to P_{king}. P_{king} reconstructs the secret and sends it to all the parties. The parties locally computes $[\![z^{(j)}]\!] = (r^{(j)} + z^{(j)}) - [\![r^{(j)}]\!]$. Finally, the parties compute $[\![z^{(\ell)}]\!] = \sum_{i=1}^{\ell-1} [\![z^{(j)}]\!]$.

Then, the inner product check is reduced to checking ℓ blocks of inner product checks with vector length k: $\langle \boldsymbol{\theta}^{(j)} * \boldsymbol{x}^{(j)}, \boldsymbol{y}^{(j)} \rangle = z^{(j)}$ for $j = 1, \ldots, \ell$. In addition, the parties compute $[(\boldsymbol{\theta}^{(j)} * \boldsymbol{x}^{(j)}) \otimes \boldsymbol{y}^{(j)}]$ by

$$\begin{aligned}[\theta_\alpha^{(j)} x_\alpha^{(j)} \cdot y_\beta^{(j)}] =& \theta_\alpha^{(j)} \cdot (x_\alpha^{(j)} + a_\alpha) \cdot (y_\beta^{(j)} + b_\beta) - \theta_\alpha^{(j)} \cdot (x_\alpha^{(j)} + a_\alpha) \cdot [b_\beta] \\ & - \theta_\alpha^{(j)} \cdot (y_\beta^{(j)} + b_\beta) \cdot [a_\alpha] + \theta_\alpha^{(j)} \cdot [c_{\alpha,\beta}]\end{aligned}$$

for each pair of $(\alpha, \beta) \in \{1, \ldots, k\}^2$, where $\{[\![a_\alpha]\!], [\![b_\alpha]\!]\}_{\alpha=1,\ldots,k}, \{[c_{\alpha,\beta}]\}_{(\alpha,\beta)\in\{1,\ldots,k\}^2}$ is the length-k tensor product used for evaluating the j-th group of multiplication gates during the evaluation phase.

2. **Recursive Check.** To check each inner product $\langle \boldsymbol{\theta}^{(j)} * \boldsymbol{x}^{(j)}, \boldsymbol{y}^{(j)} \rangle = z^{(j)}$, we use a recursive reduction on vector length. The parties initialize $[\![\boldsymbol{u}]\!] = \boldsymbol{\theta}^{(j)} * [\![\boldsymbol{x}^{(j)}]\!], [\![\boldsymbol{v}]\!] = [\![\boldsymbol{y}^{(j)}]\!], [\![w]\!] = [\![z^{(j)}]\!]$, and let $[\boldsymbol{u} \otimes \boldsymbol{v}] = [(\boldsymbol{\theta}^{(j)} * \boldsymbol{x}^{(j)}) \otimes \boldsymbol{y}^{(j)}]$. Then, the parties repeat the following until the length of $\boldsymbol{u}, \boldsymbol{v}$ is reduced to 1:
   (a) The parties partition $\boldsymbol{u}$ into λ vectors $(\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\lambda)$, where each $\boldsymbol{u}_i$ has the same length ℓ' (if λ is not a factor of the vector length, the parties add zeros after $\boldsymbol{u}$). Similarly, $\boldsymbol{v}$ is partitioned into $(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\lambda)$.
   (b) For each $i = 1, \ldots, \lambda - 1$, note that $\boldsymbol{u}_i = (\langle \boldsymbol{c}_{u_\alpha}, \boldsymbol{x}^{(j)} \rangle)_{\alpha=(i-1)\ell'+1}^{i\ell'}$ (representing the vector consisting of $\langle \boldsymbol{c}_{u_\alpha}, \boldsymbol{x}^{(j)} \rangle$ for $\alpha = (i-1)\ell' + 1, \ldots, i\ell'$) and $\boldsymbol{v}_i = (\langle \boldsymbol{c}_{v_\alpha}, \boldsymbol{v}^{(j)} \rangle)_{\alpha=(i-1)\ell'+1}^{i\ell'}$. The parties locally compute $[w_i]$ with $w_i = \langle \boldsymbol{u}_i, \boldsymbol{v}_i \rangle$ from $[\boldsymbol{u} \otimes \boldsymbol{v}]$.
   (c) For each $i = 1, \ldots, \lambda - 1$, the parties consume a random sharing $[\![r_i]\!]$ and locally compute $[r_i + w_i] = [r_i] + [w_i]$. Then, they send their shares to P_{king}. P_{king} reconstructs the secret and sends it to all the parties. The parties locally compute $[\![w_i]\!] = (r_i + w_i) - [\![r_i]\!]$. Finally, the parties compute $[\![w_\lambda]\!] = \sum_{i=1}^{\lambda-1} [\![w_i]\!]$.
   (d) Each party P_j computes a vector of ℓ degree-$(\lambda - 1)$ polynomials $\boldsymbol{f}_j(\cdot)$ such that $\boldsymbol{f}_j(i)$ equals his share of $[\boldsymbol{u}_i]$ for each $i = 1, \ldots, \lambda$. Similarly, P_j computes a vector of degree-$(\lambda - 1)$ polynomials $\boldsymbol{g}_j(\cdot)$ such that $\boldsymbol{g}_j(i)$ equals his share of $[\boldsymbol{v}_i]$ for each $i = 1, \ldots, \lambda$. Let $\boldsymbol{f}(\cdot) = \boldsymbol{f}_1(\cdot) + \cdots + \boldsymbol{f}_n(\cdot)$ and $\boldsymbol{g}(\cdot) = \boldsymbol{g}_1(\cdot) + \cdots + \boldsymbol{g}_n(\cdot)$.
   (e) Let $h(\cdot) = \langle \boldsymbol{f}(\cdot), \boldsymbol{g}(\cdot) \rangle$, which is a degree-$(2\lambda - 2)$ polynomial. Then, we need to verify that $w_i = h(i)$ for $i = 1, \ldots, \lambda$. For each $i = \lambda + 1, \ldots, 2\lambda - 1$, let $\boldsymbol{c}_i = (c_i^{(1)}, \ldots, c_i^{(\lambda)})$ be the coefficient vector such that $\boldsymbol{f}(i) = \sum_{j=1}^{\lambda} c_i^{(j)} \boldsymbol{u}_j$. The parties then locally compute

$$[h(i)] = \sum_{\alpha=1}^{\lambda} \sum_{\beta=1}^{\lambda} c_i^{(\alpha)} c_i^{(\beta)} [\langle \boldsymbol{u}_\alpha, \boldsymbol{v}_\beta \rangle],$$

   where each $[\langle \boldsymbol{u}_\alpha, \boldsymbol{v}_\beta \rangle]$ is computed locally from $[\boldsymbol{u} \otimes \boldsymbol{v}]$.
   (f) For each $i = \lambda + 1, \ldots, 2\lambda - 1$, the parties consume a random sharing $[\![r_i]\!]$ and locally compute $[r_i + h(i)] = [r_i] + [h(i)]$. Then, they send their shares to P_{king}. P_{king} reconstructs the secret and sends it to all the parties. The parties locally compute $[\![h(i)]\!] = (r_i + h(i)) - [\![r_i]\!]$.
   (g) The parties send $\mathsf{RandCoin}$ to $\mathcal{F}_{\mathsf{Coin}}$ and receive $r \in \mathbb{F}$[a]. Since $\boldsymbol{f}(\cdot), \boldsymbol{g}(\cdot)$ are vectors of degree-$(\lambda - 1)$ polynomials, $\boldsymbol{f}(r), \boldsymbol{g}(r)$ can be computed from $(\boldsymbol{f}(1), \ldots, \boldsymbol{f}(\lambda)), (\boldsymbol{g}(1), \ldots, \boldsymbol{g}(\lambda))$ respectively via a linear function L. The parties then locally compute $[\![\boldsymbol{f}(r)]\!] = L([\![\boldsymbol{f}(1)]\!], \ldots, [\![\boldsymbol{f}(\lambda)]\!])$ and $[\![\boldsymbol{g}(r)]\!] = L([\![\boldsymbol{f}(1)]\!], \ldots, [\![\boldsymbol{f}(\lambda)]\!])$. Similarly, since $h(\cdot)$ is of degree $(2\lambda - 2)$, there exists a linear function L' such that $h(x) = L'(h(1), \ldots, h(2\lambda - 1))$. The parties then locally compute $[\![h(r)]\!] = L'([\![w_1]\!], \ldots, [\![w_\lambda]\!], [\![h(\lambda + 1)]\!], \ldots, [\![h(2\lambda - 1)]\!])$.
   (h) Assume $L(\boldsymbol{s}) = \langle \boldsymbol{c}, \boldsymbol{s} \rangle$. The parties update $[\boldsymbol{u} \otimes \boldsymbol{v}]$ by

$$[\boldsymbol{u} \otimes \boldsymbol{v}] = \langle \boldsymbol{c} \otimes \boldsymbol{c}, [(\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\lambda) \otimes (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\lambda)] \rangle,$$

   where $(\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\lambda) \otimes (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\lambda)$ means the length-λ^2 vector in $(\mathbb{F}^{\ell'^2})^{\lambda^2}$ containing $\boldsymbol{u}_\alpha \otimes \boldsymbol{v}_\beta$ for each pair of $\alpha, \beta \in \{1, \ldots, \lambda\}$. In particular, $[(\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\lambda) \otimes (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\lambda)]$ can be extracted from $[\boldsymbol{u} \otimes \boldsymbol{v}]$. Finally, the parties update $[\![\boldsymbol{u}]\!] = [\![\boldsymbol{f}(r)]\!], [\![\boldsymbol{v}]\!] = [\![\boldsymbol{g}(r)]\!]$, and $[\![w]\!] = [\![h(r)]\!]$.
3. **Verify the Final Triple.** After Step 2, the parties finally obtain a single triple $([\![u]\!], [\![v]\!], [\![w]\!])$ for each group of multiplication gates, where we need to verify that $w = u \cdot v$. Then:

(a) The parties consume an authenticated triple $([\![a]\!], [\![b]\!], [\![c]\!])$ obtained from $\mathcal{F}_{\mathsf{prep}}$, locally compute $[\![u+a]\!], [\![v+b]\!]$, and run Π_{Open} on them. Then, the parties locally compute

$$[\![\tau]\!] = (u+a)\cdot(v+b) - (u+a)\cdot[\![b]\!] - (v+b)\cdot[\![a]\!] + [\![c]\!] - [\![w]\!].$$

(b) The parties run Π_{Open} on $[\![\tau]\!]$ and check whether $\tau = 0$. If not, they abort the protocol.

Finally, the parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to check all the opened values during the check, i.e., the values $u+a, v+b, \tau$ for all ℓ groups of multiplication gates.

[a] This random coin can be used to check all the blocks of inner products.

Figure 17: Inner-Product Check.

During verification, separating the vectors into blocks requires $\ell - 1$ random authenticated sharings. In addition, $2(\lambda-1)$ random sharings are consumed during each reduction in vector length. Since the reduction needs to be performed $\ell \cdot \lceil \log_\lambda k \rceil$ times, $2\ell(\lambda-1)\cdot\lceil \log_\lambda k \rceil$ random sharings are required in total. Moreover, for the check of final triples, $\ell = \lceil |C|/k \rceil$ authenticated triples are required.

8 Main Protocol

Finally, we put everything together in our main protocol Π_{main}. The protocol runs the preprocessing, online, and verification phases successively. Finally, the parties run an output reconstruction protocol. We provide Π_{main} in Figure 18.

Protocol Π_{main}

1. **Preprocessing.** Let m_O be the number of output wires in C, and let $\ell = \lceil |C|/k \rceil$. The parties invoke $\mathcal{F}_{\mathsf{prep}}$ with inputs $(\mathsf{Prep}, m_R, (I_1, \ldots, I_n), m_A, m_T)$ to $\mathcal{F}_{\mathsf{prep}}$ and receive the preprocessing data from $\mathcal{F}_{\mathsf{prep}}$, where $m_R = m_O + 2\ell(\lambda-1)\cdot\lceil \log_\ell k \rceil + \ell - 1$, each I_j is set to the total number of input/output wires attached to C_j in circuit C, $m_A = \ell$, and $m_T = \lceil |C|/k \rceil$.
2. **Evaluation.** The parties run Π_{eval} (Figure 15).
3. **Verification.** The parties run Π_{ver} (Figure 16).
4. **Output Reconstruction.** For each output wire with value v of C', the parties run Π_{Open} on $[\![v]\!]$. Then, the parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ on all the opened output values. After that, all the parties send each output wire value to the designated output party. Each party then checks whether the received outputs are all the same. If not, the party aborts the protocol. Otherwise, he locally computes his output of C from his output of C' and the masks he inputs to C'.

Figure 18: The main protocol.

Theorem 1 *In the $\{\mathcal{F}_{\mathsf{prep}}, \mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}\}$-hybrid model, Π_{main} is a secure multiparty computation protocol against $\mathcal{A}$.*

The security proof for the main protocol is provided in Section B, and we give a detailed cost analysis in Section C. Regardless of instantiating the PCGs, the total communication cost is $4|C|n + 5m_I n + 7m_O n + O(|C|n\lambda k^{-1}\cdot\log_\lambda k) + \mathsf{poly}(n,\kappa)$, where $|C|, m_I, m_O$ are the numbers of multiplication gates, input wires, and output wires in C, respectively. This results in an overall communication cost of $4|C|n+5m_In+7m_On+o(|C|)$ by taking $\lambda = 2$ and $k = \omega(n\log n)$, giving an amortized communication cost of $4n$ per multiplication gate, $5n$ per input wire, and $7n$ per output wire. The total computational cost is $O(|C|nk)$ beyond the PCGs.

9 Further Reducing Communication Cost via Turbospeedz

In this section, we show how to further reduce the amortized communication cost of our protocol to $2n$ per multiplication gate using Turbospeedz online evaluation [BNO19]. The protocol Π_{turbo} achieving this target

requires the preprocessing of tensor product correlations for length-$|C|$ vectors and computing, which incurs a quadratic computational complexity in $|C|$. Furthermore, the online computational cost is also quadratic in $|C|$. This provides a trade-off between communication and computational complexities.

Preprocessing. In particular, the preprocessing is similar to that of our main protocol. The only difference is that the length-k tensor products and the input mask sharings are replaced by the preprocessing of $[\![\boldsymbol{w}]\!], [\boldsymbol{w} \otimes \boldsymbol{w}]$ for a random vector $\boldsymbol{w} = (w_1, \ldots, w_{m_C})$ with $m_C = m_I + m_O + |C|$, where we denote the sharing $[w_i \cdot w_j]$ from $[\boldsymbol{w} \otimes \boldsymbol{w}]$ by $[w_{i,j}]$ for each pair of $(i,j) \in \{1, \ldots, m_C\}^2$. In particular, $[\![w_i]\!]$ for $i = 1, \ldots, m_I + m_O$ serves as the input mask sharings, where $[\Delta_j \cdot w_i]$ is preserved if the corresponding input wire of index i is attached to P_j. The preprocessing functionality $\mathcal{F}_{\mathsf{Turboprep}}$ is provided in Figure 19.

Functionality $\mathcal{F}_{\mathsf{Turboprep}}$

On receiving $(\mathsf{Prep}, m_R, (I_1, \ldots, I_n), m_A, m_C)$ from all the honest parties, the trusted party run **Init**, **Random Values**, and **Authenticated Triples** of $\mathcal{F}_{\mathsf{prep}}$ and additional does the following:

1. Run steps 1 and 2 from **Random Values** of $\mathcal{F}_{\mathsf{prep}}$ $m = I_1 + \cdots + I_n + m_C$ times to create sharings $\{[\![w_i]\!]\}_{i=1,\ldots,m}$.
2. For each $j = 1, \ldots, n$ and $i = I_1 + \cdots + I_{j-1} + \alpha$ for $\alpha = 1, \ldots, I_j$, the trusted party computes $\Delta_j \cdot w_i$ and receives corrupted parties' shares of $[\Delta_j \cdot w_i]$ from Sim. Then, the trusted party samples honest parties' shares of $[\Delta_j \cdot w_i]$ based on the secret and corrupted parties' shares. Then, the trusted party distributes $[\Delta_j \cdot w_i]$ to the parties.
3. For each pair of $(i,j) \in \{1, \ldots, m\}^2$, the trusted party computes $w_{i,j} = w_i \cdot w_j$ and receives corrupted parties' shares of $[w_{i,j}]$ from Sim. Then, the trusted party samples the honest parties' shares of it based on the secret and the corrupted parties' shares.
4. The trusted party distributes $\{[\![w_i]\!]\}_{i=1,\ldots,m}, \{[w_{i,j}]\}_{i,j \in \{1,\ldots,m\}}$ to the parties.

Abort: If (abort, P) is received from Sim for an honest party P, the trusted party sends Δ to Sim and sends abort to P.

Figure 19: Functionality for the preprocessing phase of Π_{turbo}.

The instantiation of $\mathcal{F}_{\mathsf{Turboprep}}$ is similar to that of $\mathcal{F}_{\mathsf{prep}}$. The additional task of preparing $\{[\![w_i]\!]\}_{i=1,\ldots,m}$, $\{[w_{i,j}]\}_{i,j \in \{1,\ldots,m\}}$ can be regarded as a single length-m partially authenticated tensor product with $a_i = b_i = w_i$ for each $i = 1, \ldots, m$. The parties only need to follow Π_{tens} to provide it, except that $[\![\boldsymbol{b}]\!]$ is not prepared in parallel with $[\![\boldsymbol{a}]\!]$ (namely $[\![\boldsymbol{w}]\!]$ in $\mathcal{F}_{\mathsf{Turboprep}}$). Instead, $[\![\boldsymbol{b}]\!]$, together with the seeds for generating it, is set to $[\![\boldsymbol{a}]\!]$ together with the seeds generating it. All other steps, including the verification steps, are the same as Π_{tens}. As we have demonstrated, the input mask sharings can be obtained from local computation while preparing $\{[\![w_i]\!]\}_{i=1,\ldots,m}$. For completeness, we provide the detailed protocol $\Pi_{\mathsf{turboprep}}$ for instantiating $\mathcal{F}_{\mathsf{Turboprep}}$ in Section D.

Evaluation. The online evaluation follows the framework of [BNO19]. More concretely, each output of a multiplication gate is computed directly from all the outputs of multiplication gates and the input wires of the circuit via a degree-2 sub-circuit, using $\{[\![w_i]\!]\}_{i=1,\ldots,m}$, $\{[w_{i,j}]\}_{i,j \in \{1,\ldots,m\}}$ prepared by $\mathcal{F}_{\mathsf{Turboprep}}$.

Protocol $\Pi_{\mathsf{turboeval}}$

For an arithmetic circuit C over $\mathbb{F}$ to compute, the parties first transform the circuit to a circuit C' with $C'(x_1 \| \mathsf{mask}_1, \ldots, x_m \| \mathsf{mask}_n) = C(x_1, \ldots, x_n) \oplus (\mathsf{mask}_1, \ldots \mathsf{mask}_n)$. The input wires of C' attached to $P_1, \ldots, P_n$ are indexed in order by $1, \ldots, m_I + m_O$. The output wires of multiplication gates are then indexed in topological order by $m_1 + m_O + 1, \ldots, m_I + m_O + |C|$. Then:

1. **Authenticating Inputs.** For each input wire of index j of C' attached to party P_i with input value z_j, the parties send their shares of $[w_j], [\Delta_i \cdot w_j]$ to P_i. P_i then reconstructs the secrets and verifies whether they match his MAC key Δ_i. If not, he aborts the protocol. Otherwise, he sends $z_j + w_j$ to all the parties.

Finally, the parties obtain $[\![z_j]\!] = (z_j + w_j) - [\![w_j]\!]$.

2. **Evaluating the Circuit.** For $i = m_I + m_O + 1$ to $m_I + m_O + |C|$:

   (a) The value $z_j + w_j$ has been publicly reconstructed for each $j = 1, \ldots, i-1$. For each $\alpha = 1, \ldots, i-2$, the parties locally compute $[z_\alpha \cdot z_{i-1}]$ by

   $$(z_\alpha + w_\alpha) \cdot (z_{i-1} + w_{i-1}) - (z_\alpha + w_\alpha) \cdot [w_{i-1}] - (z_{i-1} + w_{i-1}) \cdot [w_\alpha] + [w_{\alpha,i-1}].$$

   (b) Let x_i, y_i be the input values for the multiplication gate whose output wire has index i, and let z_i be its output wire value. Then, x_i can be written as $\sum_{j=1}^{i-1} \beta_j \cdot z_j$ for some coefficients $\beta_1, \ldots, \beta_{i-1} \in \mathbb{F}$ defined by the circuit C'. y_i can be written similarly as $\sum_{j=1}^{i-1} \gamma_j \cdot z_j$. Then, the parties locally compute

   $$[z_i] = \sum_{j_1=1}^{i-1} \sum_{j_2=1}^{i-1} \beta_{j_1} \gamma_{j_2} [z_{j_1} \cdot z_{j_2}].$$

   (c) The parties locally compute their shares of $[z_i + w_i]$ and send them to P_{king}. P_{king} then reconstructs the secret $z_i + w_i$ and sends it to all the parties. The parties then locally compute $[\![z_i]\!] = (z_i + w_i) - [\![w_i]\!]$.

   After evaluating all the multiplication gates, the parties obtain $[\![z_1]\!], \ldots, [\![z_m]\!]$ for $m = m_I + m_O + |C|$. Since all the wire values in C' can be written as linear combinations of $z_1, \ldots, z_m$, the parties can locally compute the authenticated sharing for all of them.

Figure 20: Online evaluation protocol of Π_{turbo}.

Verification. After the evaluation, the parties have obtained an authenticated sharing for all the wire values in C', together with public values $z_i + w_i$ for each $i = 1, \ldots, m$. Since outputs of multiplication gates are authenticated, we only need to verify the correctness of multiplications ($\mathcal{S}_{\mathsf{check}}$ contains all outputs for multiplication gates in C').

In particular, for each multiplication gate with inputs $x_i = \sum_{j=1}^{i-1} \beta_j \cdot z_j, y_i = \sum_{j=1}^{i-1} \gamma_j \cdot z_j$, we can regard that the parties as using a triple $[\![a_i]\!] = \sum_{j=1}^{i-1} \beta_j \cdot [\![w_j]\!], [\![b_i]\!] = \sum_{j=1}^{i-1} \gamma_j \cdot [\![w_j]\!], [c_i] = \sum_{j_1=1}^{i-1} \sum_{j_2=1}^{i-1} \beta_{j_1} \gamma_{j_2} [w_{j_1,j_2}]$ to evaluate the multiplication gate.

Then, the parties have obtained $x_i + a_i, y_i + b_i$ for each multiplication gate with output z_i, which is the same as in the evaluation phase Π_{eval} for our main protocol Π_{main}. Furthermore, each $[a_i \cdot b_j]$ is still a linear combination of $\{[w_{\alpha,\beta}]\}_{\alpha,\beta \in \{1,\ldots,m\}}$, so they can be computed by the parties locally. In other words, we can regard the multiplication gates as being evaluated using a single length-$|C|$ tensor product. Therefore, the correctness of the evaluation can be verified in the same way as in Π_{ver}. We refer the reader to Section E for a detailed description of the protocol Π_{turbover} for the verification phase of Π_{turbo}.

Protocol Description. Finally, we provide the protocol Π_{turbo} as follows in Figure 21.

Protocol Π_{turbo}

1. **Preprocessing.** Let m_O be the number of output wires in C. The parties invoke $\mathcal{F}_{\mathsf{Turboprep}}$ with inputs $(\mathsf{Prep}, m_R, (I_1, \ldots, I_n), 1, m_C)$ to $\mathcal{F}_{\mathsf{Turboprep}}$ and receive the preprocessing data from $\mathcal{F}_{\mathsf{prep}}$, where $m_R = 2(\lambda - 1) \cdot \lceil \log_\lambda |C| \rceil$, each I_j is set to the total number of input/output wires attached to C_j in circuit C, and $m_C = |C|$.
2. **Evaluation.** The parties run $\Pi_{\mathsf{turboeval}}$.
3. **Verification.** The parties run Π_{turbover}.
4. **Output Reconstruction.** For each output wire with value v of C', the parties run Π_{Open} on $[\![v]\!]$. Then, the parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ on all the opened output values. After that, all the parties send each output wire value to the designated output party. Each party then checks whether the received outputs are all the same. If not, the party aborts the protocol. Otherwise, he locally computes his output of C from his output of C' and the masks he inputs to C'.

Figure 21: The protocol achieving $2n$ amortized communication cost per multiplication gate.

Theorem 2 *In the* $\{\mathcal{F}_{\mathsf{Turboprep}}, \mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}\}$*-hybrid model,* Π_{turbo} *is a secure multiparty computation protocol against* $\mathcal{A}$*.*

The security proof for Π_{turbo} is similar to that of Π_{main}. We refer the readers to Section F for a proof sketch. Clearly, except for the input authentication step, output reconstruction step, and the evaluation step in $\Pi_{\mathsf{turboeval}}$, the communication cost of all other steps is equal to the cost of Π_{main} in the case $k = |C|$, which is $O(n\lambda \log_\lambda |C|) + \mathsf{poly}(n, \kappa) = o(|C|)$ by taking $\lambda = 2$. Since the parties only need to authenticate the output wire values of multiplication gates, we save an authentication process per multiplication gate compared to Π_{main}. In addition, the output gates are authenticated automatically after evaluating the multiplications, so another $2m_O n$ communication cost is saved for authenticating them. In conclusion, the amortized communication of Π_{turbo} is $3n$ per input wire, $2n$ per multiplication gate, and $5n$ per output wire.

References

[BCCD23] Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 567–601, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

[BCG+19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.

[BCG+20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 387–416, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.

[BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 896–912, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[BCS19] Carsten Baum, Daniele Cozzo, and Nigel P. Smart. Using TopGear in overdrive: A more efficient ZKPoK for SPDZ. In Kenneth G. Paterson and Douglas Stebila, editors, *SAC 2019: 26th Annual International Workshop on Selected Areas in Cryptography*, volume 11959 of *Lecture Notes in Computer Science*, pages 274–302, Waterloo, ON, Canada, August 12–16, 2019. Springer, Cham, Switzerland.

[BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[BGIN20] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Efficient fully secure computation via distributed zero-knowledge proofs. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 244–276, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

[BGIN21] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Sublinear GMW-style compiler for MPC with preprocessing. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 457–485, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[BGIN22] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Secure multiparty computation with sublinear preprocessing. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 427–457, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 1–10. ACM, 1988.

[BNO19] Aner Ben-Efraim, Michael Nielsen, and Eran Omri. Turbospeedz: Double your online SPDZ! Improving SPDZ using function dependent preprocessing. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *ACNS 2019: 17th International Conference on Applied Cryptography and Network Security*, volume 11464 of *Lecture Notes in Computer Science*, pages 530–549, Bogota, Colombia, June 5–7, 2019. Springer, Cham, Switzerland.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 11–19. ACM, 1988.

[CGH+18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 34–64, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

[DKL+13] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In Jason Crampton, Sushil Jajodia, and Keith Mayes, editors, *ESORICS 2013: 18th European Symposium on Research in Computer Security*, volume 8134 of *Lecture Notes in Computer Science*, pages 1–18, Egham, UK, September 9–13, 2013. Springer Berlin Heidelberg, Germany.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *Advances in Cryptology – CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590, Santa Barbara, CA, USA, August 19–23, 2007. Springer Berlin Heidelberg, Germany.

[DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.

[EGPS22] Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. TurboPack: Honest majority MPC with constant online communication. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 951–964, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

[GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.

[GLM+24] Vipul Goyal, Junru Li, Ankit Kumar Misra, Rafail Ostrovsky, Yifan Song, and Chenkai Weng. Dishonest majority constant-round MPC with linear communication from DDH. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part VI*, volume 15489 of *Lecture Notes in Computer Science*, pages 167–199, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.

[GLO+21] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. ATLAS: Efficient and scalable MPC in the honest majority setting. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 244–274, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred V. Aho, editor, *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA*, pages 218–229. ACM, 1987.

[GS20] Vipul Goyal and Yifan Song. Malicious security comes free in honest-majority MPC. Cryptology ePrint Archive, Report 2020/134, 2020.

[KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 830–842, Vienna, Austria, October 24–28, 2016. ACM Press.

[KPR18] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 158–189, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

[LLX+26] Zhe Li, Hongqing Liu, Chaoping Xing, Yizhou Yao, and Chen Yuan. Faster pseudorandom correlation generators via walsh-hadamard transform. Cryptology ePrint Archive, Paper 2026/196, 2026.

[LN17] Yehuda Lindell and Ariel Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 259–276, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

[LOS14] Enrique Larraia, Emmanuela Orsini, and Nigel P. Smart. Dishonest majority multi-party computation for binary circuits. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part II*, volume 8617 of *Lecture Notes in Computer Science*, pages 495–512, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.

[RB89] Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In David S. Johnson, editor, *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*, pages 73–85. ACM, 1989.

[RS22] Rahul Rachuri and Peter Scholl. Le mans: Dynamic and fluid MPC for dishonest majority. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 719–749, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.

[WYKW21] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.

[Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3-5 November 1982*, pages 80–91. IEEE Computer Society, 1982.

A Security Proof for the Preprocessing Protocol

We prove the security of Π_{prep} by constructing an ideal adversary Sim. Then we will show that the output in the ideal world is indistinguishable from that in the real world against the adversary $\mathcal{A}$ using hybrid arguments. Since the parties do not have inputs in $\mathcal{F}_{\mathsf{prep}}$, we can assume that only one party P_h is honest in the protocol execution of Π_{prep}. If another party is honest, Sim just needs to honestly emulate him to run the protocol. Without loss of generality, we assume that P_{king} is corrupted. The ideal adversary Sim is provided below in Figure 22.

Simulator Sim

Init: Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to receive Δ_i from each corrupted party P_i. During the emulation of $\mathcal{F}_{\mathsf{nVOLE}}$, if a successful global key query is received from a corrupted party, Sim aborts the simulation. At the beginning of the simulation, Sim sets $\mathsf{TripCheck} = \mathsf{ErrorCheck} = \mathsf{MACCheck} = 0$.

Random Values:

1. Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to receive Extend and $s_r^{(i)}$ from each corrupted party P_i.
2. For each pair of parties (P_h, P_j), Sim receives $\boldsymbol{v}_h^{(j)}, \boldsymbol{w}_h^{(j)}$ from $\mathcal{A}$. For each pair of corrupted parties P_i and P_j, Sim receives $\boldsymbol{w}_j^{(i)}$ from $\mathcal{A}$ and computes $\boldsymbol{v}_i^{(j)}$ honestly.
3. For each pair of parties (P_h, P_j), if Sim receives a set I from $\mathcal{A}$, Sim samples a random element in $S \backslash I$ as $s_r^{(h)}$. Then Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to send abort and $s_r^{(h)}$ to P_j, sends abort to $\mathcal{F}_{\mathsf{prep}}$, and aborts the protocol on behalf the honest party. After the simulation completes, Sim outputs the adversary's view.
4. Sim faithfully emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to compute its outputs to corrupted parties. Then, Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to send the output $(s_r^{(i)}, (\boldsymbol{M}_j^{(i)}, \boldsymbol{K}_j^{(i)})_{j \neq i})$ to each corrupted party P_i.
5. Sim computes the shares of each $[\![r]\!]$ of corrupted parties and sends them to $\mathcal{F}_{\mathsf{prep}}$.
6. Whenever $(\mathsf{guess}, \boldsymbol{\Delta}')$ is received from $\mathcal{A}$ while Sim is emulating $\mathcal{F}_{\mathsf{nVOLE}}$, Sim sends abort to $\mathcal{F}_{\mathsf{prep}}$, receives Δ from $\mathcal{F}_{\mathsf{prep}}$, and computes the honest party's share of $[\Delta]$ based on Δ and the corrupted parties' shares. If $\boldsymbol{\Delta} = (\Delta^{(1)}, \ldots, \Delta^{(n)}) = \boldsymbol{\Delta}'$, Sim aborts the simulation. Otherwise, Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ to send $(\mathsf{abort}, \boldsymbol{\Delta})$ to $\mathcal{A}$ and follows the protocol to abort the protocol on behalf of the honest party. After the simulation completes, Sim outputs the adversary's view.

Input Masks: The same as the simulation of **Random Values**.

Authenticated Triples: Sim runs $\mathsf{Sim}_{\mathsf{auth}}$ (Figure 23).

Tensor Products: Sim runs $\mathsf{Sim}_{\mathsf{tens}}$ (Figure 24).

Figure 22: The simulator for Π_{prep}.

Simulator $\mathsf{Sim}_{\mathsf{auth}}$

– *Setup:*

1. Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ 5 times in the same way as above to generate seeds $s_a^{(i)}, s_b^{(i)}, s_a^{(i)'}, s_\ell^{(i)}, s_\ell^{(i)'}$ for each corrupted party P_i.
2. For the honest party P_h and each corrupted party P_j, Sim receives $\tilde{s}_a^{(j)}, \tilde{s}_b^{(j)}$ and the output to P_j from him. Sim emulates $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ to send P_j's outputs back to him and extract $\boldsymbol{u}_{j,h}, \boldsymbol{v}_{j,h}$ from the output.
3. For the honest party P_h and each corrupted party P_j, Sim receives $\tilde{s}_a^{(j)'}, \tilde{s}_b^{(j)'}$ and the output to P_j from him. Sim emulates $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ to send P_j's outputs back to him and extract $\boldsymbol{u}'_{j,h}, \boldsymbol{v}'_{j,h}$ from the output.
4. Sim samples a random element $\alpha \in \mathbb{F}^*$.

– To get the η-th authenticated triple $([\![a]\!], [\![b]\!], [\![c]\!])$:

1. Sim sets δ_a to the η-th bit of $\sum_{j \neq h} \mathsf{Expand}_{m_A}(\tilde{s}_b^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_{m_A}(s_b^{(j)})$, sets δ_b to the η-th bit of $\sum_{j \neq h} \mathsf{Expand}_{m_A}(\tilde{s}_a^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_{m_A}(s_a^{(j)})$, sets $\delta_{a'}$ to the η-th bit of $\sum_{j \neq h} \mathsf{Expand}_{m_A}(\tilde{s}_b^{(j)'}) - \sum_{j \neq h} \mathsf{Expand}_{m_A}(s_b^{(j)})$, and sets $\delta_{b'}$ to the η-th bit of

$\sum_{j\neq h} \mathsf{Expand}_{m_A}(\tilde{s}_a^{(j)'}) - \sum_{j\neq h} \mathsf{Expand}_{m_A}(s_a^{(j)'})$.

2. Sim computes the corrupted parties' shares of $[\ell]$ using each corrupted party P_j's $s_\ell^{(j)}$. Then Sim computes the corrupted parties' shares of $[c]$ by simulating $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ for each pair of corrupted parties (P_i, P_j) assuming that both P_i, P_j correctly send their seeds. Then Sim follows the protocol to compute all the corrupted parties' shares of $[\ell + c]$.
3. Sim samples a random element in $\mathbb{F}$ as P_h's share of $[\ell + c]$. Then Sim sends it to P_{king} on behalf of P_h.
4. Sim computes $\ell + c$ based on all the parties' shares of $[\ell + c]$. Then Sim receives $\overline{\ell + c}$ from P_{king} and sets $\delta_\ell = \overline{\ell + c} - (\ell + c)$.
5. Sim follows the protocol to compute corrupted parties' shares of $[\![c]\!] = \overline{\ell + c} - [\![\ell]\!]$.
6. Repeat step 2-5 on ℓ', c' instead of ℓ, c and compute the corresponding $\delta_{\ell'}$.
7. If it holds that $\delta_a = \delta_b = \delta_\ell = \delta_{a'} = \delta_{b'} = \delta_{\ell'} = 0$, then Sim sets $\tilde{\tau_\eta} = 0$. Otherwise, Sim samples $a, b, a' \in \mathbb{F}$ randomly. Let the sum of corrupted parties' shares of $[a], [b], [a']$ be $\alpha_\eta, \beta_\eta, \alpha'$. Sim sets $\delta_c = \delta_\ell - \delta_a \cdot \alpha_\eta - \delta_b \cdot \beta_\eta$ and $\delta_{c'} = \delta_{\ell'} - \delta_{a'} \cdot \alpha'_\eta - \delta_{b'} \cdot \beta_\eta$. Sim then computes

$$\tilde{\tau_\eta} = \alpha^\eta \cdot (\delta_a \cdot a + \delta_b \cdot b + \delta_c) - (\delta_{a'} \cdot a' + \delta_{b'} \cdot b + \delta_{c'}).$$

Here $\tilde{\tau_\eta}$ is the value such that $[\tilde{\tau_\eta} \cdot \Delta]$ is shared among all the parties, and the sharing will be used in the second execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ while doing verification.

– *Verification:*

1. Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send α to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.
2. For each authenticated triple (indexed η), Sim follows the protocol to compute each corrupted party's share of $[\![e]\!]$ and obtains his share of $[e]$ from it. Then:
   * If it holds that $\delta_a = \delta_b = \delta_\ell = \delta_{a'} = \delta_{b'} = \delta_{\ell'} = 0$, Sim samples a random elements in $\mathbb{F}$ as P_h's share of $[e]$ and reconstruct e based on all the parties' shares of $[e]$. Then Sim sends P_h's share of $[e]$ to P_{king} on behalf of P_h.
   * Otherwise, Sim computes $e = \alpha^\eta \cdot a - a'$ and computes P_h's share of $[e]$ based on the secret and corrupted parties' shares. Then Sim sends P_h's share of $[e]$ to P_{king} on behalf of P_h. Sim sets $\mathsf{TripCheck} = 1$.

   Sim then receives e from P_{king} and checks whether it is correctly sent. If not, Sim sets $\mathsf{MACCheck} = 1$.
3. Sim computes $\tilde{\tau'} = \sum_{\eta=1}^{m_A} \tilde{\tau_\eta}$. If $\mathsf{TripCheck} = 1$ but $\tilde{\tau'} = 0$, Sim aborts the simulation.
4. If $\mathsf{TripCheck} = 1$ or $\mathsf{MACheck} = 1$, Sim samples all P_h's shares of $[a], [b], [a']$ for each triple (where the shares have not been sampled) based on the corresponding shares of $[e]$. Then, Sim follows the protocol to compute P_h's share of $[\tau']$. Otherwise, Sim follows the protocol to compute corrupted parties' shares of $[\tau']$ and computes P_h's share based on their shares and the secret $\tau' = 0$.
5. Sim sends P_h's share of $[\tau']$ to P_{king} and then receives τ' from P_{king}. If $\tau' \neq \tilde{\tau'}$, Sim sets $\mathsf{MACCheck} = 1$. If $\tau' \neq 0$, Sim aborts the protocol on behalf of P_h. After the simulation completes, Sim outputs the adversary's view.
6. Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as follows:
   (a) Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send γ to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.
   (b) Sim follows the protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to compute corrupted parties' shares of $[\sigma]$.
   (c) Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$ from each corrupted party P_i.
   (d)
   * If $\mathsf{MACCheck} = 0$, Sim computes the P_h's share of $[\sigma]$ based on the corrupted parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \sigma_i = 0$.
   * If $\mathsf{MACCheck} = 1$, Sim sends abort to $\mathcal{F}_{\mathsf{prep}}$ and receives Δ from $\mathcal{F}_{\mathsf{prep}}$. For each value A_i opened in Π_{Open} protocol:

- Sim computes $\Delta \cdot A_i$ and computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. Each A_i is reconstructed based on all the parties' shares, where the corrupted parties' shares are computed by following the protocol. In particular, the opened value τ' is set to $\tilde{\tau}'$.
- Sim follows the protocol to compute the P_h's share σ_h of $[\sigma]$.

(e) For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties. For the honest party P_h, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_h, \mathsf{mid}_{\sigma_h})$ to all the corrupted parties.

(f) For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\sigma_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties.

(g) Sim follows the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(h) If $\mathsf{MACCheck} = 1$, Sim aborts the simulation.

Figure 23: The simulator for Π_{auth}.

Simulator $\mathsf{Sim}_{\mathsf{tens}}$

1. Sim emulates $\mathcal{F}_{\mathsf{nVOLE}}$ 2 times in the same way as above to generate seeds $s_a^{(i)}, s_b^{(i)}$ for each corrupted party P_i.
2. For the honest party P_h and each corrupted party P_j, Sim emulates $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ to receive $\tilde{s}_a^{(j)}, \tilde{s}_b^{(j)}$ and $\boldsymbol{u}_{j,h}, \boldsymbol{v}_{j,h}$ from P_j and send $\boldsymbol{u}_{j,h}, \boldsymbol{v}_{j,h}$ back to P_j.
3. Sim sets $\boldsymbol{\delta}_a = \left(\sum_{j \neq h} \mathsf{Expand}_m(\tilde{s}_b^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_m(s_b^{(j)})\right)$ and $\boldsymbol{\delta}_b = \left(\sum_{j \neq h} \mathsf{Expand}_m(\tilde{s}_a^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_m(s_a^{(j)})\right)$. If either of them contains a non-zero entry, Sim sets $\mathsf{ErrorCheck} = 1$.

Verification:

1. For each length-k tensor product with index α, Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send r_α to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view. Sim then follows the protocol to expand the seed.
2. Sim follows the protocol to compute each corrupted party P_j's shares of $[\![x_\alpha]\!], [\![y_\alpha]\!], [v_\alpha]$, corresponding to each length-k tensor product. Then, Sim samples random elements as P_h's shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ and sends them to P_{king} on behalf of P_h. Sim then receives $\overline{x_\alpha + x'_\alpha}, \overline{y_\alpha + y'_\alpha}$ from P_{king}.
3. For each length-k tensor product with index α, Sim locally computes $x_\alpha + x'_\alpha, y_\alpha + y'_\alpha$ with the corrupted parties' shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ and P_h's shares of them sampled in the last step. Then, Sim checks whether $\overline{x_\alpha + x'_\alpha} = x_\alpha + x'_\alpha$ and $\overline{y_\alpha + y'_\alpha} = y_\alpha + y'_\alpha$. If not, Sim sets $\mathsf{MACCheck} = 1$.
4. Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as follows:
   (a) Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send γ to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.
   (b) Sim follows the protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to compute corrupted parties' shares of $[\sigma]$.
   (c) Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$ from each corrupted party P_i.
   (d)
   - If $\mathsf{MACCheck} = 0$, Sim computes the P_h's share of $[\sigma]$ based on the corrupted parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \sigma_i = 0$.
   - If $\mathsf{MACCheck} = 1$, Sim sends abort to $\mathcal{F}_{\mathsf{prep}}$ and receives Δ from $\mathcal{F}_{\mathsf{prep}}$. For each value A_i opened in Π_{Open} protocol:
     * Sim computes $\Delta \cdot A_i$ and computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. Each A_i is reconstructed based on all the parties' shares, where the corrupted parties' shares are computed by following the protocol.
     * Sim follows the protocol to compute the P_h's share σ_h of $[\sigma]$.

(e) For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties. For the honest party P_h, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_h, \mathsf{mid}_{\sigma_h})$ to all the corrupted parties.

(f) For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\sigma_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties.

(g) Sim follows the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(h) If $\mathsf{MACCheck} = 1$, Sim aborts the simulation.

5. Sim follows the protocol to compute the corrupted parties' shares of $[\tau]$, where the seeds they send to $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ running among two corrupted parties are all set to the seeds Sim sends to them on behalf of $\mathcal{F}_{\mathsf{nVOLE}}$.
6. Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \tau_i, \mathsf{mid}_{\tau_i})$ from each corrupted party P_i. Then
   - If $\mathsf{ErrorCheck} = 0$, Sim computes the P_h's share of $[\tau]$ based on the corrupted parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \tau_i = 0$.
   - If $\mathsf{ErrorCheck} = 1$, let $\boldsymbol{\delta}_a = (\boldsymbol{\delta}_{a,1}, \ldots, \boldsymbol{\delta}_{a,m_T}) \in (\mathbb{F}^k)^{m_T}$ and $\boldsymbol{\delta}_b = (\boldsymbol{\delta}_{b,1}, \ldots, \boldsymbol{\delta}_{b,m_T}) \in (\mathbb{F}^k)^{m_T}$. Denote P_h's shares of $[x'_\alpha], [y'_\alpha]$ by $x'_{h,\alpha}, y'_{h,\alpha}$, which are the γ_α-th bits of $\mathsf{Expand}_m(s_a^{(h)}), \mathsf{Expand}_m(s_b^{(h)})$ respectively. Let the γ_α-th bits of $\boldsymbol{\delta}_a, \boldsymbol{\delta}_b$ be $\delta_{x'_\alpha}, \delta_{y'_\alpha}$ respectively. Sim randomly samples $x'_{h,\alpha}, y'_{h,\alpha} \in \mathbb{F}$ and then computes $\boldsymbol{a}_\alpha^{(h)}, \boldsymbol{b}_\alpha^{(h)}$ from P_h's shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$. For the α-th length-k tensor product, Sim computes
   $$\delta_\alpha = \delta_{x'_\alpha} x'_{h,\alpha} + \delta_{y'_\alpha} y'_{h,\alpha} - \langle \boldsymbol{r}_a \otimes \boldsymbol{r}_b, (\boldsymbol{a}_\alpha^{(h)} \otimes \boldsymbol{\delta}_{a,\alpha} + \boldsymbol{\delta}_{b,\alpha} \otimes \boldsymbol{b}_\alpha^{(h)}) \rangle,$$
   where $\boldsymbol{r}_a = (r_a^{(1)}, \ldots, r_a^{(k)})$ and $\boldsymbol{r}_b = (r_b^{(1)}, \ldots, r_b^{(k)})$. Sim then computes the P_h's share of $[\tau]$ based on the corrupted parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and the secret $\tau = \sum_{i=1}^{n} \tau_i = \delta_1 + \cdots + \delta_{m_T}$.
7. For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\tau_i})$ to all the corrupted parties. For the honest party P_h, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_h, \mathsf{mid}_{\tau_h})$ to all the corrupted parties.
8. For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\tau_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\tau_i, \mathsf{mid}_{\tau_i})$ to all the corrupted parties.
9. Sim follows the protocol to check whether $\sum_{i=1}^{n} \tau_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.
10. If $\mathsf{ErrorCheck} = 1$, Sim aborts the simulation. Otherwise, Sim follows the protocol to compute the corrupted parties' outputs of $\mathcal{F}_{\mathsf{prep}}$, where the seeds they send to $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ running among two corrupted parties are all set to the seeds Sim sends to them on behalf of $\mathcal{F}_{\mathsf{nVOLE}}$. Then, Sim sends the outputs to $\mathcal{F}_{\mathsf{prep}}$.
11. Sim outputs the adversary's view.

Figure 24: The simulator for Π_{tens}.

We construct the following hybrids:

$\mathbf{Hyb}_0$: In this hybrid, Sim runs the protocol honestly on behalf of the honest parties and communicates with $\mathcal{A}$. This corresponds to the real-world scenario.

$\mathbf{Hyb}_1$: In this hybrid, Sim additionally initializes $\mathsf{TripCheck} = \mathsf{ErrorCheck} = \mathsf{MACCheck} = 0$ at the beginning. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ have the same output distribution.

$\mathbf{Hyb}_2$: In this hybrid, for each seed s generated for the honest party by $\mathcal{F}_{\mathsf{nVOLE}}$, Sim doesn't generated it while emulating $\mathcal{F}_{\mathsf{nVOLE}}$. Instead, when Sim needs to compute an expansion function $\mathsf{Expand}_m(s)$ to expand it to m field elements on behalf of the honest party or $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$, Sim samples a uniformly random vector in $\mathbb{F}^m$ as the result $\mathsf{Expand}_m(s)$. Note that each seed generated by $\mathcal{F}_{\mathsf{nVOLE}}$ isn't used anywhere without expanding it, so this hybrid is well-defined. We argue that $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are computationally indistinguishable.

For the sake of contradiction, assume that $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are computationally distinguishable. By the standard hybrid argument, there exists a PPT algorithm $\mathcal{D}$ that can distinguish the output distributions before and after we change $\mathsf{Expand}_m(s)$ to a random vector for some seed s with a non-negligible probability. Now we construct a PPT algorithm $\mathcal{D}'$ that breaks the security of $\mathsf{Expand}_m(\cdot)$, which is modeled as a PRG.

Concretely, $\mathcal{D}'$ receives a string from the challenger, which is either the output of $\mathsf{Expand}_m(s)$ or a random vector. Then $\mathcal{D}'$ runs Sim with the change that $\mathsf{Expand}_m(s)$ is replaced by the string received from the challenger. Finally, $\mathcal{D}'$ uses $\mathcal{D}$ to distinguish whether the string received from the challenger is generated from $\mathsf{Expand}_m(s)$ or sampled randomly. Note that the advantage of $\mathcal{D}'$ is the same as that of $\mathcal{D}$, which breaks the security of the underlying PRG $\mathsf{Expand}_m(\cdot)$. This leads to a contradiction. Thus, the distributions of $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are computationally indistinguishable.

$\mathbf{Hyb}_3$: In this hybrid, while Sim is emulating $\mathcal{F}_{\mathsf{nVOLE}}$, if Sim receives a set I from $\mathcal{A}$, Sim aborts the simulation and sends a random seed in $S\backslash I$ to the adversary. This only changes the distribution when $s_r^{(h)}$ happens to be in I. During each simulation of the interaction between corrupted parties and $\mathcal{F}_{\mathsf{nVOLE}}$, $s_r^{(h)}$ isn't sampled before $\mathcal{A}$ sends I to Sim, the probability that $s_r^{(h)} \in I$ is $|I|/|S|$. Since $\mathcal{A}$ is a PPT adversary, $|I| = \mathsf{poly}(\lambda)$. However, $|S|$ is exponential in λ, so the probability is negligible. Thus, the distributions of $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are statistically close.

$\mathbf{Hyb}_4$: In this hybrid, while generating the authenticated triples, Sim doesn't generates a random vector as $\mathsf{Expand}(s_a^{(i)})$ for the honest party P_i while emulating $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$. Instead, Sim generates it when it is needed, i.e. when P_i needs to compute his share of each $[\ell + c]$ and send it to P_{king}. Similar for $\mathsf{Expand}(s_b^{(i)})$ and $\mathsf{Expand}(s_a^{(i)'})$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ have the same output distribution.

$\mathbf{Hyb}_5$: In this hybrid, while generating the η-th authenticated triple, Sim additionally sets δ_a to the η-th bit of $\sum_{j\neq h} \mathsf{Expand}_{m_A}(\tilde{s}_b^{(j)}) - \sum_{j\neq h} \mathsf{Expand}_{m_A}(s_b^{(j)})$, sets δ_b to the η-th bit of $\sum_{j\neq h} \mathsf{Expand}_{m_A}(\tilde{s}_a^{(j)}) - \sum_{j\neq h} \mathsf{Expand}_{m_A}(s_a^{(j)})$, sets $\delta_{a'}$ to the η-th bit of $\sum_{j\neq h} \mathsf{Expand}_{m_A}(\tilde{s}_b^{(j)'}) - \sum_{j\neq h} \mathsf{Expand}_{m_A}(s_b^{(j)})$, and sets $\delta_{b'}$ to the η-th bit of $\sum_{j\neq h} \mathsf{Expand}_{m_A}(\tilde{s}_a^{(j)'}) - \sum_{j\neq h} \mathsf{Expand}_{m_A}(s_a^{(j)'})$. Here $s_a^{(j)}$ is the seed generated when Sim is emulating $\mathcal{F}_{\mathsf{nVOLE}}$, and $\tilde{s}_a^{(j)}$ is received from P_j when emulating $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ between the honest party P_h and P_j. Similar for the seeds for generating b, a'. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ have the same output distribution.

$\mathbf{Hyb}_6$: In this hybrid, while generating the η-th authenticated triple, Sim doesn't compute the honest party P_h's share of each $[\ell + c]$ by himself. Instead, Sim generates a random field element as P_h's share of $[\ell + c]$ and then computes P_h's share of $[\ell]$ by $[\ell + c] - [c]$. P_h's share of $[\Delta \cdot \ell]$ is then computed from Δ, ℓ and corrupted parties' shares. Since P_h's share of $[\ell]$ is an entry from $\mathsf{Expand}(s_\ell^{(j)})$, which we have replaced the result by a uniformly random vector in $\mathbf{Hyb}_2$, it's a uniformly random element in $\mathbb{F}$. Therefore, P_j's share of $[\ell + c]$ is also a uniformly random variable in $\mathbb{F}$, so we only change the order of generating P_j's shares of $[\ell + c]$ and $[\ell]$ without changing their distributions. Thus, $\mathbf{Hyb}_6$ and $\mathbf{Hyb}_5$ have the same output distribution.

$\mathbf{Hyb}_7$: In this hybrid, while generating the η-th authenticated triple, let $\overline{\ell + c}$ be what Sim received from P_{king} after P_{king} receives the honest party's share of $[\ell + c]$, Sim additionally sets $\delta_\ell = \overline{\ell + c} - (\ell + c)$. Similarly, Sim also computes $\delta_{\ell'}$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_6$ have the same output distribution.

$\mathbf{Hyb}_8$: In this hybrid, while generating the authenticated triples, Sim does not generate the random value α while emulating $\mathcal{F}_{\mathsf{Coin}}$ during the verification. Instead, Sim generates it earlier while computing the triples. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_8$ and $\mathbf{Hyb}_7$ have the same output distribution.

$\mathbf{Hyb}_9$: In this hybrid, while generating the η-th authenticated triple, if it holds that $\delta_a = \delta_b = \delta_\ell = \delta_{a'} = \delta_{b'} = \delta_{\ell'} = 0$, then Sim sets $\tilde{\tau_\eta} = 0$. This means that we can regard the corrupted parties as invoking $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ honestly, and the values $\ell + c, \ell' + c'$ are opened correctly. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_9$ and $\mathbf{Hyb}_8$ have the same output distribution.

$\mathbf{Hyb}_{10}$: In this hybrid, while generating the η-th authenticated triple, if some of $\delta_a, \delta_b, \delta_\ell, \delta_{a'}, \delta_{b'}, \delta_{\ell'}$ is non-zero, Sim doesn't obtain the honest party's share of $[a], [b], [a']$ from the result of expanding the seeds from $\mathcal{F}_{\mathsf{nVOLE}}$. Instead, Sim randomly samples $a, b, a' \in \mathbb{F}$ and then computes the honest party's share of $[a], [b], [a']$ with a, b, a' and corrupted parties' shares. Since the honest party's shares of $[a], [b], [a']$ are all entries from vectors expanded by seeds generated by Sim while emulating $\mathcal{F}_{\mathsf{nVOLE}}$ which have been replaced by uniformly random vectors in $\mathbf{Hyb}_2$, a, b, a' are also uniformly random elements in $\mathbb{F}$. We only change the

order of generating a, b, a' and the honest party's shares of $[a], [b], [a']$ without changing their distributions. Thus, $\mathbf{Hyb}_{10}$ and $\mathbf{Hyb}_9$ have the same output distribution.

$\mathbf{Hyb}_{11}$: In this hybrid, while generating the η-th authenticated triple, if some of $\delta_a, \delta_b, \delta_\ell, \delta_{a'}, \delta_{b'}, \delta_{\ell'}$ is non-zero, let the sum of corrupted parties' shares of $[a], [b], [a']$ be $\alpha_\eta, \beta_\eta, \alpha'_\eta$. Sim additionally sets $\delta_c = \delta_\ell - \delta_a \cdot \alpha_\eta - \delta_b \cdot \beta$ and $\delta_{c'} = \delta_{\ell'} - \delta_{a'} \cdot \alpha'_\eta - \delta_{b'} \cdot \beta$. Sim additionally computes

$$\tilde{\tau_\eta} = \alpha^\eta \cdot (\delta_a \cdot a + \delta_b \cdot b + \delta_c) - (\delta_{a'} \cdot a' + \delta_{b'} \cdot b + \delta_{c'}).$$

Sim also sets $\mathsf{TripCheck} = 1$ in this case.

Now we explain why $\tilde{\tau_\eta}$ is shared among all the parties. Note that the additive error of the honest party's share of $[\Delta \cdot c]$ is δ_ℓ plus the error $c - a \cdot b$ on c and then multiplied with Δ, where the error $c - a \cdot b$ comes from the error of the honest party's share of $[c]$ computed from the output of $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$. P_h's share of $[a]$ is $a^{(h)} = a - \alpha_\eta$, and his share of $[b]$ is $b^{(h)} = b - \beta_\eta$. Each invocation of $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ between (P_h, P_j) generates additive shares of $a^{(h)} \cdot b^{(j)}$, where sending incorrect seeds by P_j causes an additive error δ_a on $b^{(j)}$. So the total error is $\delta_a \cdot (a - \alpha_\eta)$ from the invocations of (P_h, P_j). For the same reason, the error generated from invocations of (P_j, P_h) is $\delta_b \cdot (b - \beta)$. Thus, the additive error on the honest party's $[c]$ is $\delta_a \cdot (a - \alpha_\eta) + \delta_b \cdot (b - \beta_\eta)$. Let the corrupted parties' share of $[\![c]\!]$ computed by $\overline{c + \ell} - [\![\ell]\!]$, the additive errors on c and ℓ lead to an error

$$\Delta \cdot (\delta_a \cdot (a - \alpha_\eta) + \delta_b \cdot (b - \beta_\eta) + \delta_\ell) = \Delta \cdot (\delta_a \cdot a + \delta_b \cdot b - \delta_c)$$

on (the secret of) $[\Delta \cdot c]$, where $\delta_c = \delta_\ell - \delta_a \cdot \alpha_\eta - \delta_b \cdot \beta$. The error on $[\Delta \cdot \tau_\eta]$ is just the error on $\alpha_\eta \cdot [\Delta \cdot c] - [\Delta \cdot c']$, which is equal to

$$\Delta \cdot (\alpha^\eta \cdot (\delta_a \cdot a + \delta_b \cdot b + \delta_c) - (\delta_{a'} \cdot a' + \delta_{b'} \cdot b + \delta_{c'}))\,.$$

This matches $\Delta \cdot \tilde{\tau_\eta}$.

The additional computation doesn't affect the output distribution. Thus, $\mathbf{Hyb}_{11}$ and $\mathbf{Hyb}_{10}$ have the same output distribution.

$\mathbf{Hyb}_{12}$: In this hybrid, during the verification of authenticated triples, Sim additionally computes $\tilde{\tau'} = \sum_{\eta=1}^{m_A} \tilde{\tau_\eta}$. If $\mathsf{TripCheck} = 1$ but $\tilde{\tau'} = 0$, Sim aborts the simulation. Clearly, $[\Delta \cdot \tilde{\tau'}]$ is shared among the parties. This only changes the distribution when there is an error attached to a triple, while the final value τ' is zero.

Suppose that for a $\eta \in \{1, \ldots, m_A\}$, there is a non-zero element among $\delta_a, \delta_b, \delta_\ell, \delta_{a'}, \delta_{b'}, \delta_{\ell'}$. Then, if

$$\tilde{\tau_\eta} = \alpha^\eta \cdot (\delta_a \cdot a + \delta_b \cdot b + \delta_c) - (\delta_{a'} \cdot a' + \delta_{b'} \cdot b + \delta_{c'})$$

is a zero polynomial in α, then it must holds that at least one of $\delta_a, \delta_b, \delta_{a'}$ is nonzero. In this case, from the randomness of a, b, a', eithor $\delta_a \cdot a + \delta_b \cdot b + \delta_c$ or $\delta_{a'} \cdot a' + \delta_{b'} \cdot b + \delta_{c'}$ is random element in $\mathbb{F}$, which equals zero with only a negligible probability.

On the other hand, if $\tilde{\tau_\eta}$ is a non-zero polynomial in α, then $\tilde{\tau'}$ is also a non-zero polynomial in α. Note that α is sent to the adversary after the errors are determined, so the distribution only changes when α happens to be a root for the non-zero polynomial. Since the polynomial is of degree $m_A = \mathsf{poly}(\kappa)$, there are at most $\mathsf{poly}(\kappa)$ roots. Therefore, the probability that α is one of them is negligible. Thus, the distributions of $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ ar statistically close.

$\mathbf{Hyb}_{13}$: In this hybrid, during the verification of authenticated triples, for each triple with index η, if $\delta_a, \delta_b, \delta_\ell, \delta_{a'}, \delta_{b'}, \delta_{\ell'} = 0$, Sim doesn't follow the protocol to compute the honest party P_h's share of $[\![e]\!]$ and get his share of $[e]$. Instead, Sim samples P_h's share of $[e]$ randomly in $\mathbb{F}$ and computes his share of $[a']$ by $\alpha^\eta \cdot [a] - [e]$. Since P_j's share of $[a']$ is an element from the expansion result of a seed generated by $\mathcal{F}_{\mathsf{nVOLE}}$ which has been replaced by a uniformly random vector in $\mathbf{Hyb}_2$, the share is uniformly random in $\mathbb{F}$. Therefore, P_h's share of $[e] = \alpha^\eta \cdot [a] - [a']$ is also uniformly random in $\mathbb{F}$. Since P_h's share of $[a']$ is not used in the previous steps of the simulation, we just change the order of generating P_h's shares of $[a']$ and $[e]$ without changing their distributions. Thus, $\mathbf{Hyb}_{13}$ and $\mathbf{Hyb}_{12}$ have the same output distribution.

$\mathbf{Hyb}_{14}$: In this hybrid, during the verification of authenticated triples, for each triple with index η, if some of $\delta_a, \delta_b, \delta_\ell, \delta_{a'}, \delta_{b'}, \delta_{\ell'}$ is non-zero, Sim doesn't follow the protocol to compute the honest party P_h's

share of $[\![e]\!]$ and get his share of $[e]$. Instead, Sim first computes $e = \alpha \cdot a - a'$ and then computes P_h's share of $[e]$ based on the secret and the corrupted parties' shares. We only change the order of generating e and P_h's share of $[e]$ without changing their distributions. Thus, $\mathbf{Hyb}_{14}$ and $\mathbf{Hyb}_{13}$ have the same output distribution.

$\mathbf{Hyb}_{15}$: In this hybrid, during the verification of authenticated triples, for each triple with index η, if P_{king} does not correctly send e, Sim additionally sets $\mathsf{MACCheck} = 1$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{15}$ and $\mathbf{Hyb}_{14}$ have the same output distribution.

$\mathbf{Hyb}_{16}$: In this hybrid, during the verification of authenticated triples, if $\mathsf{TripCheck} = \mathsf{MACCheck} = 0$, Sim does not follow the protocol to compute P_h's shares of $[\tau']$. Sim follows the protocol to compute corrupted parties' shares of $[\tau']$ and then computes P_h's share of $[\tau']$ based on the secret $\tau' = 0$ and corrupted parties' shares. Note that in this case, the corrupted parties generate valid triples and honestly compute $[\tau']$. Therefore, $\tau' = 0$ must hold, so we only change the way of computing P_h's share of $[\tau']$ without changing its distribution. Thus, $\mathbf{Hyb}_{16}$ and $\mathbf{Hyb}_{15}$ have the same output distribution.

$\mathbf{Hyb}_{17}$: In this hybrid, during the verification of authenticated triples, if τ' is not opened correctly by P_{king}, Sim additionally sets $\mathsf{MACCheck} = 1$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{17}$ and $\mathbf{Hyb}_{16}$ have the same output distribution.

$\mathbf{Hyb}_{18}$: In this hybrid, during the verification of authenticated triples, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 0$, Sim does not follow the protocol to compute P_h's share of $[\sigma]$. Instead, Sim computes the P_h's share of $[\sigma]$ based on the corrupted parties' shares and $\sum_{i=1}^{n} \sigma_i = 0$. Note that when $\mathsf{MACCheck} = 0$, all the opened values are opened correctly via Π_{Open}. In this case, it must hold that $[\sigma] = [0]$, so we only change the way of generating P_h's share of $[\sigma]$ without changing its distribution. Thus, $\mathbf{Hyb}_{18}$ and $\mathbf{Hyb}_{17}$ have the same output distribution.

$\mathbf{Hyb}_{19}$: In this hybrid, during the verification of authenticated triples, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 1$, Sim doesn't directly compute the honest party P_h's share of $[\Delta \cdot A_i]$ for each opened value A_i. Instead, Sim first computes each $\Delta \cdot A_i$, and then Sim computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. We just change the way the honest party's share of each $[\Delta \cdot A_i]$ is generated without changing its distribution. Thus, $\mathbf{Hyb}_{19}$ and $\mathbf{Hyb}_{18}$ have the same output distribution.

$\mathbf{Hyb}_{20}$: In this hybrid, during the verification of authenticated triples, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, after following the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$, Sim aborts the simulation if $\mathsf{MACCheck} = 1$. The only difference between $\mathbf{Hyb}_{20}$ and $\mathbf{Hyb}_{19}$ is that the corrupted parties may not open all the values of SPDZ sharing checked in this execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ correctly, but the verification of MACs may pass.

We suppose that $A_1, \ldots, A_m$ are opened with additive errors $\delta^{(1)}, \ldots, \delta^{(m)}$, and the sum of all the committed shares $[\sigma]$ of corrupted parties is with an additive error δ_σ. Then

$$\begin{aligned}\sigma =& \sum_{i\in[1,n], i\neq h} \left(\sum_{\alpha=1}^{m} \chi^\alpha \cdot (\Delta \cdot A_\alpha)^{(i)} - \Delta_i \cdot \sum_{\alpha=1}^{m} \chi^\alpha \cdot A_\alpha \right) + \delta_\sigma \\ &+ \left(\sum_{\alpha=1}^{m} \chi^\alpha \cdot (\Delta \cdot A_\alpha)^{(h)} - \Delta_h \cdot \sum_{\alpha=1}^{m} \chi^\alpha \cdot (A_\alpha + \delta^{(\alpha)}) \right) \\ =& \delta_\sigma + \Delta_h \cdot \sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)}.\end{aligned}$$

Here $(\Delta \cdot A_\alpha)^{(i)}$ is used to denote P_i's share of $[\Delta \cdot A_\alpha]$. Since Δ_h is not used in computing any message sent to the corrupted parties at this moment, $\Delta_h \cdot \sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)}$ is a random element in $\mathbb{F}$ as long as $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)}$ is non-zero. When $(\delta^{(1)}, \ldots, \delta^{(m)})$ is a nonzero vector, since a non-zero degree-m polynomial only has at most m zeros, so $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)} = 0$ only happens with at most $m/|\mathbb{F}|$ probability. Since $m = \mathsf{poly}(n, \kappa)$, the probability is negligible. On the other hand, in the case where $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)} = 0$, σ is opened to a random element in $\mathbb{F}$, which equals 0 with $1/|\mathbb{F}|$ probability, which is also negligible.

Thus, the distributions of $\mathbf{Hyb}_{20}$ and $\mathbf{Hyb}_{19}$ are statistically close.

$\mathbf{Hyb}_{21}$: In this hybrid, while generating the length-k tensor products, Sim doesn't generates a random vector as $\mathsf{Expand}_m(s_a^{(h)})$ while emulating $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ (similar for $\mathsf{Expand}_m(s_b^{(h)})$). Instead, Sim generates it when

it is needed, i.e., when P_i needs to compute his shares of $[\![x_\alpha + x'_\alpha]\!], [\![y_\alpha + y'_\alpha]\!]$ for each length-k tensor product indexed α. Similar for $\mathsf{Expand}(s_b^{(h)})$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_{21}$ and $\mathbf{Hyb}_{20}$ have the same output distribution.

$\mathbf{Hyb}_{22}$: In this hybrid, while generating the length-k tensor products, Sim additionally sets

$$\boldsymbol{\delta}_a = \left(\sum_{j \neq h} \mathsf{Expand}_m(\tilde{s}_b^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_m(s_b^{(j)}) \right)$$

and

$$\boldsymbol{\delta}_b = \left(\sum_{j \neq h} \mathsf{Expand}_m(\tilde{s}_a^{(j)}) - \sum_{j \neq h} \mathsf{Expand}_m(s_a^{(j)}) \right).$$

Here $s_a^{(h)}$ is the seed generated when Sim is emulating $\mathcal{F}_{\mathsf{nVOLE}}$, and $\tilde{s}_a^{(h)}$ is received from P_j when emulating $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ between P_h and P_j. If either of $\boldsymbol{\delta}_a, \boldsymbol{\delta}_b$ contains a non-zero entry, Sim additional sets $\mathsf{ErrorCheck} = 1$. The additional computations do not affect the output distribution. Thus, $\mathbf{Hyb}_{22}$ and $\mathbf{Hyb}_{21}$ have the same output distribution.

$\mathbf{Hyb}_{23}$: In this hybrid, during the verification of tensor products, for each length-k tensor product (with index α), Sim does not generates P_h's shares of $[\![x'_\alpha]\!], [\![y'_\alpha]\!]$ and then follows the protocol to compute $[\![x_\alpha + x'_\alpha]\!], [\![y_\alpha + y'_\alpha]\!]$. Instead, Sim samples random elements as P_h's shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ randomly and then use them to compute his shares of $\boldsymbol{a}_\alpha^{(h)}, \boldsymbol{b}_\alpha^{(h)}$ and samples P_i's shares of $[x'_\alpha], [y'_\alpha]$. P_h's shares of $[\Delta \cdot x'_\alpha], [\Delta \cdot y'_\alpha]$ are then computed accordingly from $\Delta, x'_\alpha, y'_\alpha$. Since P_h's shares of $[x'_\alpha], [y'_\alpha]$ are sampled randomly and are added into P_h's shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$, so P_h's shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ is also random. Therefore, we only change the order of generating P_h's shares of $[x'_\alpha], [y'_\alpha]$ and $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ without changing their distributions. Thus, $\mathbf{Hyb}_{23}$ and $\mathbf{Hyb}_{22}$ have the same output distribution.

$\mathbf{Hyb}_{24}$: In this hybrid, during the verification of tensor products, for each length-k tensor product (with index α), Sim locally computes $x_\alpha + x'_\alpha, y_\alpha + y'_\alpha$ with the corrupted parties' shares of $[x_\alpha + x'_\alpha], [y_\alpha + y'_\alpha]$ and P_h's shares of them sampled in the last hybrid. Then, Sim additionally checks whether $\overline{x_\alpha + x'_\alpha} = x_\alpha + x'_\alpha$ and $\overline{y_\alpha + y'_\alpha} = y_\alpha + y'_\alpha$. If not, Sim sets $\mathsf{MACCheck} = 1$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{24}$ and $\mathbf{Hyb}_{23}$ have the same output distribution.

$\mathbf{Hyb}_{25}$: In this hybrid, during the verification of tensor products, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 0$, Sim does not follow the protocol to compute P_h's share of $[\sigma]$. Instead, Sim computes the P_h's share of $[\sigma]$ based on the corrupted parties' shares and $\sum_{i=1}^n \sigma_i = 0$. Note that when $\mathsf{MACCheck} = 0$, all the opened values are opened correctly via Π_{Open}. In this case, it must hold that $[\sigma] = [0]$, so we only change the way of generating P_h's share of $[\sigma]$ without changing its distribution. Thus, $\mathbf{Hyb}_{25}$ and $\mathbf{Hyb}_{24}$ have the same output distribution.

$\mathbf{Hyb}_{26}$: In this hybrid, during the verification of tensor products, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 1$, Sim doesn't directly compute the honest party P_h's share of $[\Delta \cdot A_i]$ for each opened value A_i. Instead, Sim first computes each $\Delta \cdot A_i$, and then Sim computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. We just change the way the honest party's share of each $[\Delta \cdot A_i]$ is generated without changing its distribution. Thus, $\mathbf{Hyb}_{26}$ and $\mathbf{Hyb}_{25}$ have the same output distribution.

$\mathbf{Hyb}_{27}$: In this hybrid, during the verification of tensor products, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, after following the protocol to check whether $\sum_{i=1}^n \sigma_i = 0$, Sim aborts the simulation if $\mathsf{MACCheck} = 1$. The only difference between $\mathbf{Hyb}_{27}$ and $\mathbf{Hyb}_{26}$ is that the corrupted parties may not open all the values of SPDZ sharing checked in this execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ correctly, but the verification of MACs may pass.

We suppose that $A_1, \ldots, A_m$ are opened with additive errors $\delta^{(1)}, \ldots, \delta^{(m)}$, and the sum of all the com-

mitted shares $[\sigma]$ of corrupted parties is with an additive error δ_σ. Then

$$\begin{aligned}\sigma =& \sum_{i\in[1,n],i\neq h}\left(\sum_{\alpha=1}^{m}\chi^\alpha\cdot(\Delta\cdot A_\alpha)^{(i)}-\Delta_i\cdot\sum_{\alpha=1}^{m}\chi^\alpha\cdot A_\alpha\right)+\delta_\sigma\\ &+\left(\sum_{\alpha=1}^{m}\chi^\alpha\cdot(\Delta\cdot A_\alpha)^{(h)}-\Delta_h\cdot\sum_{\alpha=1}^{m}\chi^\alpha\cdot(A_\alpha+\delta^{(\alpha)})\right)\\ =&\delta_\sigma+\Delta_h\cdot\sum_{\alpha=1}^{m}\chi^\alpha\cdot\delta^{(\alpha)}.\end{aligned}$$

Here $(\Delta\cdot A_\alpha)^{(i)}$ is used to denote P_i's share of $[\Delta\cdot A_\alpha]$. Since Δ_h is not used in computing any message sent to the corrupted parties at this moment, $\Delta_h\cdot\sum_{\alpha=1}^{m}\chi^\alpha\cdot\delta^{(\alpha)}$ is a random element in $\mathbb{F}$ as long as $\sum_{\alpha=1}^{m}\chi^\alpha\cdot\delta^{(\alpha)}$ is non-zero. When $(\delta^{(1)},\ldots,\delta^{(m)})$ is a nonzero vector, since a non-zero degree-m polynomial only has at most m zeros, so $\sum_{\alpha=1}^{m}\chi^\alpha\cdot\delta^{(\alpha)}=0$ only happens with at most $m/|\mathbb{F}|$ probability. Since $m=\mathsf{poly}(n,\kappa)$, the probability is negligible. On the other hand, in the case where $\sum_{\alpha=1}^{m}\chi^\alpha\cdot\delta^{(\alpha)}=0$, σ is opened to a random element in $\mathbb{F}$, which equals 0 with $1/|\mathbb{F}|$ probability, which is also negligible.

Thus, the distributions of $\mathbf{Hyb}_{27}$ and $\mathbf{Hyb}_{26}$ are statistically close.

$\mathbf{Hyb}_{28}$: In this hybrid, during the verification of tensor products, after simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ (so $\mathsf{MACCheck}=0$ must hold), if $\mathsf{ErrorCheck}=0$, Sim does not follow the protocol to compute P_h's share of $[\tau]$. Instead, Sim computes the P_h's share of $[\tau]$ based on the corrupted parties' shares and $\sum_{i=1}^{n}\tau_i=0$. Note that when $\mathsf{ErrorCheck}=0$ and $\mathsf{MACCheck}=0$, the corrupted parties open all the values honestly and interact honestly with $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ with P_h. Therefore, the length-k tensor products are correctly computed. In this case, it must hold that $[\tau]=[0]$, so we only change the way of generating P_h's share of $[\tau]$ without changing its distribution. Thus, $\mathbf{Hyb}_{28}$ and $\mathbf{Hyb}_{27}$ have the same output distribution.

$\mathbf{Hyb}_{29}$: In this hybrid, during the verification of tensor products, after simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ (so $\mathsf{MACCheck}=0$ must hold), if $\mathsf{ErrorCheck}=1$, Sim does not follow the protocol to compute P_h's share of $[\tau]$. Instead, let $\boldsymbol{\delta}_a=(\boldsymbol{\delta}_{a,1},\ldots,\boldsymbol{\delta}_{a,m_T})\in(\mathbb{F}^k)^{m_T}$ and $\boldsymbol{\delta}_b=(\boldsymbol{\delta}_{b,1},\ldots,\boldsymbol{\delta}_{b,m_T})\in(\mathbb{F}^k)^{m_T}$. Denote P_h's shares of $[x'_\alpha],[y'_\alpha]$ by $x'_{h,\alpha},y'_{h,\alpha}$, which are the γ_α-th bits of $\mathsf{Expand}_m(s_a^{(h)})$ and $\mathsf{Expand}_m(s_b^{(h)})$ respectively. Let the γ_α-th bits of $\boldsymbol{\delta}_a,\boldsymbol{\delta_b}$ be $\delta_{x'_\alpha},\delta_{y'_\alpha}$ respectively. For the α-th length-k tensor product, Sim computes

$$\delta_\alpha=\delta_{x'_\alpha}x'_{h,\alpha}+\delta_{y'_\alpha}y'_{h,\alpha}-\langle\boldsymbol{r}_a\otimes\boldsymbol{r}_b,(\boldsymbol{a}_\alpha^{(h)}\otimes\boldsymbol{\delta}_{a,\alpha}+\boldsymbol{\delta}_{b,\alpha}\otimes\boldsymbol{b}_\alpha^{(h)})\rangle,$$

where $\boldsymbol{r}_a=(r_a^{(1)},\ldots,r_a^{(k)})$ and $\boldsymbol{r}_b=(r_b^{(1)},\ldots,r_b^{(k)})$. Sim then computes the P_h's share of $[\tau]$ based on the corrupted parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and the secret $\tau=\sum_{i=1}^{n}\tau_i=\delta_1+\cdots+\delta_{m_T}$.

Note that comparing to the case of $\mathsf{ErrorCheck}=0$ where corrupted parties honestly send their seeds to $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$, the attached error to $[\boldsymbol{a}_\alpha\otimes\boldsymbol{b}_\alpha]$ for each $\alpha=1,\ldots,m_T$ is linear to P_h's shares of $[\boldsymbol{a}_\alpha],[\boldsymbol{b}_\alpha]$, where we have computed the corresponding coefficients $\boldsymbol{\delta}_{a,\alpha},\boldsymbol{\delta}_{b,\alpha}$ on P_h's shares of $[\boldsymbol{a}_\alpha],[\boldsymbol{b}_\alpha]$. Therefore, the attached error to $[\boldsymbol{a}_\alpha\otimes\boldsymbol{b}_\alpha]$ is $\boldsymbol{a}_\alpha^{(h)}\otimes\boldsymbol{\delta}_{a,\alpha}+\boldsymbol{\delta}_{b,\alpha}\otimes\boldsymbol{b}_\alpha^{(h)}$. During the verification, each $[c_{i,j}]=[a_i\cdot b_j]$ is scaled by $r_a^{(i)}\cdot r_b^{(j)}$ in the computation of $[v_\alpha]$. Therefore, the attached error on P_h's share of $[v_\alpha]$ can be computed by $\langle\boldsymbol{r}_a\otimes\boldsymbol{r}_b,(\boldsymbol{a}_\alpha^{(h)}\otimes\boldsymbol{\delta}_{a,\alpha}+\boldsymbol{\delta}_{b,\alpha}\otimes\boldsymbol{b}_\alpha^{(h)})\rangle$. Similarly as computed during the verification on the triple $(\llbracket u'\rrbracket,\llbracket v'\rrbracket,\llbracket w'\rrbracket)$, the error on v'_α is $\delta_{x'_\alpha}x'_{h,\alpha}+\delta_{y'_\alpha}y'_{h,\alpha}$. Therefore, the total error on P_h's share of $[\tau^{(\alpha)}]$ is

$$\delta_\alpha=\delta_{x'_\alpha}x'_{h,\alpha}+\delta_{y'_\alpha}y'_{h,\alpha}-\langle\boldsymbol{r}_a\otimes\boldsymbol{r}_b,(\boldsymbol{a}_\alpha^{(h)}\otimes\boldsymbol{\delta}_{a,\alpha}+\boldsymbol{\delta}_{b,\alpha}\otimes\boldsymbol{b}_\alpha^{(h)})\rangle.$$

Note that the other parties' shares are computed honestly by Sim, so it holds that $\sum_{i=1}^{n}\tau_i=\sum_{j=1}^{m_T}\delta_j$. This shows that we only change the way of computing τ_h without changing its distribution.

Thus, $\mathbf{Hyb}_{29}$ and $\mathbf{Hyb}_{28}$ have the same output distribution.

$\mathbf{Hyb}_{30}$: In this hybrid, during the verification of tensor products, after following the protocol to check whether $\sum_{i=1}^{n}\tau_i=0$, Sim aborts the simulation if $\mathsf{ErrorCheck}=1$. The only difference between $\mathbf{Hyb}_{30}$ and

$\mathbf{Hyb}_{29}$ is that corrupted parties may not send their seeds honestly to $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ when invoking it with P_h, but the verification of linear errors may pass, i.e., the additional error δ attached to τ when corrupted parties commit to their shares of $[\tau]$ matches the error $\delta_1 + \cdots + \delta_{m_T}$ attached to P_h's share of $[\tau]$.

Without loss of generality, we assume that the θ-th bit of $\boldsymbol{\delta}_{a,\alpha}$ is $\gamma \neq 0$. Let the θ-th bit of $\boldsymbol{r}_b$ be γ_1. Then, an error of $-\langle \gamma_1 \boldsymbol{r}_a, \gamma \boldsymbol{a}_\alpha^{(h)} \rangle$ is attached to $\delta_1 + \cdots + \delta_{m_T}$. For $j = 1, \ldots, k$, as long as the j-th bit of $\gamma_2 \boldsymbol{r}'_a - \gamma_1 \boldsymbol{r}_a$ is non-zero, this error term is uniformly random in $\mathbb{F}$ based on the randomness of $\boldsymbol{a}_\alpha^{(h)}$, and all other error terms are independent of γ. In this case, the total error $\delta_1 + \cdots + \delta_{m_T}$ is also uniformly random in $\mathbb{F}$, so the probability that $\delta_1 + \cdots + \delta_{m_T} = \delta$ is no more than $1/|\mathbb{F}|$, which is negligible.

Note that if $\boldsymbol{r}_a, \boldsymbol{r}_b$ are truly random, then $-\gamma_1 \boldsymbol{r}_a$ is uniformly random in $\mathbb{F}$, which is non-zero with an overwhelming probability. Since $\boldsymbol{r}_a, \boldsymbol{r}_b$ are generated from a PRG with a truly random seed, the probability that $-\gamma_1 \boldsymbol{r}_a = 0$ is still negligible. Otherwise, the PRG can be broken due to the same reason as in $\mathbf{Hyb}_2$. Therefore, the distribution only changes with a negligible probability.

Thus, the distributions of $\mathbf{Hyb}_{30}$ and $\mathbf{Hyb}_{29}$ are computationally indistinguishable.

$\mathbf{Hyb}_{31}$: In this hybrid, when emulating $\mathcal{F}_{\mathsf{nVOLE}}$ and $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$, Sim delays the generation of the P_h's shares when they are needed. This does not change the output distribution. Thus, the distributions of $\mathbf{Hyb}_{31}$ and $\mathbf{Hyb}_{30}$ are identical.

Note that if the protocol does not abort at the end of the execution and Sim does not abort the simulation, the honest party's shares are not used during the interaction with corrupted parties. In this case, by construction, corrupted parties honestly follow the protocol and Sim only need the honest party's shares from $\mathcal{F}_{\mathsf{nVOLE}}$ and $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ to prepare the honest party's output of Π_{Prep}.

$\mathbf{Hyb}_{32}$: In this hybrid, we change the preparation of the honest party's output when the protocol does not abort at the end of the execution and Sim does not abort the simulation. Sim no longer generates the P_h's output shares when emulating $\mathcal{F}_{\mathsf{nVOLE}}$ and $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$. Instead, for each $[\![r]\!]$ (including each $[\![a_i]\!], [\![b_i]\!]$ in the length-k tensor products), Sim samples a random value as r and then computes the shares of honest parties using the secret r, the MAC key Δ, and the shares of corrupted parties. By the correctness of the construction, the distributions of $\mathbf{Hyb}_{32}$ and $\mathbf{Hyb}_{31}$ are identical.

$\mathbf{Hyb}_{33}$: In this hybrid, while Sim is emulating $\mathcal{F}_{\mathsf{nVOLE}}$, if Sim receives a global key query $(\mathsf{guess}, \boldsymbol{\Delta}')$ from $\mathcal{A}$, when $\boldsymbol{\Delta}' = \boldsymbol{\Delta}$, Sim aborts the simulation. Otherwise, Sim sends $\boldsymbol{\Delta}$ to $\mathcal{A}$ and aborts the protocol honestly on behalf of the honest party. This only changes the distribution when $\boldsymbol{\Delta}' = \boldsymbol{\Delta}$, and this only happens when $\mathcal{A}$ correctly guesses Δ_h. Note that the previous transcripts sent to $\mathcal{A}$ can be generated by Sim without knowing Δ_h, so the probability that $\boldsymbol{\Delta}' = \boldsymbol{\Delta}$ is no more than $1/|\mathbb{F}|$, which is negligible. Thus, the distributions of $\mathbf{Hyb}_{33}$ and $\mathbf{Hyb}_{32}$ are statistically close.

$\mathbf{Hyb}_{34}$: In this hybrid, during the simulation, whenever Sim needs to use Δ, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ to obtain Δ. At the end of the simulation, Sim sends the corrupted parties' outputs of $\mathcal{F}_{\mathsf{prep}}$ and lets $\mathcal{F}_{\mathsf{prep}}$ compute the honest party's output. Since the process of generating Δ by Sim himself and by $\mathcal{F}_{\mathsf{prep}}$ is the same, this doesn't change the output distribution. Thus, $\mathbf{Hyb}_{34}$ and $\mathbf{Hyb}_{33}$ have the same output distribution.

Note that $\mathbf{Hyb}_{34}$ is the ideal-world scenario. Thus, Π_{Prep} computes $\mathcal{F}_{\mathsf{prep}}$ with computational security.

B Security Proof for the Main Protocol

We prove the security of Π_{main} by constructing an ideal adversary Sim. Then we will show that the output in the ideal world is indistinguishable from that in the real world against the adversary $\mathcal{A}$ using hybrid arguments. Without loss of generality, we assume that P_{king} is corrupted. Let P_h be an honest party. The ideal adversary Sim is provided below in Figure 25.

Simulator Sim

At the beginning of the simulation, Sim sets $\mathsf{CompCheck} = \mathsf{MACCheck} = 0$. Then:

1. **Preprocessing.** Sim faithfully emulates $\mathcal{F}_{\mathsf{prep}}$ to receive the requests from corrupted parties and the corrupted parties' outputs from $\mathcal{A}$. Then, Sim sends the corrupted parties' outputs to them. If (abort, P) is received for an honest party P, then Sim randomly samples $\Delta \in \mathbb{F}$, sends it to $\mathcal{A}$, and aborts the protocol

on behalf of P. After the simulation completes, Sim outputs the adversary's view.

2. **Evaluation.** Sim runs $\mathsf{Sim}_{\mathsf{eval}}$ (Figure 26).
3. **Verification.** Sim runs $\mathsf{Sim}_{\mathsf{ver}}$ (Figure 27).
4. **Output Reconstruction.**
   - For each output wire value v of C' attached to an honest party, Sim randomly samples a field element as P_h's share of $[v]$ and follows the protocol to compute other parties' shares of $[v]$.
   - For each output wire value v of C' attached to a corrupted party, Sim receives the corresponding output wire value of C from $\mathcal{F}$, and then computes v from the corrupted party's inputs of C' and his outputs of C. Sim then follows the protocol to compute other parties' shares of $[v]$ and computes P_h's share of it based on the secret and other parties' shares.

   After that, Sim sends the honest parties' shares of $[v]$ to P_{king} on their behalf and receives $\overline{v}$ from P_{king}. Sim then checks whether all parties' shares of $[v]$ reconstruct to $\overline{v}$. If not, Sim sets $\mathsf{MACCheck} = 1$. After opening all the outputs of C', Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as follows:

(a) Sim receives RandCoin from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send the seed to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(b) For each party $P_i \neq P_h$, Sim follows the protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to compute P_i's share of $[\sigma]$.

(c) Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$ from each corrupted party P_i.

(d)
- If $\mathsf{MACCheck} = 0$, Sim computes the P_h's share of $[\sigma]$ based on the other parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \sigma_i = 0$, where the parties' shares are computed using the regarded messages from corrupted parties.
- If $\mathsf{MACCheck} = 1$, Sim randomly samples $\Delta \in \mathbb{F}$. For each value A_i opened in Π_{Open} protocol:
  - Sim computes $\Delta \cdot A_i$ and computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. Each A_i is reconstructed based on all the parties' shares, where the corrupted parties' shares are computed by following the protocol.
  - Sim follows the protocol to compute the P_h's share σ_h of $[\sigma]$.

(e) For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties. For each honest party P_j, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_j, \mathsf{mid}_{\sigma_j})$ to all the corrupted parties, where each σ_j with $j \neq h$ is computed using the actual messages received from corrupted parties.

(f) For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\sigma_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties.

(g) Sim follows the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(h) If $\mathsf{MACCheck} = 1$, Sim aborts the simulation.

Finally, Sim outputs the view of the adversary.

Figure 25: The simulator for the main protocol.

Simulator $\mathsf{Sim}_{\mathsf{eval}}$

Sim follows the protocol to obtain circuit C'. Then:

1. **Authenticating Inputs.** For each input wire of C' attached to a party P_i:
   - If P_i is honest, Sim receives corrupted parties' shares of $([r], [\Delta_i \cdot r])$ from the consumed input mask sharings on behalf of P_i. Then, Sim checks whether the sum of corrupted parties' shares of $[r]$ (resp. $[\Delta_i \cdot r]$) received from them is the same as the sum of corrupted parties' shares of $[r]$ (resp. $[\Delta_i \cdot r]$) sent to them by Sim while emulating $\mathcal{F}_{\mathsf{prep}}$. If not, Sim aborts the protocol on behalf of P_i. After completing the simulation, Sim outputs the adversary's view. Otherwise, Sim samples a random element as $x + r$ and sends it to all the corrupted parties.
   - If P_i is corrupted, Sim randomly samples a field element as each honest party's share of $[r]$ from the

consumed input mask sharings and then computes $r, \Delta_i \cdot r$. Then, Sim samples the honest parties' shares of $[\Delta_i \cdot r]$ based on the secret and corrupted parties' shares and sends them to P_i on behalf of the honest parties. Sim then receives $x + r$ from P_i on behalf of P_h and computes $x = (x + r) - r$ as P_i's input on this wire. Sim regards that all parties receive the same message $x + r$ as P_h.

After obtaining all the corrupted parties' inputs to C, Sim sends them to $\mathcal{F}$ and obtains the outputs of corrupted parties for the circuit C.

2. **Evaluating the Circuit.** For each honest party $P_i \neq P_h$, Sim faithfully samples the remaining outputs for P_i from $\mathcal{F}_{\mathsf{prep}}$. During the evaluation, the transcripts between these parties and the corrupted parties are all computed honestly from the actual transcripts they received. For each input wire with input x, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![x]\!]$. Sim then follows the protocol to group the multiplication gates. For each gate of C':
   - If the gate is an addition gate, Sim follows the protocol to evaluate each $P_i \neq P_h$'s share of the additive sharing for the output wire value of this gate.
   - If the gate is a multiplication gate with input wires $x_\alpha^{(j)}, y_\alpha^{(j)}$:
     (a) Sim samples P_h's shares of $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$ randomly.
     (b) Sim sends the honest parties' shares of $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$ to P_{king} on behalf of the honest parties (the shares of honest parties except P_h are computed by following the protocol using actual transcripts they receive from the corrupted parties). Sim then reconstructs $x_\alpha^{(j)} + a_\alpha, y_\alpha^{(j)} + b_\alpha$ from all parties' shares (the shares of all parties except P_h are computed using the regarded messages, i.e., when a corrupted party needs to send a message to all parties, we regard that all honest parties receive the message for P_h. Same below.).
     (c) Sim receives $\overline{x_\alpha^{(j)} + a_\alpha}, \overline{y_\alpha^{(j)} + b_\alpha}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{x_\alpha^{(j)} + a_\alpha}, \overline{y_\alpha^{(j)} + b_\alpha}$ from P_{king} as $x_\alpha^{(j)} + a_\alpha, y_\alpha^{(j)} + b_\alpha$. Then, Sim additionally follows the protocol to compute each $P_i \neq P_h$'s share of $[\![x_\alpha^{(j)}]\!], [\![y_\alpha^{(j)}]\!], [z_\alpha^{(j)}]$. The honest parties' shares among them are only used for doing reconstructions (e.g., in Step (b)), while the transcripts are still computed with the actual values they receive from corrupted parties.
     (d) * If $x_\alpha^{(j)}$ (resp. $y_\alpha^{(j)}$) can be computed directly from input wires of C', Sim sets $\mathsf{MACCheck} = 1$ if $\overline{x_\alpha^{(j)} + a_\alpha} \neq x_\alpha^{(j)} + a_\alpha$ (resp. $\overline{y_\alpha^{(j)} + b_\alpha} \neq y_\alpha^{(j)} + b_\alpha$).
     * Otherwise, Sim computes $\delta_{x_\alpha^{(j)}} = \overline{x_\alpha^{(j)} + a_\alpha} - (x_\alpha^{(j)} + a_\alpha)$ (resp. $\delta_{y_\alpha^{(j)}} = \overline{y_\alpha^{(j)} + b_\alpha} - (y_\alpha^{(j)} + b_\alpha)$). If $\overline{x_\alpha^{(j)} + a_\alpha} \neq x_\alpha^{(j)} + a_\alpha$ (resp. $\overline{y_\alpha^{(j)} + b_\alpha} \neq y_\alpha^{(j)} + b_\alpha$), Sim sets $\mathsf{CompCheck} = 1$.

   Finally, for each output wire with value v of C':
   (a) Sim samples P_h's share of $[r + v]$ randomly.
   (b) Sim sends the honest parties' shares of $[r + v]$ to P_{king} on behalf of the honest parties. Sim then reconstructs $r + v$ from all parties' shares.
   (c) Sim receives $\overline{r + v}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{r + v}$ from P_{king} as $r + v$. Then, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![v]\!]$.
   (d) – If v an be computed directly from input wires of C', Sim sets $\mathsf{MACCheck} = 1$ if $\overline{r + v} \neq r + v$.
   – Otherwise, Sim computes $\delta_v = \overline{r + v} - (r + v)$. If $\overline{r + v} \neq r + v$, Sim sets $\mathsf{CompCheck} = 1$.

3. **MAC Check.** Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as follows:
   (a) Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send the seed to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.
   (b) For each party $P_i \neq P_h$, Sim follows the protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to compute P_i's share of $[\sigma]$.
   (c) Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$ from each corrupted party P_i.
   (d) – If $\mathsf{MACCheck} = 0$, Sim computes the P_h's share of $[\sigma]$ based on the other parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \sigma_i = 0$, where the parties' shares are computed using the regarded messages from corrupted parties.
   – If $\mathsf{MACCheck} = 1$, Sim randomly samples $\Delta \in \mathbb{F}$. For each value A_i opened in Π_{Open} protocol:

- ∗ Sim computes $\Delta \cdot A_i$ and computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. Each A_i is reconstructed based on all the parties' shares, where the corrupted parties' shares are computed by following the protocol.
- ∗ Sim follows the protocol to compute the P_h's share σ_h of $[\sigma]$.

(e) For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties. For each honest party P_j, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_j, \mathsf{mid}_{\sigma_j})$ to all the corrupted parties, where each σ_j with $j \neq h$ is computed using the actual messages received from corrupted parties.

(f) For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\sigma_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties.

(g) Sim follows the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(h) If $\mathsf{MACCheck} = 1$, Sim aborts the simulation.

Figure 26: The simulator for the evaluation phase.

Simulator $\mathsf{Sim}_{\mathsf{ver}}$

1. **Reducing to Inner-Product Check.**

(a) Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send s to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(b) For each party $P_i \neq P_h$, Sim follows the protocol to compute each P_i's shares of $[\![\boldsymbol{x}]\!], [\![\boldsymbol{y}]\!], [\![z]\!]$. Moreover, Sim computes $\delta_z = \sum_{j=1}^{M} s^j \delta_{z_j}$, where δ_{z_j} is the additive error attached to the value z_j computed during the evaluation phase.

(c) If $\mathsf{CompCheck} = 1$ but $\delta_z = 0$, Sim aborts the simulation.

(d) For $[\![\boldsymbol{x}]\!] = \left(\sum_{i=1}^{m} s^i \cdot \alpha_i^{(1)} \cdot [\![x_1]\!], \ldots, \sum_{i=1}^{m} s^i \cdot \alpha_i^{(|C|)} \cdot [\![x_{|C|}]\!] \right)$, if there exists $j \in \{1, \ldots, |C|\}$ such that the coefficient $\theta_j = \sum_{i=1}^{M} s^i \cdot \alpha_i^{(j)} = 0$, Sim aborts the simulation.

2. **Separate into Blocks.**

(a) For each $j = 1, \ldots, \ell - 1$:

i. Sim samples P_h's share of $[r^{(j)} + z^{(j)}]$ randomly.

ii. Sim sends the honest parties' shares of $[r^{(j)} + z^{(j)}]$ to P_{king} on behalf of the honest parties. Sim then reconstructs $r^{(j)} + z^{(j)}$ from all parties' shares.

iii. Sim receives $\overline{r^{(j)} + z^{(j)}}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{r^{(j)} + z^{(j)}}$ from P_{king} as $r^{(j)} + z^{(j)}$. Then, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![z^{(j)}]\!]$.

iv. Sim computes $\delta_{z^{(j)}} = \overline{r^{(j)} + z^{(j)}} - (r^{(j)} + z^{(j)})$.

Sim then follows the protocol to compute each $P_i \neq P_h$'s share of $[\![z^{(\ell)}]\!]$ and then computes $\delta_{z^{(\ell)}} = \delta_z - \sum_{j=1}^{\ell-1} \delta_{z^{(j)}}$.

3. **Recursive Check.** For each group of multiplication gates (with index j), Sim follows the protocol to do initializations on behalf of each $P_i \neq P_h$ and compute P_i's share of $[\boldsymbol{u} \otimes \boldsymbol{v}]$. Sim additionally sets $\delta_w = \delta_{z^{(j)}}$. For each reduction in vector length:

(a) For each $i = 1, \ldots, \lambda - 1$:

i. Sim samples P_h's share of $[r_i + w_i]$ randomly.

ii. Sim sends the honest parties' shares of $[r_i + w_i]$ to P_{king} on behalf of the honest parties. Sim then reconstructs $r_i + w_i$ from all parties' shares.

iii. Sim receives $\overline{r_i + w_i}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{r_i + w_i}$ from P_{king} as $r_i + w_i$. Then, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![w_i]\!]$.

iv. Sim computes $\delta_{w_i} = \overline{r_i + w_i} - (r_i + w_i)$.

Finally, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![w_\lambda]\!]$ and then computes $\delta_{w_\lambda} = \delta_w - \sum_{i=1}^{\lambda-1} \delta_{w_i}$.

(b) For each $i = \lambda+1, \ldots, 2\lambda-1$:

i. Sim samples P_h's share of $[r_i + h(i)]$ randomly.

ii. Sim sends the honest parties' shares of $[r_i + h(i)]$ to P_{king} on behalf of the honest parties. Sim then reconstructs $r_i + h(i)$ from all parties' shares.

iii. Sim receives $\overline{r_i + h(i)}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{r_i + h(i)}$ from P_{king} as $r_i + h(i)$. Then, Sim follows the protocol to compute each $P_i \neq P_h$'s share of $[\![h(i)]\!]$.

iv. Sim computes $\delta_{h(i)} = \overline{r_i + h(i)} - (r_i + h(i))$.

(c) Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send r to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

(d) For the linear function L' such that $h(r) = L'(h(1), \ldots, h(2\lambda-1))$, Sim computes $\delta_{h(r)} = L'(\delta_{w_1}, \ldots, \delta_{w_\lambda}, \delta_{h(\lambda+1)}, \ldots, \delta_{h(2\lambda-1)})$.

(e) If $\delta_w \neq 0$ but $\delta_{h(r)} = 0$, Sim aborts the simulation. Otherwise, Sim updates $\delta_w = \delta_{h(x)}$.

(f) Sim follows the protocol to update each $P_i \neq P_h$'s shares of $[\![\boldsymbol{u}]\!], [\![\boldsymbol{v}]\!], [\![w]\!], [\boldsymbol{u} \otimes \boldsymbol{v}]$.

4. **Verify the Final Triple.** After the parties obtain the final triple $([\![u]\!], [\![v]\!], [\![w]\!])$ with error δ_w for each group of multiplication gates:

(a) Sim samples P_h's shares of $[u+a], [v+b]$ randomly.

(b) Sim sends the honest parties' shares of $[u+a], [v+b]$ to P_{king} on behalf of the honest parties (the shares of honest parties except P_h are computed by following the protocol using actual transcripts they receive from the corrupted parties). Sim then reconstructs $u+a, v+b$ from all parties' shares.

(c) Sim receives $\overline{u+a}, \overline{v+b}$ from P_{king} on behalf of P_h. Sim then regards that all the honest parties receive $\overline{u+a}, \overline{v+b}$ from P_{king} as $u+a, v+b$.

(d) Sim sets $\mathsf{MACCheck} = 1$ if $\overline{u+a} \neq u+a$ or $\overline{v+b} \neq v+b$.

(e)
- If $\mathsf{MACCheck} = 1$ or $\delta_w \neq 0$, Sim samples P_h's shares of $[u], [a], [v], [b]$ based on his shares of $[u+a], [v+b]$. Then, Sim reconstructs u, v, a, b, computes $w = uv + \delta_w, c = ab$, and computes P_h's shares of $[w], [c]$ based on corrupted parties' shares and the secrets. Sim then follows the protocol to compute P_h's share of $[\tau]$ and reconstruct τ (with the regarded messages).
- Otherwise, Sim honestly computes the shares of $[\tau]$ for each $P_i \neq P_h$ with the regarded messages and then computes P_h's share based on the secret $\tau = 0$ and other parties' shares.

(f) Sim sends the honest parties' shares of $[\tau]$ to P_{king} on behalf of the honest parties and receives $\overline{\tau}$ from P_{king}. Sim follows the protocol to check whether $\overline{\tau} = 0$ on behalf of each honest party. If not, Sim aborts the protocol and outputs the adversary's view after completing the simulation. Furthermore, if $\overline{\tau} \neq \tau$, Sim sets $\mathsf{MACCheck} = 1$.

(g) Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as follows:

i. Sim receives $\mathsf{RandCoin}$ from corrupted parties and emulates $\mathcal{F}_{\mathsf{Coin}}$ to send the seed to them. If Sim receives abort from $\mathcal{A}$, he sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

ii. For each party $P_i \neq P_h$, Sim follows the protocol $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ to compute P_i's share of $[\sigma]$.

iii. Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to receive $(\mathsf{commit}, P_i, \sigma_i, \mathsf{mid}_{\sigma_i})$ from each corrupted party P_i.

iv.
- If $\mathsf{MACCheck} = 0$, Sim computes the P_h's share of $[\sigma]$ based on the other parties' shares (computed by Sim honestly, not the received shares by $\mathcal{F}_{\mathsf{Commit}}$) and $\sum_{i=1}^{n} \sigma_i = 0$, where the parties' shares are computed using the regarded messages from corrupted parties.
- If $\mathsf{MACCheck} = 1$, Sim randomly samples $\Delta \in \mathbb{F}$. For each value A_i opened in Π_{Open} protocol:
  * Sim computes $\Delta \cdot A_i$ and computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. Each A_i is reconstructed based on all the parties' shares, where the corrupted parties' shares are computed by following the protocol.
  * Sim follows the protocol to compute the P_h's share σ_h of $[\sigma]$.

v. For each party corrupted P_i, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties. For each honest party P_j, Sim emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\mathsf{committed}, P_j, \mathsf{mid}_{\sigma_j})$ to all the corrupted parties, where each σ_j with $j \neq h$ is computed using the actual messages received from corrupted parties.

vi. For each party P_i, Sim receives $(\mathsf{open}, P_i, \mathsf{mid}_{\sigma_i})$ from the corrupted parties and emulates $\mathcal{F}_{\mathsf{Commit}}$ to send $(\sigma_i, \mathsf{mid}_{\sigma_i})$ to all the corrupted parties.

vii. Sim follows the protocol to check whether $\sum_{i=1}^{n} \sigma_i = 0$ holds. If not, Sim aborts the protocol on behalf of the honest party. After completing the simulation, Sim outputs the adversary's view.

viii. If $\mathsf{MACCheck} = 1$, Sim aborts the simulation.

Figure 27: The simulator for the verification phase.

$\mathbf{Hyb}_0$: In this hybrid, Sim receives the honest parties' inputs and runs the protocol honestly. This corresponds to the real-world scenario.

$\mathbf{Hyb}_1$: In this hybrid, Sim additionally sets $\mathsf{CompCheck} = \mathsf{MACCheck} = 0$ at the beginning. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ have the same output distribution.

$\mathbf{Hyb}_2$: In this hybrid, Sim does not samples each honest party P_i's share Δ_i of $[\Delta]$ while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples them when they are used for computation, i.e., when P_i needs to verify $[r], [\Delta_i \cdot r]$ during the authentication process for his inputs. This does not affect the output distribution. Thus, $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ have the same output distribution.

$\mathbf{Hyb}_3$: In this hybrid, while authenticating inputs, for each input x of an honest party P_i, Sim does not follow the protocol to verify the sharings $[r], [\Delta_i \cdot r]$. Instead, he verifies whether the sum of corrupted parties' shares of $[r]$ (resp. $[\Delta_i \cdot r]$) received from them is the same as the sum of corrupted parties' shares of $[r]$ (resp. $[\Delta_i \cdot r]$) sent to them by Sim while emulating $\mathcal{F}_{\mathsf{prep}}$. This only changes the output distribution if the adversary attaches an error e to the sum of the corrupted parties' shares of $[r]$ and attaches $e' = \Delta_i \cdot e$ to the sum of the corrupted parties' shares of $[\Delta_i \cdot r]$. Since Δ_i is sampled randomly after e, e' are determined, the probability that $e' = \Delta_i \cdot e$ holds is no more than $1/|\mathbb{F}|$, which is negligible. Thus, $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ have the same output distribution.

$\mathbf{Hyb}_4$: In this hybrid, while authenticating inputs, Sim does not generate honest parties' shares of $[r]$ in the input mask sharings for honest parties' inputs. Instead, for each input x of an honest party P_i, Sim samples a random element as $x + r$ and then samples honest parties' shares of $[r]$ based on $x, x + r$ and corrupted parties' shares of $[r]$. Since Sim samples r randomly and then samples honest parties' shares of $[r]$ based on r and corrupted parties' shares of $[r]$ while faithfully emulating $\mathcal{F}_{\mathsf{prep}}$, we only change the way r is sampled. Since r is uniformly random in $\mathbb{F}$, so is $x + r$. Thus, we only change the order of generating r and $x + r$ without changing their distributions. Thus, $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ have the same output distribution.

$\mathbf{Hyb}_5$: In this hybrid, whenever a corrupted party needs to send a message to all the parties, Sim regards that all parties receive the same message as the actual message received from P_h. These messages are only used for additional computations later, but are not used for computing the transcripts sent from honest parties to the corrupted parties. This does not affect the output distribution. Thus, $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ have the same output distribution.

$\mathbf{Hyb}_6$: In this hybrid, for each multiplication gate of C' (with inputs $x_\alpha^{(j)}, y_\alpha^{(j)}$), Sim does not sample P_h's shares of $[\![a_\alpha]\!], [\![b_\alpha]\!]$ in the j-th length-k tensor product while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples P_h's shares of $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$ randomly during the evaluation phase and then computes his shares of $[a_\alpha] = [x_\alpha^{(j)} + a_\alpha] - [x_\alpha^{(j)}]$ and $[b_\alpha] = [y_\alpha^{(j)} + b_\alpha] - [y_\alpha^{(j)}]$. After that, a_α, b_α are computed accordingly and then be used to compute P_h's shares of $[\Delta \cdot a_\alpha], [\Delta \cdot b_\alpha]$. Since P_h's shares of $[a_\alpha], [b_\alpha]$ are uniformly random in $\mathbb{F}$, so are his shares of $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$. Thus, we only change the order of generating P_h's shares of $[a_\alpha], [b_\alpha]$ and $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$ without changing their distributions. Thus, $\mathbf{Hyb}_6$ and $\mathbf{Hyb}_5$ have the same output distribution.

$\mathbf{Hyb}_7$: In this hybrid, during the evaluation, for each multiplication gate of C' (with inputs $x_\alpha^{(j)}, y_\alpha^{(j)}$), Sim additionally follows the protocol to compute the shares of $[x_\alpha^{(j)} + a_\alpha], [y_\alpha^{(j)} + b_\alpha]$ for each party $P_i \neq P_h$ using the regarded values (in $\mathbf{Hyb}_5$) sent from corrupted parties. These shares, together with the sampled

shares for P_h, are used for reconstructing $x_\alpha^{(j)} + a_\alpha, y_\alpha^{(j)} + b_\alpha$ additionally.

Then, after receiving $\overline{x_\alpha^{(j)} + a_\alpha}, \overline{y_\alpha^{(j)} + b_\alpha}$,

- If $x_\alpha^{(j)}$ (resp. $y_\alpha^{(j)}$) can be computed directly from input wires of C', Sim additionally sets $\mathsf{MACCheck} = 1$ if $\overline{x_\alpha^{(j)} + a_\alpha} \neq x_\alpha^{(j)} + a_\alpha$ (resp. $\overline{y_\alpha^{(j)} + b_\alpha} \neq y_\alpha^{(j)} + b_\alpha$).
- Otherwise, Sim additionally computes $\delta_{x_\alpha^{(j)}} = \overline{x_\alpha^{(j)} + a_\alpha} - (x_\alpha^{(j)} + a_\alpha)$ (resp. $\delta_{y_\alpha^{(j)}} = \overline{y_\alpha^{(j)} + b_\alpha} - (y_\alpha^{(j)} + b_\alpha)$). If $\overline{x_\alpha^{(j)} + a_\alpha} \neq x_\alpha^{(j)} + a_\alpha$ (resp. $\overline{y_\alpha^{(j)} + b_\alpha} \neq y_\alpha^{(j)} + b_\alpha$), Sim sets $\mathsf{CompCheck} = 1$.

This does not affect the output distribution. Thus, $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_6$ have the same output distribution.

$\mathbf{Hyb}_8$: In this hybrid, for each output wire of C' (with wire value v), Sim does not sample P_h's share of the consumed sharing $[\![r]\!]$ for authenticating the output value while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples P_h's share of $[r+v]$ randomly during the evaluation phase and then computes his share of $[r] = [r+v] - [v]$. After that, r are computed accordingly and then be used to compute P_h's share of $[\Delta \cdot r]$. For the same reason as in $\mathbf{Hyb}_6$, $\mathbf{Hyb}_8$ and $\mathbf{Hyb}_7$ have the same output distribution.

$\mathbf{Hyb}_9$: In this hybrid, during the evaluation, for each output wire of C' (with wire value v), Sim additionally follows the protocol to compute the shares of $[r+v]$ for each party $P_i \neq P_h$ using the regarded values (in $\mathbf{Hyb}_5$) sent from corrupted parties. These shares, together with the sampled shares for P_h, are used for reconstructing $r + v$ additionally.

Then, after receiving $\overline{r+v}$,

- If v an be computed directly from input wires of C', Sim additionally sets $\mathsf{MACCheck} = 1$ if $\overline{r+v} \neq r+v$.
- Otherwise, Sim additionally computes $\delta_v = \overline{r+v} - (r+v)$. If $\overline{r+v} \neq r+v$, Sim sets $\mathsf{CompCheck} = 1$.

This does not affect the output distribution. Thus, $\mathbf{Hyb}_9$ and $\mathbf{Hyb}_8$ have the same output distribution.

$\mathbf{Hyb}_{10}$: In this hybrid, during the evaluation, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 0$, Sim does not follow the protocol to compute P_h's share of $[\sigma]$. Instead, Sim computes the P_h's share of $[\sigma]$ based on the other parties' shares (computed using regarded messages) and $\sum_{i=1}^n \sigma_i = 0$. Note that when $\mathsf{MACCheck} = 0$, all the opened values are opened correctly via Π_{Open}. In this case, it must hold that $[\sigma] = [0]$, so we only change the way of generating P_h's share of $[\sigma]$ without changing its distribution. Thus, $\mathbf{Hyb}_{10}$ and $\mathbf{Hyb}_9$ have the same output distribution.

$\mathbf{Hyb}_{11}$: In this hybrid, during the evaluation, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, if $\mathsf{MACCheck} = 1$, Sim doesn't directly compute the honest party P_h's share of $[\Delta \cdot A_i]$ for each opened value A_i. Instead, Sim first computes each $\Delta \cdot A_i$, and then Sim computes the honest party's share of $[\Delta \cdot A_i]$ based on the corrupted parties' shares and $\Delta \cdot A_i$. We just change the way the honest party's share of each $[\Delta \cdot A_i]$ is generated without changing its distribution. Thus, $\mathbf{Hyb}_{11}$ and $\mathbf{Hyb}_{10}$ have the same output distribution.

$\mathbf{Hyb}_{12}$: In this hybrid, during the evaluation, while simulating $\Pi_{\mathsf{SPDZ\text{-}MAC}}$, after following the protocol to check whether $\sum_{i=1}^n \sigma_i = 0$, Sim aborts the simulation if $\mathsf{MACCheck} = 1$. The only difference between $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ is that the corrupted parties may not open all the values of SPDZ sharing checked in this execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ correctly, but the verification of MACs may pass.

We suppose that $A_1, \ldots, A_m$ are opened with additive errors $\delta^{(1)}, \ldots, \delta^{(m)}$, and the sum of all the committed shares $[\sigma]$ of corrupted parties is with an additive error δ_σ. Then

$$\begin{aligned}\sigma =& \sum_{i\in[1,n], i\neq h} \left(\sum_{\alpha=1}^m \chi^\alpha \cdot (\Delta \cdot A_\alpha)^{(i)} - \Delta_i \cdot \sum_{\alpha=1}^m \chi^\alpha \cdot A_k \right) + \delta_\sigma \\ &+ \left(\sum_{\alpha=1}^m \chi^\alpha \cdot (\Delta \cdot A_\alpha)^{(h)} - \Delta_h \cdot \sum_{\alpha=1}^m \chi^\alpha \cdot (A_\alpha + \delta^{(\alpha)}) \right) \\ =& \delta_\sigma + \Delta_h \cdot \sum_{\alpha=1}^m \chi^\alpha \cdot \delta^{(\alpha)}.\end{aligned}$$

Here $(\Delta \cdot A_\alpha)^{(i)}$ is used to denote P_i's share of $[\Delta \cdot A_\alpha]$. Since Δ_h is not used in computing any message sent to the corrupted parties at this moment, $\Delta_h \cdot \sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)}$ is a random element in $\mathbb{F}$ as long as $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)}$ is non-zero. When $(\delta^{(1)}, \ldots, \delta^{(m)})$ is a nonzero vector, since a non-zero degree-m polynomial only has at most m zeros, so $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)} = 0$ only happens with at most $m/|\mathbb{F}|$ probability. Since $m = \mathsf{poly}(n, \kappa)$, the probability is negligible. On the other hand, in the case where $\sum_{\alpha=1}^{m} \chi^\alpha \cdot \delta^{(\alpha)} = 0$, σ is opened to a random element in $\mathbb{F}$, which equals 0 with $1/|\mathbb{F}|$ probability, which is also negligible.

Thus, the distributions of $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ are statistically close.

$\mathbf{Hyb}_{13}$: In this hybrid, during the reduction to inner-product check, Sim additionally computes the error $\delta_z = \sum_{j=1}^{M} s^j \delta_{z_j}$ on z for the sharing $[\![z]\!]$, where δ_{z_j} is the additive error attached to the value z_j computed during the evaluation phase. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{13}$ and $\mathbf{Hyb}_{12}$ have the same output distribution.

$\mathbf{Hyb}_{14}$: In this hybrid, during the reduction to inner-product check, after computing δ_z, Sim aborts the simulation if $\mathsf{ErrorCheck} = 1$ but $\delta_z = 0$. This only changes the output distribution when there is some $j \in \{1, \ldots, M\}$ such that $\delta_{z_j} \neq 0$, but it happens that $\delta_z = 0$.

Since s is randomly determined after $\delta_{z_1}, \ldots, \delta_{z_M}$ are all fixed, this only happens when s is a solution for the equation $\sum_{i=1}^{M} s^i \cdot \delta_{z_i} = 0$. Since the left-hand side is a degree-M polynomial in s, it has at most $M = \mathsf{poly}(\kappa)$ zeros. Therefore, the probability that $\delta_z = 0$ is at most $M/|\mathbb{F}|$, which is negligible. Thus, the distributions of $\mathbf{Hyb}_{14}$ and $\mathbf{Hyb}_{13}$ are statistically close.

$\mathbf{Hyb}_{15}$: In this hybrid, during the separation into blocks, for the sharing $[\![r^{(j)}]\!]$ $(j = 1, \ldots, \ell - 1)$ used to authenticate $[z^{(j)}]$, Sim does not generate P_h's share of it while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples P_h's share of $[r^{(j)} + z^{(j)}]$ randomly and then computes his share of $[r^{(j)}] = [r^{(j)} + z^{(j)}] - [z^{(j)}]$. Then, P_h's share of $[\Delta \cdot r^{(j)}]$ is computed based on $\Delta, r^{(j)}$ and other parties' shares. Note that P_h's share of $[r^{(j)}]$ is sampled randomly in $\mathcal{F}_{\mathsf{prep}}$, so his share of $[r^{(j)} + z^{(j)}]$ is also uniformly random. Therefore, we only change the order of generating P_h's shares of $[r^{(j)} + z^{(j)}]$ and $[r^{(j)}]$ without changing their distributions. Thus, $\mathbf{Hyb}_{15}$ and $\mathbf{Hyb}_{14}$ have the same output distribution.

$\mathbf{Hyb}_{16}$: In this hybrid, during the first reduction in vector length, after receiving $\overline{r^{(j)} + z^{(j)}}$ for each $j = 1, \ldots, \ell - 1$ on behalf of P_h, Sim regards that all parties receive without changing the transcripts. Then, Sim reconstructs $r^{(j)} + z^{(j)}$ from all parties' shares (obtained by using the regarded messages to run the protocol) and additionally computes $\delta_{z^{(j)}} = \overline{r^{(j)} + z^{(j)}} - (r^{(j)} + z^{(j)})$. Finally, Sim computes $\delta_{z^{(\ell)}} = \delta_z - \sum_{j=1}^{\ell-1} \delta_{z^{(j)}}$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{16}$ and $\mathbf{Hyb}_{15}$ have the same output distribution.

$\mathbf{Hyb}_{17}$: In this hybrid, before doing the recursive check for each group of multiplication gates (with index j), Sim additionally initializes $\delta_w = \delta_{z^{(j)}}$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{17}$ and $\mathbf{Hyb}_{16}$ have the same output distribution.

Let $\mathbf{Hyb}_{18.0.5} = \mathbf{Hyb}_{17}$. We then construct $\mathbf{Hyb}_{18.\gamma.1}, \ldots, \mathbf{Hyb}_{15.\gamma.5}$ $(\gamma = 1, \ldots, \lceil \log_\lambda k \rceil)$ for the γ-th reduction in vector length.

$\mathbf{Hyb}_{18.\gamma.1}$: In this hybrid, during the γ-th reduction in vector length for each group of multiplication gates, for the sharing $[\![r_i]\!]$ $(i = 1, \ldots, \lambda - 1)$ used to authenticate $[w_i]$, Sim does not generate P_h's share of it while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples P_h's share of $[r_i + w_i]$ randomly and then computes his share of $[r_i] = [r_i + w_i] - [w_i]$. Then, P_h's share of $[\Delta \cdot r_i]$ is computed based on Δ, r_i and other parties' shares. Note that P_h's share of $[r_i]$ is sampled randomly in $\mathcal{F}_{\mathsf{prep}}$, so his share of $[r_i + w_i]$ is also uniformly random. Therefore, we only change the order of generating P_h's shares of $[r_i + w_i]$ and $[r_i]$ without changing their distributions. Thus, $\mathbf{Hyb}_{18.\gamma.1}$ and $\mathbf{Hyb}_{18.(\gamma-1).5}$ have the same output distribution.

$\mathbf{Hyb}_{18.\gamma.2}$: In this hybrid, during the γ-th reduction in vector length for each group of multiplication gates, after receiving $\overline{r_i + w_i}$ for each $i = 1, \ldots, \lambda - 1$ on behalf of P_h, Sim regards that all parties receive without changing the transcripts. Then, Sim reconstructs $r_i + w_i$ from all parties' shares (obtained by using the regarded messages to run the protocol) and additionally computes $\delta_{w_i} = \overline{r_i + w_i} - (r_i + w_i)$. Finally, Sim computes $\delta_{w_\lambda} = \delta_w - \sum_{i=1}^{\lambda-1} \delta_{w_i}$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{18.\gamma.2}$ and $\mathbf{Hyb}_{18.\gamma.1}$ have the same output distribution.

$\mathbf{Hyb}_{18.\gamma.3}$: In this hybrid, during the γ-th reduction in vector length for each group of multiplication gates, for the sharing $[\![r_i]\!]$ $(i = \lambda + 1, \ldots, 2\lambda - 1)$ used to authenticate $[h(i)]$, Sim does not generate P_h's share of it while emulating $\mathcal{F}_{\mathsf{prep}}$. Instead, Sim samples P_h's share of $[r_i + h(i)]$ randomly and then computes his

share of $[r_i] = [r_i + h(i)] - [h(i)]$. Then, P_h's share of $[\Delta \cdot r_i]$ is computed based on Δ, r_i and other parties' shares. Note that P_h's share of $[r_i]$ is sampled randomly in $\mathcal{F}_{\mathsf{prep}}$, so his share of $[r_i + h(i)]$ is also uniformly random. Therefore, we only change the order of generating P_h's shares of $[r_i + h(i)]$ and $[r_i]$ without changing their distributions. Thus, $\mathbf{Hyb}_{18.\gamma.3}$ and $\mathbf{Hyb}_{18.\gamma.2}$ have the same output distribution.

$\mathbf{Hyb}_{18.\gamma.4}$: In this hybrid, during the first reduction in vector length for each group of multiplication gates, after receiving $\overline{r_i + h(i)}$ for each $i = \lambda+1, \ldots, 2\lambda-1$ on behalf of P_h, Sim regards that all parties receive without changing the transcripts. Then, Sim reconstructs $r_i + h(i)$ from all parties' shares (obtained by using the regarded messages to run the protocol) and additionally computes $\delta_{h(i)} = \overline{r_i + h(i)} - (r_i + h(i))$. Finally, Sim computes $\delta_{h(r)} = L'(\delta_{w_1}, \ldots, \delta_{w_\lambda}, \delta_{h(\lambda+1)}, \ldots, \delta_{h(2\lambda-1)})$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{18.\gamma.4}$ and $\mathbf{Hyb}_{18.\gamma.3}$ have the same output distribution.

$\mathbf{Hyb}_{18.\gamma.5}$: In this hybrid, during the first reduction in vector length for each group of multiplication gates, if $\delta_w \neq 0$ but $\delta_{h(x)} = 0$, Sim aborts the simulation. Otherwise, Sim updates $\delta_w = \delta_{h(x)}$. This only changes the output distribution if δ_w is non-zero but $\delta_{h(x)} = 0$.

Note that $\sum_{i=1}^{\lambda} \delta_{w_i} = \delta_w$, so $\delta_w \neq 0$ implies that $(\delta_{w_1}, \ldots, \delta_{w_\lambda})$ is a non-zero vector. Thus, the degree-$(2\lambda - 2)$ polynomial $h_\delta(\cdot)$ with $h_\delta(i) = z_i$ for $i = 1, \ldots, \lambda$ and $h_\delta(i) = h(i)$ for $i = \lambda + 1, \ldots, 2\lambda - 1$ is a non-zero polynomial. Thus, $h_\delta(\cdot)$ only has at most $2k - 2$ zeros. Since $\delta_{h(r)} = h_\delta(r)$ for a random r, the probability that $\delta_{h(r)}$ is at most $(2\lambda - 2)/|\mathbb{F}|$, which is negligible. Thus, the distributions of $\mathbf{Hyb}_{15.\gamma.5}$ and $\mathbf{Hyb}_{15.\gamma.4}$ are statistically close.

$\mathbf{Hyb}_{19}$: In this hybrid, after obtaining the final triple $(\llbracket u \rrbracket, \llbracket v \rrbracket, \llbracket w \rrbracket)$ for each group of multiplication gates during the verification phase, Sim does not sample P_h's shares of $\llbracket a \rrbracket, \llbracket b \rrbracket$ for the consumed triple while emulating $\mathcal{F}_{\mathsf{prep}}$ and uses them to compute his shares of $\llbracket u+a \rrbracket, \llbracket v+b \rrbracket$ here. Instead, Sim samples P_h's shares of $[u + a], [v + b]$ randomly during the evaluation phase and then computes his shares of $[a] = [u + a] - [u]$ and $[b] = [v + b] - [v]$. After that, a, b are computed accordingly and then be used to compute P_h's shares of $[\Delta \cdot a], [\Delta \cdot b], \llbracket c \rrbracket$. Since P_h's shares of $[a], [b]$ are uniformly random in $\mathbb{F}$, so are his shares of $[u + a], [v + b]$. Thus, we only change the order of generating P_h's shares of $[a], [b]$ and $[u + a], [v + b]$ without changing their distributions. Thus, $\mathbf{Hyb}_{19}$ and $\mathbf{Hyb}_{18.\lceil \log_\lambda k \rceil.5}$ have the same output distribution.

$\mathbf{Hyb}_{20}$: In this hybrid, after obtaining $\overline{u + a}, \overline{v + b}$ for each group of multiplication gates during the verification phase, Sim additionally follows the protocol to compute the shares of $[u + a], [v + b]$ for each party $P_i \neq P_h$ using the regarded values sent from corrupted parties. These shares, together with the sampled shares for P_h, are used for reconstructing $u + a, v + b$ additionally. Then, after receiving $\overline{u + a}, \overline{v + b}$, Sim additionally sets $\mathsf{MACCheck} = 1$ if $\overline{u + a} \neq u + a$ or $\overline{v + b} \neq v + b$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{20}$ and $\mathbf{Hyb}_{19}$ have the same output distribution.

$\mathbf{Hyb}_{21}$: In this hybrid, after obtaining $\overline{u + a}, \overline{v + b}$ for each group of multiplication gates during the verification phase, if $\mathsf{MACCheck} = 1$ or $\delta_w \neq 0$, Sim does not follow the protocol to compute $[w]$. Instead, Sim computes $w = uv + \delta_w$ and then computes P_h's share of $[w]$ based on corrupted parties' shares and the secret. Since δ_w is the pre-computed additive error attached to w, the sharing $[w]$ should be a sharing of the secret $uv + \delta_w$. We only change the way of generating P_h's share of $[w]$ without changing its distribution. Thus, $\mathbf{Hyb}_{21}$ and $\mathbf{Hyb}_{20}$ have the same output distribution.

$\mathbf{Hyb}_{22}$: In this hybrid, after obtaining $\overline{u + a}, \overline{v + b}$ for each group of multiplication gates during the verification phase, if $\mathsf{MACCheck} = \delta_w = 0$, Sim does not follow the protocol to compute P_h's share of $[\tau]$. Instead, Sim additionally follows the protocol to compute the share of $[\tau]$ for each party $P_i \neq P_h$ using the regarded values sent from corrupted parties. Then, P_h's share is computed based on the secret $\tau = 0$ and other parties' shares. Note that in this case, $(\llbracket u \rrbracket, \llbracket v \rrbracket, \llbracket w \rrbracket)$ is a valid triple, and $[\tau] = [uv - w]$ is computed honestly using the other triple $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$, so it must be a sharing of secret 0. Therefore, we only change the way Sim generates P_h's share of $[\tau]$ without changing its distribution. Thus, $\mathbf{Hyb}_{22}$ and $\mathbf{Hyb}_{21}$ have the same output distribution.

$\mathbf{Hyb}_{23}$: In this hybrid, after obtaining $\overline{\tau}$ from P_{king} during the verification phase, if $\tau \neq \overline{\tau}$ (where τ is reconstructed from all parties' shares), Sim additionally sets $\mathsf{MACCheck} = 1$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{22}$ and $\mathbf{Hyb}_{23}$ have the same output distribution.

$\mathbf{Hyb}_{24}$: In this hybrid, at the end of the verification, Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as the execution in the evaluation phase. For the same reason as in $\mathbf{Hyb}_{10}, \mathbf{Hyb}_{11}, \mathbf{Hyb}_{12}$, the distributions of

$\mathbf{Hyb}_{24}$ and $\mathbf{Hyb}_{23}$ are statistically close.

$\mathbf{Hyb}_{25}$: In this hybrid, during the output reconstruction, Sim does not follow the protocol to compute $[\![v]\!]$ for each party's output wire value v of C'. Instead, Sim evaluates the circuit C with the parties' inputs to get v and then computes P_h's share of $[v]$ based on the secret and other parties' shares. Then, v is reconstructed from all parties' shares, and P_h's share of $[\Delta \cdot v]$ is computed from Δ, v together with other parties' shares.

Note that when the protocol is not aborted before the output reconstruction, then it is guaranteed that all authenticated wire values in C' are computed honestly from the multiplication outputs from previous layers. If not, $\mathsf{CompCheck} = 1$. This implies that $\delta_z \neq 0$. Since $\delta_z = \sum_{j=1}^{\ell} \delta_{z^{(j)}}$, there exists one j such that $\delta_{z^{(j)}}$, implying the $\delta_w \neq 0$ for the final triple $([\![u]\!], [\![v]\!], [\![w]\!])$ for checking the j-th group of multiplication gates. In this case, the protocol or simulation must be aborted, which leads to a contradiction.

Therefore, each output v of C' is honestly computed from the circuit inputs, so we only change the way of generating P_h's share of $[v]$ without changing its distribution. Similarly, P_h's share of $[\Delta \cdot v]$ is determined by the secret and other parties' shares, i.e. determined by P_h's share of $[v]$, which means that the distribution of P_j's share of $[\![v]\!]$ remains unchanged. Thus, $\mathbf{Hyb}_{25}$ and $\mathbf{Hyb}_{24}$ have the same output distribution.

$\mathbf{Hyb}_{26}$: In this hybrid, during the output reconstruction, for each output wire value v of C', Sim additionally checks whether the reconstructed value $\overline{v}$ equals v. If not, Sim additionally sets $\mathsf{MACCheck} = 1$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{26}$ and $\mathbf{Hyb}_{25}$ have the same output distribution.

$\mathbf{Hyb}_{27}$: In this hybrid, during the output reconstruction, Sim simulates the execution of $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ as the execution in the evaluation phase. For the same reason as in $\mathbf{Hyb}_{10}, \mathbf{Hyb}_{11}, \mathbf{Hyb}_{12}$, the distributions of $\mathbf{Hyb}_{27}$ and $\mathbf{Hyb}_{26}$ are statistically close.

$\mathbf{Hyb}_{28}$: In this hybrid, when emulating $\mathcal{F}_{\mathsf{prep}}$, Sim delays the generation of the P_h's shares when they are needed. This does not change the output distribution. Thus, $\mathbf{Hyb}_{28}$ and $\mathbf{Hyb}_{27}$ have the same output distribution.

Note that if the protocol does not abort at the end of the execution and Sim does not abort the simulation, the honest party's shares are not used during the interaction with corrupted parties. Furthermore, honest parties' inputs to C' are only used for computing the outputs of C' in this case.

$\mathbf{Hyb}_{29}$: In this hybrid, we change the preparation of the honest party's output when the protocol does not abort at the end of the execution and Sim does not abort the simulation. Sim no longer generates the P_h's output shares when emulating $\mathcal{F}_{\mathsf{prep}}$ and no longer generates honest parties' inputs. Instead, Sim obtains corrupted parties' outputs of C from $\mathcal{F}$ and then uses their inputs to C' (which includes the masks for outputs of C) to compute their outputs of C'. For honest parties' outputs of C', Sim directly samples random values as these outputs. Note that the masks are randomly sampled, so the honest parties' outputs of C' are also random. By the correctness of the construction, $\mathbf{Hyb}_{29}$ and $\mathbf{Hyb}_{28}$ have same output distribution.

$\mathbf{Hyb}_{30}$: In this hybrid, Sim does not sample honest parties' outputs of C' and then use them to compute P_h's shares of them. Instead, Sim directly samples P_h's shares. Since the outputs are random, so are P_h's shares. Therefore, we only change the order of generating v and P_h's share of $[v]$ for each output v of C' for honest parties without changing their distributions. Furthermore, Sim only generates P_h's intermediate shares when they are needed. If the protocol is not aborted, then they are not required to be computed. This also does not affect the output distribution. Thus, $\mathbf{Hyb}_{30}$ and $\mathbf{Hyb}_{30}$ have the same output distribution.

Note that $\mathbf{Hyb}_{30}$ is the ideal-world scenario. Thus, Π_{main} computes $\mathcal{F}$ with statistical security.

C Cost Analysis

We analyze the communication cost of our main protocol as follows. The cost is measured by field elements, and we assume that $m_I, m_O = O(|C|)$. We first analyze the communication cost in the $\{\mathcal{F}_{\mathsf{nVOLE}}, \mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}, \mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}\}$-hybrid model ($\mathcal{F}_{\mathsf{prep}}$ is instantiated with Π_{prep}), without instantiating the functionalities.

- **Preprocessing Phase.**
  - **Random Values:** The preprocessing of random values only involves invocations of $\mathcal{F}_{\mathsf{nVOLE}}$ and local computation, which requires no additional communication.

- **Input Masks:** Similar to the preprocessing of random authenticated sharings, this step requires no communication.
- **Authenticated Triples:** Authenticating $[c], [c']$ during the preparation for each authenticated triple requires $4n$ communication. During the verification of each triple, a value e needs to be opened. Thus, the amortized communication cost is $6n$ per authenticated triple. Since $m_A = \ell = O(|C|/k)$, $O(|C|n/k)$ total communication cost is required.
- **Tensor Products:** Only the verification of length-k tensor products requires communication in this step. In particular, to verify each length-k tensor product, the servers need to run Π_{Open} on 2 elements, which requires $4n$ communication. Since $O(|C|/k)$ length-k tensor products are required, the communication cost of this step is $O(|C|n/k)$.

- **Evaluation Phase.**
  1. **Authenticating Inputs:** The servers need to send two additive sharings to a client for each input wire of C' attached to this client, and the client will send an element back to all the servers. This requires $3n$ communication in total. Since the number of input wires in C' is $m_I + m_O$, the communication cost of this step is $3(m_I + m_O)n$.
  2. **Evaluating the Circuit:** The communication cost during the evaluation of the circuit is the same as that of the semi-honest SPDZ, which is $4|C|n$. Then, authenticating the output values of C' costs communication of another $2m_O n$ field elements.
  3. **MAC Check:** This step only involves invocations of $\mathcal{F}_{\mathsf{Coin}}$ and $\mathcal{F}_{\mathsf{Commit}}$.

- **Verification Phase.**
  1. **Reducing to Inner-Product Check:** This reduction only involves an invocation of $\mathcal{F}_{\mathsf{Coin}}$ and local computations, which do not require any additional communication.
  2. **Separate into Blocks:** This step requires authentication of the sharings $[z^{(1)}], \ldots, [z^{(\ell-1)}]$, which requires $2(\ell-1)n = O(|C|n/k)$ communication cost.
  3. **Recursive Check:** During each reduction in vector length, the sharings $[w_1], \ldots, [w_{\lambda-1}]$ and $[h(\lambda+1), \ldots, h(2\lambda-1)]$ needs to be authenticated, which requires $2(\lambda-1)$ authentication steps. Since the reduction needs to be performed $\ell \cdot \log_\lambda k$ times, there are in total $2\ell(\lambda-1) \cdot \log_\lambda k$ authentication steps, which requires communication of $4n\ell(\lambda-1) \cdot \log_\lambda k = O(|C|n\lambda k^{-1} \cdot \log_\lambda k)$ field elements.
  4. **Verify the Final Triple.** During the check of each final triple, three values need to be opened via Π_{Open}, which requires $6n$ communication cost. Thus, the total communication cost of this step is $6\ell n = O(|C|n/k)$.

- **Output Reconstruction.** For each output wire of C', the parties need to publicly reconstruct the wire value, which requires $2n$ communication. Since the number of output wires in C' is the same as the number of output wires in C. The communication cost of this step is $2m_O$.

The instantiations of $\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Commit}}$ require $\mathsf{poly}(n, \kappa)$ communication. Therefore, regardless of the communication cost from PCGs, the total communication cost is $4|C|n + 5m_I n + 7m_O n + O(|C|n\lambda k^{-1} \cdot \log_\lambda k) + \mathsf{poly}(n, \kappa))$ field elements.

Regardless of the cost of PCGs, the computational cost during the preprocessing phase mainly comes from the preparation of $O(|C|/k)$ length-k tensor products, which requires $O(|C|nk)$ field operations. During the evaluation phase and the output reconstruction, the computational cost is clearly $O(|C|n)$. During the verification, the computational cost mainly comes from the computation of $[\boldsymbol{x}^{(j)} \otimes \boldsymbol{y}^{(j)}]$ and the recursive check. The former requires $O(|C|nk)$ field operations, and the latter requires $O(n\ell'^2\lambda^2)$ for each reduction in vector length, resulting in a total computational cost of $O(|C|nk)$ field operations. Thus, the total computational cost (beyond PCG) is $O(|C|nk)$.

D The Protocol Description of $\Pi_{\mathsf{turboprep}}$

We provide the protocol $\Pi_{\mathsf{turboprep}}$ here in Figure 28.

Protocol $\Pi_{\mathsf{turboprep}}$

Each party P_i samples a random element in $\mathbb{F}$ as his share Δ_i of $[\Delta]$. Then, P_i sends $(\mathsf{Init}, \Delta_i)$ to $\mathcal{F}_{\mathsf{nVOLE}}$. The parties run Π_{rand} and Π_{auth}, and then:

1. Each party P_i calls $\mathcal{F}_{\mathsf{nVOLE}}$ with input (Extend, m') with $m' = I_1 + \cdots + I_n + m_C + 2$. P_i receives the seeds $s_w^{(i)}$ and obtains shares of $\langle\langle \boldsymbol{w} \rangle\rangle$, where $s_w^{(i)}$ can be expanded to P_i's shares of $[\boldsymbol{w}]$ with $\boldsymbol{w} = (w_1, \ldots, w_m)$ plus two random sharings $[x'], [y']$, where $m = I_1 + \cdots + I_n + m_C$. The parties then locally convert $\langle\langle \boldsymbol{w} \rangle\rangle$ to $[\![\boldsymbol{w}]\!]$. For each $j = 1, \ldots, n$ and $i = I_1 + \cdots + I_{j-1} + \alpha$ for $\alpha = 1, \ldots, I_j$, the parties also extract $\langle\langle w_i \rangle\rangle$ from $\langle\langle \boldsymbol{w} \rangle\rangle$ and locally convert it to $[\Delta_j \cdot w_i]$ (and similar for $[\![x']\!], [\![y']\!]$).
2. Each (ordered) pair of parties (P_i, P_j) calls $\mathcal{F}_{\mathsf{tensor}}^{\mathsf{prog}}$ with P_i sending $s_w^{(i)}$ and P_j sending $s_w^{(j)}$. It sends $\boldsymbol{u}_{i,j}$ back to P_i and $\boldsymbol{v}_{i,j}$ to P_j.
3. Let $\boldsymbol{w}' = (\boldsymbol{w}, x', y')$. Each party P_i computes $[\boldsymbol{w}' \otimes \boldsymbol{w}'] = \boldsymbol{w}^{(i)'} \otimes \boldsymbol{w}^{(i)'} + \sum_{j \neq i} (\boldsymbol{u}_{i,j} + \boldsymbol{v}_{j,i})$, where $\boldsymbol{w}^{(i)'}$ is P_i's share of $[\boldsymbol{w}']$. Then, the parties extract $[w_i \cdot w_j]$ for each $(i, j) \in \{1, \ldots, m\}^2$ from $[\boldsymbol{w}' \otimes \boldsymbol{w}']$ and set $[w_{i,j}] = [w_i \cdot w_j]$. The parties also extract $[x' \cdot y']$ and set it to $[v']$.
4. *Verification:* Let PRG be a public pseudorandom generator with seed space $\mathbb{F}^*$ and image space $(\mathbb{F}^*)^{2m}$. To verify $\{([\![w_i]\!], [\![w_i]\!])\}_{i=1,\ldots,m}, \{[w_{i,j}]\}_{i,j \in \{1,\ldots,m\}}$:
   (a) The parties send $\mathsf{RandCoin}$ to $\mathcal{F}_{\mathsf{Coin}}$ and receive a random element $r \in \mathbb{F}^*$ from it. Then, the parties locally compute
   $$\mathsf{PRG}(r) \to (r_a^{(1)}, \ldots, r_a^{(k)}, r_b^{(1)}, \ldots, r_b^{(k)})^{2m}.$$
   (b) The parties locally compute $[\![x]\!] = \sum_{i=1}^m r_a^{(i)} [\![w_i]\!]$ and $[\![y]\!] = \sum_{i=1}^m r_b^{(i)} [\![w_i]\!]$. Then, the parties locally compute
   $$[v] = \sum_{i=1}^m \sum_{j=1}^m r_a^{(i)} r_b^{(j)} [w_{i,j}].$$
   (c) The parties locally compute $[\![x + x']\!], [\![y + y']\!]$ and run Π_{Open} on them. Then, the parties locally compute
   $$[\tau] = (x + x') \cdot (y + y') - (x + x') \cdot [y'] - (y + y') \cdot [x'] + [v'] - [v].$$
   After computing $[\tau]$, the parties run $\Pi_{\mathsf{SPDZ\text{-}MAC}}$ on all the opened values during the verification, i.e., $x + x', y + y'$. Finally, each party P_i calls $\mathcal{F}_{\mathsf{Commit}}$ with input $(\mathsf{commit}, P_i, \tau_i, \mathsf{mid}_{\tau_i})$, where τ_i is P_i's share of $[\tau]$. The parties then open their commitments via sending $(\mathsf{open}, P_i, \mathsf{mid}_{\tau_i})$ for each party P_i and check that $\sum_{i=1}^n \tau_i = 0$. If not, the parties abort the protocol.

Figure 28: Protocol for the preprocessing phase of Π_{turbo}.

E The Verification Phase of Π_{turbo}

We present the verification phase Π_{turbover} of Π_{turbo} as follows in Figure 29.

Protocol Π_{turbover}

1. **Reducing to Inner-Product Check.** Denote all pairs of input wire values for multiplication gates in $|C|$ by $(x'_1, y'_1), \ldots, (x'_{|C|}, y'_{|C|})$ and all the output wire values of multiplication gates by $z'_1, \ldots, z'_{|C|}$, where each $x'_i = x_{i+m_I+m_O}$ and similar for y_i, z_i. Each z'_i is computed by $z'_i = x'_i \cdot y'_i$. Then:
   (a) The parties send $\mathsf{RandCoin}$ to $\mathcal{F}_{\mathsf{Coin}}$ and obtain a random element $s \in \mathbb{F}^*$.

(b) The parties locally compute

$$[\![\boldsymbol{x}]\!] = \left(\sum_{i=1}^{|C|} s^i [\![x'_1]\!], \ldots, \sum_{i=1}^{|C|} s^i [\![x'_{|C|}]\!]\right), \quad [\![\boldsymbol{y}]\!] = ([\![y'_1]\!], \ldots, [\![y'_{|C|}]\!])$$

and

$$[\![z]\!] = \sum_{i=1}^{|C|} s^i [\![z'_i]\!].$$

With high probability, the coefficient $\sum_{i=1}^{|C|} s^i$ corresponding to each $[\![x'_j]\!] = [\![x_{j+m_I+m_O}]\!]$ in $[\![\boldsymbol{x}]\!]$ is non-zero for each $j = 1, \ldots, |C|$. If not, the parties abort the protocol.

2. **Perform the Inner-Product Check.** Assume that $x_i = \sum_{j=1}^{i-1} \beta_{j,i} \cdot z_j, y_i = \sum_{j=1}^{i-1} \gamma_{j,i} \cdot z_j$. The parties set $[\![a_i]\!] = \sum_{j=1}^{i_\alpha - 1} \beta_{j,i} \cdot [\![w_j]\!], [\![b_\alpha]\!] = \sum_{j=1}^{i_\alpha - 1} \gamma_{j,i} \cdot [\![w_j]\!]$. Then, the parties locally compute $[a_\alpha \cdot b_\beta] = \sum_{j_1=1}^{i-1} \sum_{j_2=1}^{i-1} \beta_{j,i}\gamma_{j,\beta}[w_{i,i}]$. The $|C|$ multiplication gates are regarded as evaluated using the length-$|C|$ partially authenticated tensor product $\{[\![a_\alpha]\!], [\![b_\alpha]\!]\}_{\alpha=m_I+m_O+1,\ldots,m_I+m_O+|C|}$, $\{[a_\alpha \cdot b_\beta]\}_{\alpha,\beta\in\{m_I+m_O+1,\ldots,m_I+m_O+|C|\}}$. Then, the parties run the second and the third steps of protocol Π_{IPcheck} (Figure 17) to verify that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = z$, where $k = |C|$ and $\ell = 1$.

Figure 29: Protocol for the verification phase of Π_{turbo}.

F Proof Sketch for the Security of Π_{turbo}

The main simulation strategy in proving the security of Π_{turbo} is similar to that of Π_{main}, so we focus on the difference between the two protocols, i.e.:

- The additional preprocessing of $[\![\boldsymbol{w}]\!], [\boldsymbol{w} \otimes \boldsymbol{w}]$.
- The online evaluation phase.

For the additional preprocessing, the only difference from the preprocessing of tensor products in Π_{tens} is that $\boldsymbol{a} = \boldsymbol{b} = \boldsymbol{w}$ during the additional preprocessing. Note that during the security proof of Π_{prep}, we do not really rely on the pseudorandomness of $\boldsymbol{b}$ to simulate the verification. Specifically, the honest party P_h's shares of $[x + x'], [y + y']$ can be sampled randomly by the simulator, which only relies on the fact that x', y' are independent pseudorandom elements. The commit-and-open process of $[\tau]$ and MAC check can be simulated in the same way as simulating Π_{tens}. Since the additional preprocessing process does not require any other communication, the protocol is computationally private.

For the correctness of $[\boldsymbol{w} \otimes \boldsymbol{w}]$, the attached error term to each $[w_i \cdot w_j]$ is linear to P_h's shares $w_i^{(h)}, w_j^{(h)}$ of $[w_i], [w_j]$, where the linear coefficients are determined before the pseudorandom vectors $\boldsymbol{r}_a = (r_a^{(1)}, \ldots, r_a^{(k)}), \boldsymbol{r}_b = (r_b^{(1)}, \ldots, r_b^{(k)})$ are obtained. Therefore, the total error term on τ contains $\langle \boldsymbol{r}_a \otimes \boldsymbol{r}_b, (\boldsymbol{w}^{(h)} \otimes \boldsymbol{\delta}_w + \boldsymbol{\delta}'_w \otimes \boldsymbol{w}^{(h)}) \rangle$ as we have analyzed in the proof of Π_{prep}, where $\boldsymbol{w}^{(h)}$ is P_h's share of $[\boldsymbol{w}]$, and $\boldsymbol{\delta}_w, \boldsymbol{\delta}'_w$ are vectors chosen by the adversary. From the pseudorandomness of $\boldsymbol{w}^{(h)}$, this error term is also pseudorandom if either of $\boldsymbol{\delta}_w, \boldsymbol{\delta}'_w$ is a non-zero vector. This guarantees that the existence of linear errors can be detected with an overwhelming probability.

For the security of online evaluation, only $z_i + w_i$ for $i = 1, \ldots, m$ are opened to the parties. Since each w_i is sampled randomly and independently by $\mathcal{F}_{\mathsf{prep}}$, all the communicated values during the evaluation phase are random. The simulator only needs to sample the honest party P_h's share of each $[z_i + w_i]$ randomly an sends it to P_{king}. Since we do not modify the verification framework, it can be simulated as in the simulation for Π_{main} with additional local computations provided in Π_{turbover}. The correctness of the evaluation can be guaranteed by the verification. Due to the above reasons, the statistical security also holds.